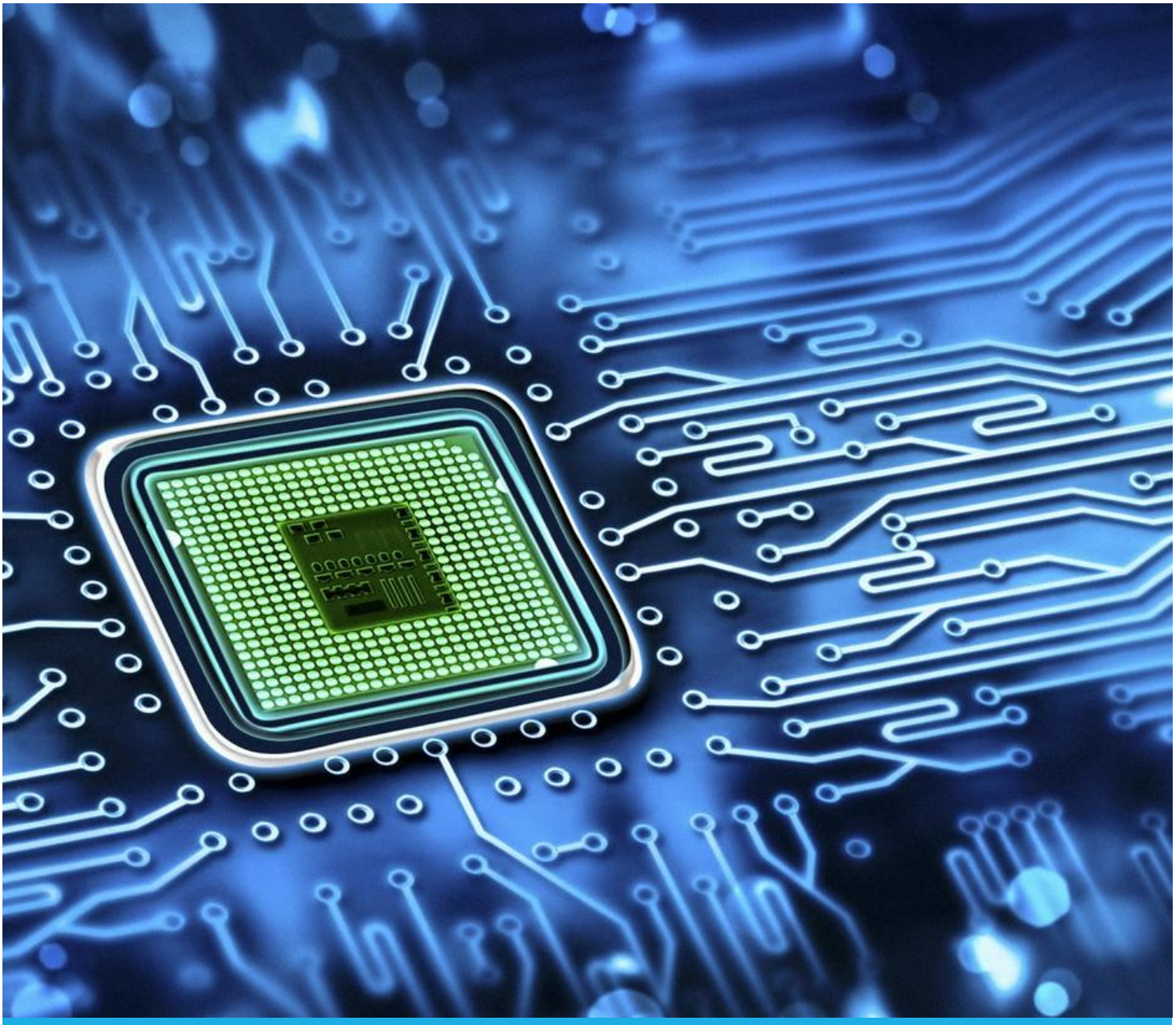

300 Beginner-Level Embedded Firmware Projects

Aschref Ben Thabet

20 September 2025

Learning Series from scratch to expert

Embedded Firmware Problems and Projects

Contents

Basic C Programming for Embedded Systems

Project 1: Write a C program to toggle an LED connected to a GPIO pin every 1 second.

- **Project Description:** Develop a simple embedded C program that toggles the state of an LED connected to a GPIO pin at regular intervals of 1 second.
- This is a fundamental task to understand GPIO manipulation and timing in embedded systems.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Infinite loop with software delay.
- **Steps**:
    1. Include necessary headers for GPIO and delay functions (simulated here for portability).
    2. Define the GPIO pin for the LED.
    3. Initialize the GPIO pin as output.
    4. Enter an infinite while loop.
    5. Inside the loop, set the LED pin high (ON), delay for 1 second, then set it low (OFF), and delay again for 1 second.
    6. Repeat indefinitely.

C Code Implementation

```c
#include <stdio.h>  // For simulation; replace with MCU-specific headers like <avr/io.h>
#define LED_PIN 0   // Example GPIO pin (e.g., PB0 in AVR)

// Simulated delay (in real systems, use timer interrupts or _delay_ms from <util/delay.h>)
void delay_1sec(void) {
    volatile unsigned long i;
    for (i = 0; i < 1000000UL; i++);  // Approximate 1s on typical MCU clock
}

int main(void) {
    // Initialize GPIO (simulated output)
    // In real: DDRB |= (1 << LED_PIN);  // Set as output
    printf("LED toggle started on pin %d.\n", LED_PIN);

    while (1) {
        // Turn LED ON (simulated)
        // In real: PORTB |= (1 << LED_PIN);
        printf("LED ON\n");
        delay_1sec();

        // Turn LED OFF (simulated)
        // In real: PORTB &= ~(1 << LED_PIN);
        printf("LED OFF\n");
        delay_1sec();
    }
    return 0;  // Never reached
}
```

Expert Tips

- Avoid busy-wait delays in production code as they waste CPU cycles and power; use hardware timers (e.g., Timer0 in AVR) for accurate, low-power timing.
- Ensure the delay loop is calibrated for your MCU's clock speed (e.g., 16MHz for Arduino).
- Add pull-up resistors if needed for stable GPIO levels, and test with an oscilloscope for precise timing.

Project 2: Create a program to read the state of a push button and print it to a serial console.

Project Description: Implement a C program for an embedded system that continuously reads the state of a push button connected to a GPIO input pin and outputs the state (pressed or released) to a serial console for monitoring.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling-based input reading in a loop.
- **Steps**:
    1. Define the input pin for the button.
    2. Initialize the GPIO pin as input (with pull-up if necessary).
    3. In an infinite loop, read the pin state (high/low).
    4. Map the state to a string (PRESSED/RELEASED) and print via serial.
    5. Add a short delay to avoid flooding the console and allow for button debouncing.

C Code Implementation

```c
#include <stdio.h>  // For printf; in real, use UART functions
#define BUTTON_PIN 1  // Example input pin (e.g., PB1 in AVR)

// Simulated button read (in real: return (PINB & (1 << BUTTON_PIN)) ? 1 : 0)
int read_button(void) {
    return (rand() % 2);  // Simulate button state (0 or 1)
}

int main(void) {
    // Initialize input pin (simulated)
    // In real: DDRB &= ~(1 << BUTTON_PIN);  // Set as input
    // In real: PORTB |= (1 << BUTTON_PIN);  // Enable pull-up
    printf("Button monitoring started on pin %d.\n", BUTTON_PIN);

    while (1) {
        int state = read_button();
        printf("Button state: %s\n", state ? "PRESSED" : "RELEASED");
        for (volatile int i = 0; i < 500000; i++);  // Short delay (~0.5s)
    }
    return 0;  // Never reached
}
```

Expert Tips

- Implement software or hardware debouncing (e.g., 10ms delay or capacitor) to handle button bounce.
- Use UART interrupts for efficient serial communication instead of polling.
- Declare GPIO input variables as volatile to prevent compiler optimizations from skipping reads.

Project 3: Implement a program to count from 0 to 255 and display the value on an 8-bit LED port.

Project Description: Write a C program that counts from 0 to 255 and displays each value on an 8-bit LED port, simulating a binary counter visible on LEDs.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Incremental loop.
- **Steps**:
    1. Initialize the 8-bit port as output.
    2. Use a for loop to iterate from 0 to 255.
    3. Set the port value to the current count.
    4. Add a delay to make the count visible on LEDs.
    5. Repeat indefinitely.

C Code Implementation

```c
#include <stdio.h>  // For simulation
#define LED_PORT 3  // Simulated 8-bit port

void delay(void) {
    volatile unsigned long i;
    for (i = 0; i < 100000; i++);  // Short delay for visibility
}

int main(void) {
    // Initialize port (simulated)
    // In real: DDRB = 0xFF;  // Set all as output
    printf("Counting on 8-bit LED port %d.\n", LED_PORT);

    while (1) {
        for (unsigned char count = 0; count <= 255; count++) {
            // Set port value (simulated)
            // In real: PORTB = count;
            printf("Count: %d (0x%02X)\n", count, count);
            delay();
        }
    }
    return 0;  // Never reached
}
```

Expert Tips

- Use unsigned char for 8-bit variables to avoid overflow issues.
- In real systems, connect LEDs to a port like PORTB on AVR and verify with a logic analyzer.
- Optimize by using timer interrupts to update the count without blocking.

Project 4: Write a program to blink an LED 5 times when a button is pressed.

Project Description: Create a C program that detects a button press and responds by blinking an LED 5 times (0.5s ON, 0.5s OFF per blink).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Event-driven loop with edge detection.
- **Steps**:
    1. Initialize button as input and LED as output.
    2. Monitor button state for a press (edge detection).
    3. On press, enter a loop to blink the LED 5 times.
    4. Return to monitoring after blinking.

C Code Implementation

```c
#include <stdio.h>
#define LED_PIN 1
#define BUTTON_PIN 2
int prev_state = 0;  // For edge detection

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_button(void) {
    return (rand() % 2);  // Simulate button
}

int main(void) {
    // Initialize pins (simulated)
    // In real: DDRB |= (1 << LED_PIN); DDRB &= ~(1 << BUTTON_PIN); PORTB |= (1 << BUTTON_PIN);
    printf("Waiting for button press on pin %d.\n", BUTTON_PIN);

    while (1) {
        int state = read_button();
        if (state && !prev_state) {  // Rising edge
            for (int i = 0; i < 5; i++) {
                printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
                delay_ms(500);
                printf("LED OFF\n");  // In real: PORTB &= ~(1 << LED_PIN);
                delay_ms(500);
            }
        }
        prev_state = state;
        delay_ms(10);  // Debounce delay
    }
    return 0;
}
```

Expert Tips

- Use edge detection (previous vs. current state) to avoid multiple triggers on a single press.
- Implement hardware debouncing with a capacitor or software debouncing with a 10-20ms delay.
- Consider interrupt-driven button detection for responsiveness.

Project 5: Create a program to toggle an LED based on a boolean variable in a loop.

Project Description: Write a C program that toggles an LED on a GPIO pin based on the value of a boolean variable, updated in a loop, with a visible delay.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: State-based toggle with delay.
- **Steps**:
    1. Define a boolean variable to control LED state.
    2. Initialize LED pin as output.
    3. In a loop, toggle the boolean and update the LED state.
    4. Add delay for visibility.

C Code Implementation

```
#include <stdio.h>
#define LED_PIN 1
int led_state = 0;  // Boolean state (0 = OFF, 1 = ON)

void delay_1sec(void) {
    volatile unsigned long i;
    for (i = 0; i < 1000000UL; i++);
}

int main(void) {
    // Initialize LED (simulated)
    // In real: DDRB |= (1 << LED_PIN);
    printf("Toggling LED on pin %d.\n", LED_PIN);

    while (1) {
        led_state = !led_state;  // Toggle boolean
        if (led_state) {
            printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
        } else {
            printf("LED OFF\n");  // In real: PORTB &= ~(1 << LED_PIN);
        }
        delay_1sec();
    }
    return 0;
}
```

Expert Tips

- Use bool type from <stdbool.h> for clarity in modern C.
- For low-power systems, toggle the LED in an interrupt service routine (ISR) driven by a timer.
- Ensure the GPIO pin is properly configured to avoid floating states.

Project 6: Implement a program to set a specific bit in a variable using bit manipulation.

Project Description: Write a C program to set (i.e., make 1) a specific bit in an 8-bit variable using bitwise operations and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise OR with a mask.
- **Steps**:
    1. Define an 8-bit variable and the bit position to set (0-7).

2. Create a mask with 1 at the desired bit position (1 << position).
3. Use bitwise OR to set the bit in the variable.
4. Print the result in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char var = 0x00;  // Initial value (00000000)
    int bit_position = 3;      // Set bit 3 (0-based)

    // Set bit using OR
    var |= (1 << bit_position);

    printf("After setting bit %d:\n", bit_position);
    print_binary(var);  // Expected: 00001000
    return 0;
}
```

Expert Tips

- Validate bit_position (0-7 for 8-bit) to prevent undefined behavior.
- Use uint8_t from <stdint.h> for portable 8-bit variables.
- For MCU registers, apply this directly to PORTx or DDRx.

Project 7: Write a program to clear a specific bit in a variable using bit manipulation.

Project Description: Write a C program to clear (i.e., make 0) a specific bit in an 8-bit variable using bitwise operations and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise AND with an inverted mask.
- **Steps**:
    1. Define an 8-bit variable and the bit position to clear.
    2. Create a mask with 1 at the bit position (1 << position).
    3. Invert the mask (~mask) and use bitwise AND to clear the bit.
    4. Print the result in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char var = 0xFF;  // Initial value (11111111)
    int bit_position = 3;      // Clear bit 3

    // Clear bit using AND with inverted mask
    var &= ~(1 << bit_position);

    printf("After clearing bit %d:\n", bit_position);
    print_binary(var);  // Expected: 11110111
    return 0;
}
```

Expert Tips

- Ensure the bit position is valid to avoid shifting errors.
- This technique is useful for clearing GPIO pins or flags in registers.
- Test with different initial values to verify correctness.

Project 8: Create a program to check if a specific bit is set in a variable.

Project Description: Write a C program to check if a specific bit in an 8-bit variable is set (1) and print the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise AND with a mask.
- **Steps**:
    1. Define an 8-bit variable and the bit position to check.
    2. Create a mask with 1 at the bit position (1 << position).
    3. Use bitwise AND to test the bit.
    4. Print whether the bit is set or not.

C Code Implementation

```c
#include <stdio.h>

int main(void) {
    unsigned char var = 0x08;  // 00001000 (bit 3 set)
    int bit_position = 3;

    // Check if bit is set
    if (var & (1 << bit_position)) {
        printf("Bit %d is SET.\n", bit_position);
    } else {
        printf("Bit %d is NOT SET.\n", bit_position);
    }
    return 0;
}
```

Expert Tips

- Use this for reading status flags in MCU registers (e.g., TIFR for timer interrupts).
- Combine with if conditions for control logic.
- Avoid magic numbers; define bit positions as constants or macros.

Project 9: Implement a program to toggle a specific bit in a variable.

Project Description: Write a C program to toggle (flip) a specific bit in an 8-bit variable using bitwise operations and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise XOR with a mask.
- **Steps**:
    1. Define an 8-bit variable and the bit position to toggle.
    2. Create a mask with 1 at the bit position (1 << position).
    3. Use bitwise XOR to toggle the bit.
    4. Print the result in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char var = 0x00;  // 00000000
    int bit_position = 3;

    // Toggle bit using XOR
    var ^= (1 << bit_position);

    printf("After toggling bit %d:\n", bit_position);
    print_binary(var);  // Expected: 00001000
    return 0;
}
```

Expert Tips

- XOR is ideal for toggling; use for GPIO pin state changes.
- Verify the mask to avoid affecting other bits.
- Useful for implementing blinking without tracking state.

Project 10: Write a program to perform a left shift on a variable and display the result.

Project Description: Write a C program to perform a left shift operation on an 8-bit variable by a specified number of positions and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise left shift.
- **Steps**:
    1. Define an 8-bit variable and shift amount.
    2. Perform left shift using << operator.
    3. Print the original and shifted values in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char var = 0x0A;  // 00001010
    int shift = 2;

    printf("Before left shift by %d:\n", shift);
    print_binary(var);

    var <<= shift;  // Shift left
    printf("After left shift by %d:\n", shift);
    print_binary(var);  // Expected: 00101000
    return 0;
}
```

Expert Tips

- Left shift multiplies by 2 per position (e.g., x << 2 is x * 4).
- Check for overflow; bits shifted out are lost in 8-bit variables.
- Useful for scaling values or preparing data for shift registers.

Project 11: Create a program to perform a right shift on a variable and display the result.

Project Description: Write a C program to perform a right shift operation on an 8-bit variable by a specified number of positions and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise right shift.
- **Steps**:
    1. Define an 8-bit variable and shift amount.
    2. Perform right shift using >> operator.
    3. Print the original and shifted values in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char var = 0x28;  // 00101000
    int shift = 2;

    printf("Before right shift by %d:\n", shift);
    print_binary(var);

    var >>= shift;  // Shift right
    printf("After right shift by %d:\n", shift);
    print_binary(var);  // Expected: 00001010
    return 0;
}
```

Expert Tips

- Right shift divides by 2 per position (e.g., x >> 2 is x / 4).
- For signed integers, beware of sign extension; use unsigned to avoid it.
- Useful for extracting bits or scaling down values.

Project 12: Implement a program to perform a bitwise AND operation on two variables.

Project Description: Write a C program to perform a bitwise AND operation on two 8-bit variables and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise AND.
- **Steps**:
    1. Define two 8-bit variables.
    2. Perform bitwise AND using &.
    3. Print the inputs and result in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}
```

```c
int main(void) {
    unsigned char var1 = 0x0F;  // 00001111
    unsigned char var2 = 0xA5;  // 10100101

    printf("Var1: ");
    print_binary(var1);
    printf("Var2: ");
    print_binary(var2);

    unsigned char result = var1 & var2;  // Bitwise AND
    printf("Result (AND): ");
    print_binary(result);  // Expected: 00000101
    return 0;
}
```

Expert Tips

- Bitwise AND is useful for masking bits (e.g., selecting specific GPIO pins).
- Ensure variables are unsigned to avoid sign issues.
- Test with various input combinations to verify logic.

Project 13: Write a program to perform a bitwise OR operation on two variables.

Project Description: Write a C program to perform a bitwise OR operation on two 8-bit variables and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise OR.
- **Steps**:
    1. Define two 8-bit variables.
    2. Perform bitwise OR using |.
    3. Print the inputs and result in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char var1 = 0x0F;  // 00001111
    unsigned char var2 = 0xA5;  // 10100101

    printf("Var1: ");
    print_binary(var1);
    printf("Var2: ");
    print_binary(var2);

    unsigned char result = var1 | var2;  // Bitwise OR
    printf("Result (OR): ");
    print_binary(result);  // Expected: 10101111
    return 0;
}
```

Expert Tips

- Use OR to combine flags or set multiple GPIO pins at once.
- Verify the mask to ensure only desired bits are affected.
- Useful for configuring multiple settings in a single register write.

Project 14: Create a program to perform a bitwise XOR operation on two variables.

Project Description: Write a C program to perform a bitwise XOR operation on two 8-bit variables and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise XOR.
- **Steps**:
    1. Define two 8-bit variables.
    2. Perform bitwise XOR using ^.
    3. Print the inputs and result in binary.

C Code Implementation

```
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char var1 = 0x0F;  // 00001111
    unsigned char var2 = 0xA5;  // 10100101

    printf("Var1: ");
    print_binary(var1);
    printf("Var2: ");
    print_binary(var2);

    unsigned char result = var1 ^ var2;  // Bitwise XOR
    printf("Result (XOR): ");
    print_binary(result);  // Expected: 10101010
    return 0;
}
```

Expert Tips

- XOR is useful for toggling bits or detecting differences between states.
- Can be used for simple encryption or checksum calculations.
- Test edge cases (e.g., all 0s, all 1s) to ensure correctness.

Project 15: Implement a program to invert all bits in a variable using bitwise NOT.

Project Description: Write a C program to invert (flip) all bits in an 8-bit variable using the bitwise NOT operator and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise NOT.
- **Steps**:
    1. Define an 8-bit variable.
    2. Apply bitwise NOT using ~.
    3. Print the original and inverted values in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char var = 0xA5;  // 10100101

    printf("Original: ");
    print_binary(var);

    var = ~var;  // Bitwise NOT
    printf("Inverted: ");
    print_binary(var);  // Expected: 01011010
    return 0;
}
```

Expert Tips

- Bitwise NOT inverts all bits; for 8-bit, ~0xA5 = 0x5A.
- Use uint8_t for explicit 8-bit operations.
- Useful for inverting GPIO states or flags in registers.

Project 16: Write a program to convert a decimal number to binary using bit manipulation.

Project Description: Write a C program to convert an 8-bit decimal number to its binary representation using bit manipulation and print the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bit extraction with right shift and masking.
- **Steps**:
    1. Define an 8-bit decimal number.
    2. Iterate through each bit (7 to 0) using right shift and AND with 1.

3. Print each bit to form the binary number.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char num = 42;  // Decimal 42 (00101010)

    printf("Decimal %d in binary: ", num);
    print_binary(num);
    return 0;
}
```

Expert Tips

- Use a loop to avoid hardcoding bit positions.
- For larger numbers, adjust the loop to handle more bits.
- Useful for debugging or displaying register values.

Project 17: Create a program to calculate the sum of two 8-bit numbers and display the result.

Project Description: Write a C program to add two 8-bit numbers and display their sum, handling potential overflow.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Arithmetic addition.
- **Steps**:
    1. Define two 8-bit numbers.
    2. Add them using +.
    3. Check for overflow (sum > 255).
    4. Print the result.

C Code Implementation

```c
#include <stdio.h>

int main(void) {
    unsigned char num1 = 150;
    unsigned char num2 = 100;

    unsigned int sum = num1 + num2;  // Use int to detect overflow

 if (sum > 255) {
        printf("Sum of %d and %d = %d (Overflow occurred!)\n", num1, num2, sum);
    } else {
        printf("Sum of %d and %d = %d\n", num1, num2, sum);
    }
```

```
    return 0;
}
```

Expert Tips

- Use uint8_t for inputs and uint16_t for sum to catch overflow.
- In embedded systems, handle overflow explicitly (e.g., set a flag).
- Test boundary cases (e.g., 255 + 1).

Project 18: Implement a program to swap two variables without using a temporary variable.

Project Description: Write a C program to swap the values of two 8-bit variables using bitwise operations without a temporary variable.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise XOR swap.
- **Steps**:
    1. Define two 8-bit variables.
    2. Use XOR operations: a = a ^ b; b = a ^ b; a = a ^ b.
    3. Print the swapped values.

C Code Implementation

```
#include <stdio.h>

int main(void) {
    unsigned char a = 10;  // 00001010
    unsigned char b = 20;  // 00010100

    printf("Before swap: a = %d, b = %d\n", a, b);

    a = a ^ b;
    b = a ^ b;
    a = a ^ b;

    printf("After swap: a = %d, b = %d\n", a, b);
    return 0;
}
```

Expert Tips

- XOR swap is compact but can be less readable; consider using a temp variable for clarity.
- Ensure no aliasing (a and b must be distinct variables).
- Not recommended for modern compilers due to optimization complexities.

Project 19: Write a program to check if a number is even or odd using bit manipulation.

Project Description: Write a C program to determine if an 8-bit number is even or odd using bitwise operations.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise AND with LSB.
- **Steps**:
    1. Define an 8-bit number.
    2. Check the least significant bit (LSB) using num & 1.
    3. If LSB is 1, the number is odd; if 0, even.
    4. Print the result.

C Code Implementation

```c
#include <stdio.h>

int main(void) {
    unsigned char num = 42;  // Example number

    if (num & 1) {
        printf("%d is ODD\n", num);
    } else {
        printf("%d is EVEN\n", num);
    }
    return 0;
}
```

Expert Tips

- Checking LSB is faster than modulo (% 2).
- Works for any integer size, not just 8-bit.
- Useful for quick parity checks in control logic.

Project 20: Create a program to multiply a number by 2 using a left shift operation.

Project Description: Write a C program to multiply an 8-bit number by 2 using a left shift operation and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise left shift.
- **Steps**:
    1. Define an 8-bit number.
    2. Perform left shift by 1 (num << 1).
    3. Check for overflow (result > 255).
    4. Print the result.

C Code Implementation

```c
#include <stdio.h>

int main(void) {
    unsigned char num = 50;
    unsigned int result = num << 1;  // Multiply by 2

    if (result > 255) {
        printf("%d * 2 = %d (Overflow!)\n", num, result);
    }
```

```
else {
        printf("%d * 2 = %d\n", num, result);
    }
    return 0;
}
```

Expert Tips

- Left shift by 1 is equivalent to multiplying by 2; generalizes to num << n for 2^n.
- Always check for overflow in constrained systems.
- Faster than arithmetic multiplication on some MCUs.

Project 21: Implement a program to divide a number by 2 using a right shift operation.

Project Description: Write a C program to divide an 8-bit number by 2 using a right shift operation and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise right shift.
- **Steps**:
    1. Define an 8-bit number.
    2. Perform right shift by 1 (num >> 1).
    3. Print the result (integer division, truncates).

C Code Implementation

```
#include <stdio.h>

int main(void) {
    unsigned char num = 100;
    unsigned char result = num >> 1;  // Divide by 2

    printf("%d / 2 = %d\n", num, result);
    return 0;
}
```

Expert Tips

- Right shift by 1 is equivalent to integer division by 2.
- Use unsigned to avoid sign extension issues.
- Faster than division on resource-constrained MCUs.

Project 22: Write a program to set multiple bits in a variable using a mask.

Project Description: Write a C program to set multiple specified bits in an 8-bit variable using a bitwise mask and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise OR with a mask.

- **Steps**:
    1. Define an 8-bit variable and a mask with 1s at desired bit positions.
    2. Use bitwise OR to set the bits.
    3. Print the result in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char var = 0x00;      // 00000000
    unsigned char mask = 0x0A;     // 00001010 (set bits 1 and 3)

    var |= mask;  // Set multiple bits
    printf("After setting bits with mask 0x%02X:\n", mask);
    print_binary(var);  // Expected: 00001010
    return 0;
}
```

Expert Tips

- Define masks as constants for readability and reuse.
- Useful for setting multiple GPIO pins or flags simultaneously.
- Verify the mask to avoid unintended bit changes.

Project 23: Create a program to clear multiple bits in a variable using a mask.

Project Description: Write a C program to clear multiple specified bits in an 8-bit variable using a bitwise mask and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise AND with inverted mask.
- **Steps**:
    1. Define an 8-bit variable and a mask with 1s at bits to clear.
    2. Invert the mask and use bitwise AND.
    3. Print the result in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}
```

```c
int main(void) {
    unsigned char var = 0xFF;      // 11111111
    unsigned char mask = 0x0A;     // 00001010 (clear bits 1 and 3)

    var &= ~mask;  // Clear multiple bits
    printf("After clearing bits with mask 0x%02X:\n", mask);
    print_binary(var);  // Expected: 11110101
    return 0;
}
```

Expert Tips

- Ensure the mask targets only the intended bits.
- Common in clearing interrupt flags or GPIO states.
- Test with different initial values to confirm behavior.

Project 24: Implement a program to toggle multiple bits in a variable using a mask.

Project Description: Write a C program to toggle multiple specified bits in an 8-bit variable using a bitwise mask and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise XOR with a mask.
- **Steps**:
    1. Define an 8-bit variable and a mask with 1s at bits to toggle.
    2. Use bitwise XOR to toggle the bits.
    3. Print the result in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char var = 0x00;      // 00000000
    unsigned char mask = 0x0A;     // 00001010 (toggle bits 1 and 3)

    var ^= mask;  // Toggle multiple bits
    printf("After toggling bits with mask 0x%02X:\n", mask);
    print_binary(var);  // Expected: 00001010
    return 0;
}
```

Expert Tips

- XOR is ideal for toggling multiple bits in one operation.
- Useful for alternating GPIO patterns or flag states.
- Define masks clearly to avoid errors in complex systems.

Project 25: Write a program to read a 4-bit value from a variable using bit masking.

Project Description: Write a C program to extract a 4-bit value (e.g., bits 0-3) from an 8-bit variable using bit masking and display it.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise AND with a mask and shift.
- **Steps**:
    1. Define an 8-bit variable.
    2. Create a mask for the lower 4 bits (0x0F).
    3. Use AND to extract the bits, optionally shift if targeting other bits.
    4. Print the 4-bit value.

C Code Implementation

```
#include <stdio.h>

int main(void) {
    unsigned char var = 0x5A;      // 01011010
    unsigned char mask = 0x0F;     // 00001111 (lower 4 bits)

    unsigned char result = var & mask;  // Extract lower 4 bits
    printf("Lower 4 bits of 0x%02X: 0x%X (%d)\n", var, result, result);  // Expected: 0x0A (10)
    return 0;
}
```

Expert Tips

- Adjust the mask and shift for other bit ranges (e.g., bits 4-7: ((var >> 4) & 0x0F)).
- Useful for reading specific fields in registers (e.g., ADC status).
- Ensure the mask aligns with the target bits.

Project 26: Create a program to implement a simple counter using a while loop.

Project Description: Write a C program to implement a counter that increments from 0 to 255 in a while loop and prints the value.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Incremental loop with wraparound.
- **Steps**:
    1. Define an 8-bit counter variable.
    2. Use a while loop to increment the counter.
    3. Print the counter value.
    4. Add a delay for visibility.

C Code Implementation

```c
#include <stdio.h>

void delay(void) {
    volatile unsigned long i;
    for (i = 0; i < 100000; i++);
}

int main(void) {
    unsigned char counter = 0;

    printf("Starting counter...\n");
    while (1) {
        printf("Counter: %d\n", counter);
        counter++;  // Wraps at 255 for unsigned char
        delay();
    }
    return 0;
}
```

Expert Tips

- Use uint8_t for explicit 8-bit counters.
- For real systems, display on LEDs or LCD instead of printf.
- Consider resetting the counter or adding a limit for specific applications.

Project 27: Implement a program to use a for loop to blink an LED 10 times.

Project Description: Write a C program to blink an LED connected to a GPIO pin 10 times (0.5s ON, 0.5s OFF) using a for loop.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Fixed iteration loop.
- **Steps**:
    1. Initialize LED pin as output.
    2. Use a for loop to iterate 10 times.
    3. In each iteration, turn LED ON, delay, turn OFF, delay.
    4. Exit after 10 blinks.

C Code Implementation

```c
#include <stdio.h>
#define LED_PIN 1

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // Initialize LED (simulated)
    // In real: DDRB |= (1 << LED_PIN);
    printf("Blinking LED 10 times on pin %d.\n", LED_PIN);
```

```
    for (int i = 0; i < 10; i++) {
        printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
        delay_ms(500);
        printf("LED OFF\n");  // In real: PORTB &= ~(1 << LED_PIN);
        delay_ms(500);
    }
    printf("Blinking complete.\n");
    return 0;
}
```

Expert Tips

- Use a timer interrupt for non-blocking blinks.
- Ensure the loop count matches the requirement (10 blinks = 10 ON/OFF cycles).
- Test with an LED to confirm timing.

Project 28: Write a program to use a switch-case statement to control LED patterns.

Project Description: Write a C program to control different LED patterns (e.g., all OFF, all ON, alternating) on an 8-bit port using a switch-case statement based on an input value.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Switch-case state selection.
- **Steps**:
    1. Define an 8-bit port and an input variable (0-2 for patterns).
    2. Use switch-case to select a pattern (e.g., 0x00, 0xFF, 0xAA).
    3. Apply the pattern to the port and print it.

C Code Implementation

```
#include <stdio.h>
#define LED_PORT 3

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char pattern;
    int choice = 1;  // Change to 0, 1, or 2 for different patterns

    // In real: DDRB = 0xFF;  // Set port as output
    printf("Setting LED pattern on port %d.\n", LED_PORT);

    switch (choice) {
        case 0:
            pattern = 0x00;  // All OFF
            break;
        case 1:
            pattern = 0xFF;  // All ON
            break;
        case 2:
            pattern = 0xAA;  // Alternating (10101010)
            break;
```

```
        default:
            pattern = 0x00;  // Default OFF
    }

    // In real: PORTB = pattern;
    printf("Pattern: ");
    print_binary(pattern);
    return 0;
}
```

Expert Tips

- Use enums for pattern choices to improve readability.
- Extend with more patterns or dynamic input (e.g., from a button).
- Test on hardware to visualize patterns on LEDs.

Project 29: Create a program to store an array of 10 values and print them to the serial console.

Project Description: Write a C program to store 10 predefined 8-bit values in an array and print them to the serial console.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Array iteration.
- **Steps**:
    1. Define an array of 10 unsigned 8-bit values.
    2. Use a for loop to iterate and print each value.
    3. Add a delay for readability.

C Code Implementation

```
#include <stdio.h>

int main(void) {
    unsigned char values[10] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};

    printf("Printing array values:\n");
    for (int i = 0; i < 10; i++) {
        printf("Value[%d] = %d\n", i, values[i]);
        for (volatile int j = 0; j < 100000; j++);  // Delay
    }
    return 0;
}
```

Expert Tips

- Use uint8_t for array elements to ensure 8-bit size.
- In real systems, use UART for serial output.
- Consider dynamic array initialization (e.g., from ADC or sensors).

Project 30: Implement a program to calculate the average of 5 ADC readings.

Project Description: Write a C program to simulate reading 5 ADC values (8-bit) and compute their average, then print the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Arithmetic mean.
- **Steps**:
    1. Define an array to store 5 ADC readings.
    2. Simulate readings (in real: read from ADC register).
    3. Sum the readings and divide by 5.
    4. Print the average.

C Code Implementation

```
#include <stdio.h>

int main(void) {
    unsigned char readings[5] = {100, 110, 105, 115, 108};  // Simulated ADC values
    unsigned int sum = 0;

    for (int i = 0; i < 5; i++) {
        sum += readings[i];
    }

    unsigned char average = sum / 5;
    printf("Average of 5 ADC readings: %d\n", average);
    return 0;
}
```

Expert Tips

- Use uint16_t for sum to prevent overflow.
- In real systems, configure ADC (e.g., ADMUX, ADCSRA on AVR) before reading.
- Apply filtering (e.g., moving average) for noisy ADC data.

Project 31: Write a program to use a pointer to modify a variable's value.

Project Description: Write a C program to modify an 8-bit variable's value using a pointer and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Pointer dereferencing.
- **Steps**:
    1. Define an 8-bit variable.
    2. Create a pointer to the variable.
    3. Modify the value via the pointer.
    4. Print the modified value.

C Code Implementation

```c
#include <stdio.h>

int main(void) {
    unsigned char value = 10;
    unsigned char *ptr = &value;

    printf("Original value: %d\n", value);
    *ptr = 50;  // Modify via pointer
    printf("Modified value: %d\n", value);
    return 0;
}
```

Expert Tips

- Ensure the pointer is correctly initialized to avoid undefined behavior.
- Useful for accessing hardware registers directly (e.g., volatile uint8_t *port = &PORTB).
- Check pointer alignment in memory-constrained systems.

Project 32: Create a program to use a function to toggle an LED.

Project Description: Write a C program that uses a function to toggle an LED on a GPIO pin and calls it in a loop.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Function call with delay.
- **Steps**:
    1. Define a function to toggle the LED state.
    2. Initialize LED pin as output.
    3. Call the toggle function in a loop with delay.

C Code Implementation

```c
#include <stdio.h>
#define LED_PIN 1
int led_state = 0;

void delay_1sec(void) {
    volatile unsigned long i;
    for (i = 0; i < 1000000UL; i++);
}


void toggle_led(void) {
    led_state = !led_state;
    printf("LED %s\n", led_state ? "ON" : "OFF");  // In real: PORTB ^= (1 << LED_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN);
    printf("Toggling LED on pin %d.\n", LED_PIN);

    while (1) {
        toggle_led();
        delay_1sec();
    }
    return 0;
}
```

Expert Tips

- Encapsulate hardware access in functions for modularity.
- Use inline functions for performance in critical code.
- Test the function with different delay values.

Project 33: Implement a program to pass a variable by reference to a function.

Project Description: Write a C program to modify an 8-bit variable by passing it by reference to a function and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Pointer parameter in function.
- **Steps**:
    1. Define a function that takes a pointer to an 8-bit variable.
    2. Modify the variable via the pointer in the function.
    3. Call the function and print the result.

C Code Implementation

```
#include <stdio.h>

void modify_value(unsigned char *ptr, unsigned char new_value) {
    *ptr = new_value;
}

int main(void) {
    unsigned char value = 10;

    printf("Before: %d\n", value);
    modify_value(&value, 50);
    printf("After: %d\n", value);
    return 0;
}
```

Expert Tips

- Use const for pointer parameters if the function shouldn't modify the data.
- Validate pointer inputs to avoid null pointer issues.
- Common for updating register values or shared variables.

Project 34: Write a program to use a macro to define a constant delay value.

Project Description: Write a C program that uses a macro to define a constant delay value and uses it to control LED blinking.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Macro substitution with delay.
- **Steps**:
    1. Define a macro for the delay duration (in loops).
    2. Implement a delay function using the macro.

3. Toggle an LED in a loop using the delay.

C Code Implementation

```c
#include <stdio.h>
#define LED_PIN 1
#define DELAY_MS 1000000UL  // Delay loop count (~1s)

void delay(void) {
    volatile unsigned long i;
    for (i = 0; i < DELAY_MS; i++);
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN);
    printf("Blinking LED with macro-defined delay.\n");

    while (1) {
        printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
        delay();
        printf("LED OFF\n");  // In real: PORTB &= ~(1 << LED_PIN);
        delay();
    }
    return 0;
}
```

Expert Tips

- Use macros for constants to improve code maintainability.
- Calibrate DELAY_MS based on MCU clock speed.
- Prefer hardware timers for precise delays in production.

Project 35: Create a program to use a macro to set a GPIO pin.

Project Description: Write a C program that uses a macro to set a GPIO pin high and applies it to an LED.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Macro for bit manipulation.
- **Steps**:
    1. Define a macro to set a specific GPIO pin.
    2. Initialize the pin as output.
    3. Use the macro to set the pin and print the action.

C Code Implementation

```c
#include <stdio.h>
#define LED_PIN 1
#define SET_PIN(port, pin) (port |= (1 << pin))  // Macro to set pin

int main(void) {
    // In real: DDRB |= (1 << LED_PIN);
    printf("Setting LED pin %d.\n", LED_PIN);

    // Simulate port
    unsigned char port = 0x00;
    SET_PIN(port, LED_PIN);  // In real: SET_PIN(PORTB, LED_PIN);
    printf("Port state after setting pin %d: 0x%02X\n", LED_PIN, port);
```

```
    return 0;
}
```

Expert Tips

- Macros simplify repetitive bit operations but can obscure code; use sparingly.
- Define clear macro names (e.g., SET_GPIO) for readability.
- Test the macro with different pins and ports.

Project 36: Implement a program to read a 16-bit value and split it into two 8-bit values.

Project Description: Write a C program to take a 16-bit value and split it into two 8-bit values (high and low bytes) using bit manipulation.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise shift and mask.
- **Steps**:
    1. Define a 16-bit value.
    2. Extract the low byte using value & 0xFF.
    3. Extract the high byte using (value >> 8) & 0xFF.
    4. Print both bytes.

C Code Implementation

```
#include <stdio.h>

int main(void) {
    unsigned short value = 0x1234;  // 16-bit value
    unsigned char low_byte = value & 0xFF;        // Lower 8 bits
    unsigned char high_byte = (value >> 8) & 0xFF; // Upper 8 bits

    printf("16-bit value: 0x%04X\n", value);
    printf("High byte: 0x%02X\n", high_byte);  // Expected: 0x12
    printf("Low byte: 0x%02X\n", low_byte);    // Expected: 0x34
    return 0;
}
```

Expert Tips

- Use uint16_t and uint8_t for portability.
- Common in SPI/I2C data handling where 16-bit data is split.
- Verify byte order (endianness) for your MCU.

Project 37: Write a program to implement a simple delay loop without a timer.

Project Description: Write a C program to implement a software-based delay loop (no hardware timer) to pause execution for approximately 1 second.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Busy-wait loop.
- **Steps**:
    1. Define a loop counter for approximate delay.
    2. Use a volatile variable to prevent optimization.
    3. Iterate the loop to achieve ~1s delay.
    4. Test with a simple LED toggle.

C Code Implementation

```c
#include <stdio.h>
#define LED_PIN 1

void delay_1sec(void) {
    volatile unsigned long i;
    for (i = 0; i < 1000000UL; i++);  // Calibrated for ~1s
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN);
    printf("Testing delay with LED toggle.\n");

    while (1) {
        printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
        delay_1sec();
        printf("LED OFF\n");  // In real: PORTB &= ~(1 << LED_PIN);
        delay_1sec();
    }
    return 0;
}
```

Expert Tips

- Calibrate the loop count based on MCU clock speed.
- Avoid busy-wait in production; use timers for efficiency.
- Use volatile to ensure the loop isn't optimized out.

Project 38: Create a program to use an enum to represent LED states (ON, OFF, BLINK).

Project Description: Write a C program that uses an enum to represent LED states (ON, OFF, BLINK) and controls an LED accordingly.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Enum-based state machine.
- **Steps**:
    1. Define an enum for LED states.
    2. Initialize LED pin and state.
    3. Use a switch-case to handle each state (BLINK toggles periodically).
    4. Print actions.

C Code Implementation

```c
#include <stdio.h>
#define LED_PIN 1

enum LedState { OFF, ON, BLINK };
void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    enum LedState state = BLINK;
    int led_on = 0;

    // In real: DDRB |= (1 << LED_PIN);
    printf("Controlling LED on pin %d.\n", LED_PIN);

    while (1) {
        switch (state) {
            case OFF:
                printf("LED OFF\n");  // In real: PORTB &= ~(1 << LED_PIN);
                break;
            case ON:
                printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
                break;
            case BLINK:
                led_on = !led_on;
                printf("LED %s\n", led_on ? "ON" : "OFF");  // In real: PORTB ^= (1 << LED_PIN);
                delay_ms(500);
                break;
        }
        delay_ms(1000);  // Slow down for non-BLINK states
    }
    return 0;
}
```

Expert Tips

- Enums improve code clarity for state machines.
- For BLINK, use a timer interrupt for precise timing.
- Extend with state transitions based on inputs (e.g., button).

Project 39: Implement a program to use a struct to store GPIO pin configurations.

Project Description: Write a C program that uses a struct to store GPIO pin configurations (pin number, direction, state) and prints them.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Struct data storage.
- **Steps**:
    1. Define a struct for GPIO configuration (pin, direction, state).
    2. Initialize a struct instance.
    3. Print the configuration details.

C Code Implementation

```c
#include <stdio.h>

struct GpioConfig {
    unsigned char pin;
    char direction;  // 'I' for input, 'O' for output
    char state;      // 'H' for high, 'L' for low
};

int main(void) {
    struct GpioConfig led = {1, 'O', 'L'};

    printf("GPIO Config:\n");
    printf("Pin: %d\n", led.pin);
    printf("Direction: %c\n", led.direction);
    printf("State: %c\n", led.state);

    // In real: DDRB |= (led.direction == 'O') ? (1 << led.pin) : 0;
    // In real: PORTB |= (led.state == 'H') ? (1 << led.pin) : 0;
    return 0;
}
```

Expert Tips

- Use structs for organizing related data (e.g., multiple pins).
- Add validation for direction and state values.
- Useful for managing multiple GPIO configurations in larger systems.

Project 40: Write a program to calculate the factorial of a number using a loop.

Project Description: Write a C program to calculate the factorial of a given number (e.g., 5) using a loop and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Iterative multiplication.
- **Steps**:
    1. Define the input number (e.g., 5).
    2. Use a for loop to multiply numbers from 1 to n.
    3. Check for overflow (use unsigned long).
    4. Print the result.

C Code Implementation

```c
#include <stdio.h>

int main(void) {
    unsigned char n = 5;
    unsigned long factorial = 1;

    for (unsigned char i = 1; i <= n; i++) {
        factorial *= i;
    }

    printf("Factorial of %d = %lu\n", n, factorial);  // Expected: 120
    return 0;
}
```

Expert Tips

- Use unsigned long to handle larger factorials (up to ~12 for 32-bit).
- Check for overflow in production code.
- For small n, consider a lookup table to save computation.

Project 41: Create a program to use a typedef to define a custom data type for sensor readings.

Project Description: Write a C program that uses a typedef to define a custom data type for sensor readings (e.g., value and unit) and prints a sample reading.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Typedef struct definition.
- **Steps**:
    1. Define a typedef struct for sensor reading (value, unit).
    2. Initialize a sample reading.
    3. Print the reading details.

C Code Implementation

```c
#include <stdio.h>

typedef struct {
    unsigned char value;
    char unit[4];  // e.g., "mV", "deg"
} SensorReading;

int main(void) {
    SensorReading temp = {25, "deg"};

    printf("Sensor Reading: %d %s\n", temp.value, temp.unit);
    return 0;
}
```

Expert Tips

- Use typedef for cleaner code and reusability.
- Ensure string fields (unit) are null-terminated.
- Useful for organizing ADC or sensor data.

Project 42: Implement a program to convert a temperature from Celsius to Fahrenheit.

Project Description: Write a C program to convert a temperature from Celsius to Fahrenheit using the formula F = (C * 9/5) + 32 and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Linear transformation.
- **Steps**:

1. Define a Celsius temperature.
2. Apply the formula F = (C * 9/5) + 32.
3. Print the result.

C Code Implementation

```c
#include <stdio.h>

int main(void) {
    float celsius = 25.0;
    float fahrenheit = (celsius * 9.0 / 5.0) + 32.0;

    printf("%.1f°C = %.1f°F\n", celsius, fahrenheit);  // Expected: 77.0°F
    return 0;
}
```

Expert Tips

- Use float or double for decimal precision.
- Validate input ranges for realistic temperatures.
- In embedded systems, avoid floating-point if possible; use fixed-point arithmetic.

Project 43: Write a program to check if a number is a power of 2 using bit manipulation.

Project Description: Write a C program to check if an 8-bit number is a power of 2 (e.g., 1, 2, 4, 8, ...) using bit manipulation.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise AND check.
- **Steps**:
    1. Define an 8-bit number.
    2. Use num & (num - 1); if 0, it's a power of 2.
    3. Print the result.

C Code Implementation

```c
#include <stdio.h>

int main(void) {
    unsigned char num = 8;  // Power of 2

    if (num && !(num & (num - 1))) {
        printf("%d is a power of 2.\n", num);
    } else {
        printf("%d is NOT a power of 2.\n", num);
    }
    return 0;
}
```

Expert Tips

- The num && check prevents 0 from being considered a power of 2.
- Fast and efficient for flag or memory alignment checks.
- Test with edge cases (0, 1, 255).

Project 44: Create a program to use a static variable to count button presses.

Project Description: Write a C program that uses a static variable to count button presses and prints the count.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Static variable increment.
- **Steps**:
    1. Define a function with a static variable to track presses.
    2. Simulate button presses and increment the counter.
    3. Print the count after each press.

C Code Implementation

```
#include <stdio.h>

void count_press(void) {
    static unsigned int count = 0;
    count++;
    printf("Button pressed %d times.\n", count);
}
int main(void) {
    for (int i = 0; i < 5; i++) {
        count_press();  // Simulate 5 presses
        for (volatile int j = 0; j < 100000; j++);  // Delay
    }
    return 0;
}
```

Expert Tips

- Static variables retain value between calls, ideal for counters.
- In real systems, tie to an interrupt-driven button press.
- Reset the counter if needed for specific applications.

Project 45: Implement a program to use a volatile variable to handle GPIO input.

Project Description: Write a C program that uses a volatile variable to read a GPIO input (simulated) and print its state.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling with volatile variable.
- **Steps**:
    1. Define a volatile variable for the GPIO input.
    2. Simulate reading the input.
    3. Print the state in a loop.

C Code Implementation

```c
#include <stdio.h>
#define BUTTON_PIN 1

int main(void) {
    volatile unsigned char input_state;  // Volatile for GPIO

    // In real: DDRB &= ~(1 << BUTTON_PIN); PORTB |= (1 << BUTTON_PIN);
    printf("Monitoring GPIO input on pin %d.\n", BUTTON_PIN);

    while (1) {
        input_state = (rand() % 2);  // Simulate: in real, read PINB & (1 << BUTTON_PIN)
        printf("Input state: %s\n", input_state ? "HIGH" : "LOW");
        for (volatile int i = 0; i < 500000; i++);  // Delay
    }
    return 0;
}
```

Expert Tips

- volatile ensures the compiler doesn't optimize out GPIO reads.
- Use for hardware registers or interrupt-shared variables.
- Prefer interrupts over polling for efficiency.

Project 46: Write a program to implement a simple checksum for an array of bytes.

Project Description: Write a C program to calculate a simple checksum (sum of bytes) for an array of 8-bit values and print it.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Summation checksum.
- **Steps**:
    1. Define an array of bytes.
    2. Sum all bytes, wrapping around for 8-bit.
    3. Print the checksum.

C Code Implementation

```c
#include <stdio.h>

int main(void) {
    unsigned char data[] = {10, 20, 30, 40, 50};
    unsigned char checksum = 0;

    for (int i = 0; i < 5; i++) {
        checksum += data[i];
    }

    printf("Checksum of array: 0x%02X (%d)\n", checksum, checksum);  // Expected: 150
    return 0;
}
```

Expert Tips

- Use 8-bit checksum for simplicity; extend to CRC for robustness.
- Common in communication protocols (e.g., UART, I2C).
- Test with edge cases (e.g., all 0xFF).

Project 47: Create a program to reverse the bits in an 8-bit variable.

Project Description: Write a C program to reverse the bits in an 8-bit variable (e.g., 10110010 to 01001101) and display the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bit manipulation with loop.
- **Steps**:
    1. Define an 8-bit variable.
    2. Iterate through each bit, shifting and building the reversed value.
    3. Print the original and reversed values in binary.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    unsigned char var = 0xB2;  // 10110010
    unsigned char reversed = 0;

    printf("Original: ");
    print_binary(var);

    for (int i = 0; i < 8; i++) {
        reversed <<= 1;
        reversed |= (var >> i) & 1;
    }

    printf("Reversed: ");
    print_binary(reversed);  // Expected: 01001101
    return 0;
}
```

Expert Tips

- Optimize with a lookup table for frequent reversals.
- Useful in protocols requiring bit-order reversal (e.g., some SPI modes).
- Verify with different input patterns.

Project 48: Implement a program to count the number of set bits in a variable.

Project Description: Write a C program to count the number of 1s (set bits) in an 8-bit variable and print the result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bitwise AND and shift.
- **Steps**:
    1. Define an 8-bit variable.
    2. Iterate through each bit, counting 1s using & 1.
    3. Print the count.

C Code Implementation

```c
#include <stdio.h>

int main(void) {
    unsigned char var = 0xA5;  // 10100101
    int count = 0;

    for (int i = 0; i < 8; i++) {
        if (var & (1 << i)) {
            count++;
        }
    }

    printf("Number of set bits in 0x%02X: %d\n", var, count);  // Expected: 5
    return 0;
}
```

Expert Tips

- Optimize with Brian Kernighan's algorithm: while (var) { count++; var &= var-1; }.
- Useful for Hamming weight or error detection.
- Test with sparse and dense bit patterns.

Project 49: Write a program to use a lookup table to map input values to LED patterns.

Project Description: Write a C program that uses a lookup table to map input values (0-3) to specific 8-bit LED patterns and displays the selected pattern.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Array-based lookup.
- **Steps**:
    1. Define a lookup table with 4 LED patterns.
    2. Select an input value (0-3).
    3. Retrieve the corresponding pattern and print it.

C Code Implementation

```c
#include <stdio.h>

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");                    }

int main(void) {
    unsigned char patterns[] = {0x00, 0xFF, 0xAA, 0x55};  // OFF, ON, ALT1, ALT2
```

```
    unsigned char input = 2;  // Select pattern 2 (0xAA)

    if (input < 4) {
        printf("Pattern for input %d: ", input);
        print_binary(patterns[input]);  // Expected: 10101010
        // In real: PORTB = patterns[input];
    } else {
        printf("Invalid input.\n");
    }
    return 0;
}
```

Expert Tips

- Lookup tables are efficient for fixed mappings.
- Validate input to prevent out-of-bounds access.
- Useful for predefined LED sequences or state mappings.

Project 50: Create a program to implement a simple state machine for LED control.

Project Description: Write a C program to implement a simple state machine with three states (OFF, ON, BLINK) to control an LED, transitioning based on a counter.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Finite state machine with counter.
- **Steps**:
    1. Define states using an enum.
    2. Initialize the state and a counter.
    3. Transition states based on counter thresholds.
    4. Control LED according to the current state.

C Code Implementation

```
#include <stdio.h>
#define LED_PIN 1

enum State { OFF, ON, BLINK };
void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    enum State state = OFF;
    unsigned int counter = 0;
    int led_on = 0;

    // In real: DDRB |= (1 << LED_PIN);
    printf("State machine for LED on pin %d.\n", LED_PIN);




    while (1) {
        switch (state) {
            case OFF:
```

```
                printf("LED OFF\n");  // In real: PORTB &= ~(1 << LED_PIN);
                if (counter >= 5) { state = ON; counter = 0; }
                break;
            case ON:
                printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
                if (counter >= 5) { state = BLINK; counter = 0; }
                break;
            case BLINK:
                led_on = !led_on;
                printf("LED %s\n", led_on ? "ON" : "OFF");  // In real: PORTB ^= (1 << LED_PIN);
                if (counter >= 10) { state = OFF; counter = 0; }
                delay_ms(500);
                break;
        }
        counter++;
        delay_ms(1000);  // Slow state transitions
    }
    return 0;
}
```

Expert Tips

- **Use `volatile` for Shared Variables**
  If the `counter` or `state` might be updated inside **interrupt service routines (ISRs)** in a real microcontroller implementation, declare them as `volatile.`
  This prevents the compiler from optimizing away repeated reads and ensures correct behavior when the variable changes asynchronously.

```
volatile unsigned int counter = 0;
volatile enum State state = OFF;
```

- **Separate State Logic from Hardware Drivers**
  Adopt a **layered design**:
  - **State Machine Module** → Handles state transitions.
  - **LED Driver Module** → Provides functions like `led_on()`, `led_off()`, and `led_toggle()`.
    This makes the code **portable** and easier to test on a PC or different MCU.
- **Replace Busy-Wait Delays with Timers**
  `delay_ms()` wastes CPU cycles and increases power consumption.
  Instead, use a **hardware timer interrupt** to increment `counter` and trigger state transitions.
  This improves **energy efficiency**, supports **low-power modes**, and keeps the CPU free for other tasks.
- **Add a Fault/Default State**
  Enhance robustness by adding a `DEFAULT` or `ERROR` state in the `switch` statement to catch unexpected values:

```
default:
state = OFF; // fail-safe
break;
```

  This protects against memory corruption or invalid transitions.
- **Enable Event-Driven Expansion**
  Design the state machine to handle **external events** (button press, UART command, sensor input).
  Use an **event queue** or a **finite state machine (FSM) table** for scalability, making it easy to add more states (e.g., DIM, SOS pattern) without rewriting large `switch` blocks.

Microcontroller GPIO and Basic Peripherals

Project 51: Configure a GPIO pin as an output and toggle it every 500 ms.

Project Description: Write a C program to configure a GPIO pin as an output and toggle its state every 500 milliseconds to control an LED.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Infinite loop with delay.
- **Steps**:
    1. Include necessary headers for GPIO control.
    2. Define the GPIO pin for the LED.
    3. Configure the GPIO pin as an output.
    4. In an infinite loop, toggle the pin state and delay for 500 ms.

C Code Implementation

```
#include <stdio.h>  // For simulation; use <avr/io.h> in real
#define LED_PIN 0   // Example: PB0 in AVR

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);  // Simulate ~1ms per 1000 loops
}

int main(void) {
    // Configure GPIO as output
    // In real: DDRB |= (1 << LED_PIN);
    printf("GPIO pin %d configured as output.\n", LED_PIN);

    while (1) {
        // Toggle pin (simulated)
        // In real: PORTB ^= (1 << LED_PIN);
        printf("LED toggled\n");
        delay_ms(500);
    }
    return 0;  // Never reached
}
```

Expert Tips

- Use hardware timers (e.g., Timer0 in AVR) for precise 500ms delays to reduce CPU load.
- Calibrate software delays based on MCU clock speed (e.g., 16MHz).
- Ensure the LED has a current-limiting resistor to prevent damage.

Project 52: Configure a GPIO pin as an input and read its state in a loop.

Project Description: Write a C program to configure a GPIO pin as an input, read its state continuously, and print it to a serial console.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling loop.
- **Steps**:
    1. Define the GPIO input pin.
    2. Configure the pin as input with an internal pull-up resistor.
    3. In a loop, read the pin state and print it.
    4. Add a delay to prevent console flooding.

C Code Implementation

```
#include <stdio.h>  // For simulation; use UART in real
#define INPUT_PIN 1  // Example: PB1 in AVR

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_pin(void) {
    return (rand() % 2);  // Simulate input; in real: return (PINB & (1 << INPUT_PIN)) ? 0 : 1;
}

int main(void) {
    // Configure as input with pull-up
    // In real: DDRB &= ~(1 << INPUT_PIN); PORTB |= (1 << INPUT_PIN);
    printf("Monitoring input pin %d.\n", INPUT_PIN);

    while (1) {
        int state = read_pin();
        printf("Pin state: %s\n", state ? "HIGH" : "LOW");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Enable internal pull-up resistors to avoid floating inputs.
- Use hardware debouncing (capacitor) or software debouncing (~10ms delay) for stable readings.
- For real UART output, configure the USART module with appropriate baud rate.

Project 53: Create a project to light up an LED when a button is pressed.

Project Description: Write a C program to light up an LED connected to a GPIO pin when a button connected to another GPIO pin is pressed.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling with conditional output.

- **Steps**:
    1. Configure LED pin as output and button pin as input with pull-up.
    2. Read the button state in a loop.
    3. If the button is pressed (low with pull-up), turn the LED on; otherwise, turn it off.
    4. Add a delay for stability.

C Code Implementation

```
#include <stdio.h>
#define LED_PIN 0
#define BUTTON_PIN 1

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_button(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << BUTTON_PIN)) ? 0 : 1
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN); DDRB &= ~(1 << BUTTON_PIN); PORTB |= (1 << BUTTON_PIN);
    printf("Button controls LED on pin %d.\n", LED_PIN);

    while (1) {
        if (read_button()) {
            printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
        } else {
            printf("LED OFF\n");  // In real: PORTB &= ~(1 << LED_PIN);
        }
        delay_ms(50);  // Debounce
    }
    return 0;
}
```

Expert Tips

- Use edge detection to avoid continuous toggling during a press.
- Implement software debouncing to handle button bounce.
- Ensure proper current-limiting resistors for the LED.

Project 54: Implement a project to control two LEDs based on two button inputs.

Project Description: Write a C program to control two LEDs based on the states of two buttons, where each button independently controls one LED.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Independent polling for each button.
- **Steps**:
    1. Configure two LED pins as outputs and two button pins as inputs with pull-ups.
    2. Read each button state in a loop.
    3. Set each LED state based on its corresponding button.
    4. Add a delay for stability.

C Code Implementation

```c
#include <stdio.h>
#define LED1_PIN 0
#define LED2_PIN 1
#define BUTTON1_PIN 2
#define BUTTON2_PIN 3

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_button(int pin) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << pin)) ? 0 : 1
}

int main(void) {
    // In real: DDRB |= (1 << LED1_PIN) | (1 << LED2_PIN); DDRB &= ~((1 << BUTTON1_PIN) | (1 <<
BUTTON2_PIN)); PORTB |= (1 << BUTTON1_PIN) | (1 << BUTTON2_PIN);
    printf("Controlling two LEDs with two buttons.\n");

    while (1) {
        if (read_button(BUTTON1_PIN)) {
            printf("LED1 ON\n");  // In real: PORTB |= (1 << LED1_PIN);
        } else {
            printf("LED1 OFF\n");  // In real: PORTB &= ~(1 << LED1_PIN);
        }
        if (read_button(BUTTON2_PIN)) {
            printf("LED2 ON\n");  // In real: PORTB |= (1 << LED2_PIN);
        } else {
            printf("LED2 OFF\n");  // In real: PORTB &= ~(1 << LED2_PIN);
        }
        delay_ms(50);
    }
    return 0;
}
```

Expert Tips

- Use interrupts for button presses to improve responsiveness.
- Add debouncing for each button to ensure reliable operation.
- Consider using a single port for LEDs to simplify bit manipulation.

Project 55: Configure a GPIO pin to control a buzzer with a 1-second pulse.

Project Description: Write a C program to configure a GPIO pin to control a buzzer, generating a 1-second pulse (ON for 1s, OFF for 1s).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timed pulse loop.
- **Steps**:
    1. Configure the buzzer pin as output.
    2. In a loop, set the pin high for 1 second, then low for 1 second.
    3. Add delays to control timing.

C Code Implementation

```c
#include <stdio.h>
#define BUZZER_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << BUZZER_PIN);
    printf("Buzzer pulsing on pin %d.\n", BUZZER_PIN);

    while (1) {
        printf("Buzzer ON\n");  // In real: PORTB |= (1 << BUZZER_PIN);
        delay_ms(1000);
        printf("Buzzer OFF\n");  // In real: PORTB &= ~(1 << BUZZER_PIN);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use a timer for precise pulse timing instead of software delays.
- Ensure the buzzer is compatible with the MCU's voltage and current.
- Add a transistor if the buzzer requires higher current than the GPIO can provide.

Project 56: Create a project to toggle an LED using an external pull-up resistor.

Project Description: Write a C program to toggle an LED connected to a GPIO pin, where the button controlling it uses an external pull-up resistor.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling with edge detection.
- **Steps**:
    1. Configure LED pin as output and button pin as input (no internal pull-up).
    2. Read button state; with external pull-up, pressed = low.
    3. Toggle LED on button press (rising edge).
    4. Add debounce delay.

C Code Implementation

```c
#include <stdio.h>
#define LED_PIN 0
#define BUTTON_PIN 1
int prev_state = 1;  // External pull-up: default high

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_button(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << BUTTON_PIN)) ? 0 : 1
}
```

```c
int main(void) {
    // In real: DDRB |= (1 << LED_PIN); DDRB &= ~(1 << BUTTON_PIN);
    printf("Toggling LED with external pull-up button.\n");
    int led_state = 0;

    while (1) {
        int state = read_button();
        if (!state && prev_state) {  // Falling edge (pressed)
            led_state = !led_state;
            printf("LED %s\n", led_state ? "ON" : "OFF");  // In real: PORTB ^= (1 << LED_PIN);
        }
        prev_state = state;
        delay_ms(50);  // Debounce
    }
    return 0;
}
```

Expert Tips

- External pull-up resistors (e.g., 10kΩ) ensure stable input states.
- Verify resistor value to match MCU voltage levels.
- Use interrupts for button presses to avoid continuous polling.

Project 57: Implement a project to read a switch state and control a relay.

Project Description: Write a C program to read the state of a switch and control a relay connected to a GPIO pin, turning the relay on when the switch is closed.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling with conditional output.
- **Steps**:
    1. Configure relay pin as output and switch pin as input with pull-up.
    2. Read switch state; closed = low (with pull-up).
    3. Set relay pin high if switch is closed, else low.
    4. Add delay for stability.

C Code Implementation

```c
#include <stdio.h>
#define RELAY_PIN 0
#define SWITCH_PIN 1

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_switch(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << SWITCH_PIN)) ? 0 : 1
}

int main(void) {
    // In real: DDRB |= (1 << RELAY_PIN); DDRB &= ~(1 << SWITCH_PIN); PORTB |= (1 << SWITCH_PIN);
    printf("Switch controls relay on pin %d.\n", RELAY_PIN);
```

```
    while (1) {
        if (read_switch()) {
            printf("Relay ON\n");  // In real: PORTB |= (1 << RELAY_PIN);
        } else {
            printf("Relay OFF\n");  // In real: PORTB &= ~(1 << RELAY_PIN);
        }
        delay_ms(50);
    }
    return 0;
}
```

Expert Tips

- Use a transistor or driver circuit for relays, as GPIO pins typically can't handle relay coil current.
- Implement debouncing for the switch to prevent chatter.
- Ensure the relay module matches the MCU's logic level (e.g., 5V or 3.3V).

Project 58: Configure a GPIO pin to generate a square wave using a loop.

Project Description: Write a C program to generate a square wave (50% duty cycle) on a GPIO pin with a period of approximately 1 ms.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Toggle loop with equal delays.
- **Steps**:
    1. Configure GPIO pin as output.
    2. In a loop, set pin high for 0.5ms, then low for 0.5ms.
    3. Repeat to create a square wave.

C Code Implementation

```
#include <stdio.h>
#define WAVE_PIN 0

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);  // Simulate ~1us per 10 loops
}

int main(void) {
    // In real: DDRB |= (1 << WAVE_PIN);
    printf("Generating square wave on pin %d.\n", WAVE_PIN);

    while (1) {
        printf("HIGH\n");  // In real: PORTB |= (1 << WAVE_PIN);
        delay_us(500);
        printf("LOW\n");  // In real: PORTB &= ~(1 << WAVE_PIN);
        delay_us(500);
    }
    return 0;
}
```

Expert Tips

- Use a hardware PWM module (e.g., Timer1 in AVR) for precise square waves.
- Calibrate software delays for the desired frequency (1ms period = 1kHz).

- Verify the wave with an oscilloscope.

Project 59: Create a project to control an LED brightness using PWM on a GPIO pin.

Project Description: Write a C program to control the brightness of an LED using software PWM on a GPIO pin, simulating different brightness levels.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Software PWM with duty cycle.
- **Steps**:
    1. Configure LED pin as output.
    2. Define PWM period (e.g., 1ms) and duty cycle (e.g., 25%, 50%, 75%).
    3. In a loop, set pin high for duty cycle duration, low for remaining period.
    4. Cycle through different duty cycles.

C Code Implementation

```
#include <stdio.h>
#define LED_PIN 0

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

void pwm_led(int duty_cycle, int period_us) {
    int on_time = (duty_cycle * period_us) / 100;
    int off_time = period_us - on_time;

    printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
    delay_us(on_time);
    printf("LED OFF\n");  // In real: PORTB &= ~(1 << LED_PIN);
    delay_us(off_time);
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN);
    printf("PWM LED control on pin %d.\n", LED_PIN);
    int duties[] = {25, 50, 75};

    while (1) {
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 100; j++) {  // Run PWM for ~100 cycles
                pwm_led(duties[i], 1000);  // 1ms period
            }
            delay_us(500000);  // Hold each brightness for ~0.5s
        }
    }
    return 0;
}
```

Expert Tips

- Use hardware PWM (e.g., AVR Timer1) for smoother and more efficient control.
- Adjust PWM frequency (>50Hz) to avoid visible flicker.
- Test brightness levels with an actual LED and resistor.

Project 60: Implement a project to read a digital sensor (e.g., DHT11) via GPIO.

Project Description: Write a C program to read a digital sensor like the DHT11 (simulated) via a GPIO pin and print the temperature and humidity.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bit-bang protocol reading.
- **Steps**:
    1. Configure GPIO pin as output to send start signal, then as input to read data.
    2. Simulate DHT11 protocol: send start pulse, read 40-bit data (temperature, humidity).
    3. Parse and print the values.

C Code Implementation

```
#include <stdio.h>
#define SENSOR_PIN 0

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

void read_dht11(unsigned char *temp, unsigned char *humid) {
    // Simulate DHT11 response: 40-bit data (16-bit temp, 16-bit humid, 8-bit checksum)
    *temp = 25;  // Simulated temperature
    *humid = 60; // Simulated humidity
    // In real: Implement bit-bang protocol with precise timing
}

int main(void) {
    // In real: DDRB and PORTB for start signal and input mode
    unsigned char temp, humid;

    printf("Reading DHT11 on pin %d.\n", SENSOR_PIN);
    while (1) {
        read_dht11(&temp, &humid);
        printf("Temperature: %d°C, Humidity: %d%%\n", temp, humid);
        delay_us(2000000);  // DHT11 requires ~2s between reads
    }
    return 0;
}
```

Expert Tips

- DHT11 requires precise microsecond timing; use a timer or RTOS for accuracy.
- Validate checksum in real implementations to ensure data integrity.
- Add error handling for sensor timeouts or invalid data.

Project 61: Configure a GPIO pin to trigger an interrupt on a button press.

Project Description: Write a C program to configure a GPIO pin to trigger an interrupt on a button press and print a message.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Interrupt-driven event handling.
- **Steps**:
    1. Configure button pin as input with pull-up and enable interrupt (e.g., INT0).
    2. Define an ISR to handle the button press.
    3. Print a message in the ISR (simulated).

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/interrupt.h>
#define BUTTON_PIN 2  // Example: INT0 on PD2 in AVR

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

// Simulated ISR
void ISR_INT0(void) {
    printf("Button pressed (interrupt)!\n");
}

int main(void) {
    // In real: DDRD &= ~(1 << BUTTON_PIN); PORTD |= (1 << BUTTON_PIN);
    // In real: EICRA |= (1 << ISC01); EIMSK |= (1 << INT0); sei();
    printf("Interrupt configured on pin %d.\n", BUTTON_PIN);

    while (1) {
        // Simulate interrupt with polling for demo
        if (rand() % 100 < 10) {  // 10% chance to simulate press
            ISR_INT0();
            delay_ms(50);  // Debounce
        }
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Use hardware interrupts (e.g., INT0/INT1 in AVR) for low latency.
- Implement debouncing in the ISR or with hardware.
- Avoid heavy processing in ISRs; set flags instead.

Project 62: Create a project to count button presses using a GPIO interrupt.

Project Description: Write a C program to count button presses using a GPIO interrupt and print the count.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Interrupt-driven counter.
- **Steps**:
    1. Configure button pin for interrupt (e.g., falling edge).
    2. Use a volatile variable to count presses in the ISR.
    3. Print the count periodically in the main loop.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/interrupt.h>
#define BUTTON_PIN 2
volatile unsigned int press_count = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void ISR_INT0(void) {
    press_count++;
}

int main(void) {
    // In real: DDRD &= ~(1 << BUTTON_PIN); PORTD |= (1 << BUTTON_PIN); EICRA |= (1 << ISC01); EIMSK
|= (1 << INT0); sei();
    printf("Counting button presses on pin %d.\n", BUTTON_PIN);

    while (1) {
        if (rand() % 100 < 5) {  // Simulate interrupt
            ISR_INT0();
            delay_ms(50);  // Debounce
        }
        printf("Button presses: %d\n", press_count);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use volatile for shared variables in ISRs to prevent optimization issues.
- Debounce in the ISR to avoid multiple counts per press.
- Reset the counter if needed for specific applications.

Project 63: Implement a project to toggle an LED on a falling edge interrupt.

Project Description: Write a C program to toggle an LED on a GPIO pin when a falling edge interrupt occurs on a button pin.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Interrupt-driven toggle.
- **Steps**:
    1. Configure LED pin as output and button pin for falling edge interrupt.
    2. In the ISR, toggle the LED state.
    3. Simulate and print the action.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/interrupt.h>
#define LED_PIN 0
#define BUTTON_PIN 2
volatile int led_state = 0;
```

```
void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void ISR_INT0(void) {
    led_state = !led_state;
    printf("LED %s\n", led_state ? "ON" : "OFF");  // In real: PORTB ^= (1 << LED_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN); DDRD &= ~(1 << BUTTON_PIN); PORTD |= (1 << BUTTON_PIN); EICRA
|= (1 << ISC01); EIMSK |= (1 << INT0); sei();
    printf("Toggling LED on falling edge interrupt.\n");

    while (1) {
        if (rand() % 100 < 5) {
            ISR_INT0();
            delay_ms(50);  // Debounce
        }
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Configure interrupt for falling edge (e.g., ISC01 in AVR EICRA).
- Use a timer for debouncing within the ISR.
- Ensure the LED circuit includes a current-limiting resistor.

Project 64: Configure a GPIO pin to read a 4-bit keypad input.

Project Description: Write a C program to read a 4-bit keypad (e.g., 4 keys) connected to GPIO pins and print the pressed key.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling with bit masking.
- **Steps**:
    1. Configure 4 GPIO pins as inputs with pull-ups.
    2. Read the 4-bit value and map to a key (0-3).
    3. Print the pressed key or "none" if no key is pressed.
    4. Add debounce delay.

C Code Implementation

```
#include <stdio.h>
#define KEYPAD_PORT 0  // Simulate 4-bit port (pins 0-3)

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned char read_keypad(void) {
    return (rand() % 16) & 0x0F;  // Simulate 4-bit input; in real: PINB & 0x0F
}
```

```
int main(void) {
    // In real: DDRB &= ~0x0F; PORTB |= 0x0F;
    printf("Reading 4-bit keypad on port %d.\n", KEYPAD_PORT);

    while (1) {
        unsigned char key = read_keypad();
        switch (key) {
            case 0x0E: printf("Key 1 pressed\n"); break;  // 1110
            case 0x0D: printf("Key 2 pressed\n"); break;  // 1101
            case 0x0B: printf("Key 3 pressed\n"); break;  // 1011
            case 0x07: printf("Key 4 pressed\n"); break;  // 0111
            default: printf("No key pressed\n");
        }
        delay_ms(50);
    }
    return 0;
}
```

Expert Tips

- Map keypad inputs to specific bit patterns (e.g., one bit low per key).
- Use a lookup table for key mapping to simplify code.
- Implement debouncing for reliable key detection.

Project 65: Create a project to control a 7-segment display using GPIO pins.

Project Description: Write a C program to display digits 0-9 on a common-cathode 7-segment display using 7 GPIO pins.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Lookup table for segment patterns.
- **Steps**:
    1. Configure 7 GPIO pins as outputs.
    2. Define a lookup table for 7-segment patterns (0-9).
    3. Cycle through digits, setting the GPIO pins according to the pattern.
    4. Add delay for visibility.

C Code Implementation

```
#include <stdio.h>
#define DISPLAY_PORT 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}
```

```
int main(void) {
    // Common-cathode patterns: a,b,c,d,e,f,g
    unsigned char patterns[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
    // In real: DDRB |= 0x7F;  // Pins 0-6 for segments
    printf("Displaying digits on 7-segment.\n");

    while (1) {
        for (int digit = 0; digit < 10; digit++) {
            // In real: PORTB = patterns[digit];
            printf("Digit %d: ", digit);
            print_binary(patterns[digit]);
            delay_ms(1000);
        }
    }
    return 0;
}
```

Expert Tips

- Verify common-cathode vs. common-anode wiring; adjust patterns accordingly.
- Use current-limiting resistors for each segment.
- For multiplexing displays, use additional pins for digit selection.

Project 66: Implement a project to drive a tri-color LED with different colors.

Project Description: Write a C program to control a tri-color (RGB) LED using three GPIO pins to display red, green, blue, and mixed colors.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: State-based GPIO control.
- **Steps**:
    1. Configure three GPIO pins for R, G, B as outputs.
    2. Define color states (e.g., red = R high, others low).
    3. Cycle through colors with delays.
    4. Print the current color.

C Code Implementation

```
#include <stdio.h>
#define RED_PIN 0
#define GREEN_PIN 1
#define BLUE_PIN 2

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void set_color(int r, int g, int b) {
    printf("Color: R=%d, G=%d, B=%d\n", r, g, b);
    // In real: PORTB = (r << RED_PIN) | (g << GREEN_PIN) | (b << BLUE_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << RED_PIN) | (1 << GREEN_PIN) | (1 << BLUE_PIN);
    printf("Cycling tri-color LED.\n");
```

```
    while (1) {
        set_color(1, 0, 0);  // Red
        delay_ms(1000);
        set_color(0, 1, 0);  // Green
        delay_ms(1000);
        set_color(0, 0, 1);  // Blue
        delay_ms(1000);
        set_color(1, 1, 0);  // Yellow
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use PWM for smooth color mixing and brightness control.
- Ensure the LED is common-cathode or common-anode and adjust logic.
- Add resistors to limit current for each LED segment.

Project 67: Configure a GPIO pin to control a motor direction via an H-bridge.

Project Description: Write a C program to control a DC motor's direction (forward/reverse) using two GPIO pins connected to an H-bridge.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: State-based GPIO control.
- **Steps**:
    1. Configure two GPIO pins for H-bridge inputs (e.g., IN1, IN2).
    2. Set pin states for forward (IN1 high, IN2 low), reverse (IN1 low, IN2 high), or stop (both low).
    3. Cycle through directions with delays.
    4. Print the motor state.

C Code Implementation

```
#include <stdio.h>
#define IN1_PIN 0
#define IN2_PIN 1

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void set_motor(int in1, int in2) {
    printf("Motor: IN1=%d, IN2=%d\n", in1, in2);  // In real: PORTB = (in1 << IN1_PIN) | (in2 <<
IN2_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << IN1_PIN) | (1 << IN2_PIN);
    printf("Controlling motor direction.\n");

    while (1) {
        set_motor(1, 0);  // Forward
        delay_ms(2000);
        set_motor(0, 0);  // Stop
        delay_ms(1000);
        set_motor(0, 1);  // Reverse
```

```
        delay_ms(2000);
        set_motor(0, 0);  // Stop
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Verify H-bridge wiring and logic levels (e.g., L293D requires 5V logic).
- Use PWM for speed control in addition to direction.
- Add flyback diodes to protect the H-bridge from motor back-EMF.

Project 68: Create a project to read a rotary encoder using two GPIO pins.

Project Description: Write a C program to read a rotary encoder's direction (clockwise/counterclockwise) using two GPIO pins and print the count.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Quadrature decoding with polling.
- **Steps**:
    1. Configure two GPIO pins (A, B) as inputs with pull-ups.
    2. Read both pins and detect state transitions to determine direction.
    3. Increment or decrement a counter based on direction.
    4. Print the counter value.

C Code Implementation

```
#include <stdio.h>
#define ENCODER_A 0
#define ENCODER_B 1

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_encoder(void) {
    static int prev_a = 0;
    int a = (rand() % 2);  // Simulate A
    int b = (rand() % 2);  // Simulate B
    int direction = 0;
    if (a && !prev_a && !b) direction = 1;   // CW
    if (a && !prev_a && b) direction = -1;   // CCW
    prev_a = a;
    return direction;  // In real: read PINB & (1 << ENCODER_A/B)
}

int main(void) {
    // In real: DDRB &= ~((1 << ENCODER_A) | (1 << ENCODER_B)); PORTB |= (1 << ENCODER_A) | (1 <<
ENCODER_B);
    int count = 0;
    printf("Reading rotary encoder.\n");

    while (1) {
        int dir = read_encoder();
        count += dir;
```

```
        if (dir != 0) {
            printf("Encoder count: %d (%s)\n", count, dir > 0 ? "CW" : "CCW");
        }
        delay_ms(50);
    }
    return 0;
}
```

Expert Tips

- Use interrupts on pin change (e.g., PCINT in AVR) for accurate detection.
- Debounce encoder signals with software or capacitors.
- Test with slow rotations to verify direction logic.

Project 69: Implement a project to control a servo motor using PWM on a GPIO pin.

Project Description: Write a C program to control a servo motor using software PWM on a GPIO pin to set different angles (e.g., 0°, 90°, 180°).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Software PWM with variable pulse width.
- **Steps**:
    1. Configure GPIO pin as output.
    2. Generate PWM with 20ms period and pulse widths (1ms for 0°, 1.5ms for 90°, 2ms for 180°).
    3. Cycle through angles with delays.
    4. Print the current angle.

C Code Implementation

```
#include <stdio.h>
#define SERVO_PIN 0

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

void set_servo_angle(int pulse_us) {
    printf("Servo pulse: %dus\n", pulse_us);  // In real: PORTB |= (1 << SERVO_PIN);
    delay_us(pulse_us);
    printf("Servo off\n");  // In real: PORTB &= ~(1 << SERVO_PIN);
    delay_us(20000 - pulse_us);  // 20ms period
}

int main(void) {
    // In real: DDRB |= (1 << SERVO_PIN);
    printf("Controlling servo on pin %d.\n", SERVO_PIN);
    int angles[] = {1000, 1500, 2000};  // 0°, 90°, 180°

    while (1) {
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 50; j++) {  // Repeat PWM for stability
                set_servo_angle(angles[i]);
            }
            printf("Angle: %d°\n", i * 90);
            delay_us(1000000);  // 1s per angle
        }
    }
```

```
    return 0;
}
```

Expert Tips

- Use hardware PWM (e.g., AVR Timer1 in fast PWM mode) for precise servo control.
- Ensure pulse widths match the servo's specs (typically 1-2ms).
- Test with a real servo to confirm angle accuracy.

Project 70: Configure a GPIO pin to interface with a simple IR sensor.

Project Description: Write a C program to read a digital IR sensor (e.g., obstacle detection) via a GPIO pin and print its state.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling input.
- **Steps**:
    1. Configure GPIO pin as input.
    2. Read the sensor state (e.g., high = no obstacle, low = obstacle).
    3. Print the state.
    4. Add delay for readability.

C Code Implementation

```c
#include <stdio.h>
#define IR_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_ir(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << IR_PIN)) ? 0 : 1
}

int main(void) {
    // In real: DDRB &= ~(1 << IR_PIN); PORTB |= (1 << IR_PIN);
    printf("Reading IR sensor on pin %d.\n", IR_PIN);

    while (1) {
        int state = read_ir();
        printf("IR sensor: %s\n", state ? "Obstacle detected" : "No obstacle");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Check the IR sensor's output type (active-high or active-low).
- Use interrupts for faster response to state changes.
- Calibrate sensor range with hardware adjustments.

Project 71: Create a project to toggle multiple LEDs in a sequence using GPIO.

Project Description: Write a C program to toggle four LEDs in a sequence (e.g., left to right) using GPIO pins with a visible delay.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Sequential GPIO control.
- **Steps**:
    1. Configure four GPIO pins as outputs.
    2. Define a sequence of LED states.
    3. Cycle through the sequence, setting pins accordingly.
    4. Add delay between steps.

C Code Implementation

```
#include <stdio.h>
#define LED1_PIN 0
#define LED2_PIN 1
#define LED3_PIN 2
#define LED4_PIN 3

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);   }

void set_leds(int led1, int led2, int led3, int led4) {
    printf("LEDs: %d %d %d %d\n", led1, led2, led3, led4);
    // In real: PORTB = (led1 << LED1_PIN) | (led2 << LED2_PIN) | (led3 << LED3_PIN) | (led4 <<
LED4_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << LED1_PIN) | (1 << LED2_PIN) | (1 << LED3_PIN) | (1 << LED4_PIN);
    printf("Sequencing LEDs.\n");

    while (1) {
        set_leds(1, 0, 0, 0); delay_ms(500);
        set_leds(0, 1, 0, 0); delay_ms(500);
        set_leds(0, 0, 1, 0); delay_ms(500);
        set_leds(0, 0, 0, 1); delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Use a single port for all LEDs to simplify bit manipulation.
- Store sequences in an array for flexibility.
- Add reverse or random sequences for variety.

Project 72: Implement a project to read a push-button matrix (e.g., 4x4 keypad).

Project Description: Write a C program to read a 4x4 keypad matrix using 8 GPIO pins (4 rows, 4 columns) and print the pressed key.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Matrix scanning.
- **Steps**:
    1. Configure 4 row pins as outputs, 4 column pins as inputs with pull-ups.
    2. Set one row low at a time and read columns.
    3. Map the row-column combination to a key.
    4. Print the key or "none."

C Code Implementation

```c
#include <stdio.h>
#define ROW_PORT 0  // Pins 0-3 for rows
#define COL_PORT 4  // Pins 4-7 for columns

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned char read_keypad(void) {
    return (rand() % 16);  // Simulate column read; in real: (PINB >> COL_PORT) & 0x0F
}

int main(void) {
    // In real: DDRB |= 0x0F; DDRB &= ~(0xF0); PORTB |= 0xF0;
    char keys[4][4] = {{'1','2','3','A'}, {'4','5','6','B'}, {'7','8','9','C'}, {'*','0','#','D'}};
    printf("Reading 4x4 keypad.\n");

    while (1) {
        for (int row = 0; row < 4; row++) {
            // In real: PORTB = ~(1 << row);
            unsigned char cols = read_keypad();
            for (int col = 0; col < 4; col++) {
                if (!(cols & (1 << col))) {
                    printf("Key pressed: %c\n", keys[row][col]);
                }
            }
        }
        delay_ms(50);
    }
    return 0;
}
```

Expert Tips

- Use pull-up resistors on columns to avoid floating inputs.
- Implement debouncing for each key press.
- Optimize scanning with interrupts or timer-based polling.

Project 73: Configure a GPIO pin to drive a piezo buzzer with different frequencies.

Project Description: Write a C program to drive a piezo buzzer on a GPIO pin with different frequencies (e.g., 500Hz, 1kHz, 2kHz).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Software square wave generation.

- **Steps**:
    1. Configure buzzer pin as output.
    2. Define frequencies and calculate half-periods (e.g., 1ms for 500Hz).
    3. Generate square waves by toggling the pin with appropriate delays.
    4. Cycle through frequencies.

C Code Implementation

```c
#include <stdio.h>
#define BUZZER_PIN 0

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

void play_tone(int freq_hz, int duration_ms) {
    int half_period_us = 500000 / freq_hz;  // Half period in us
    int cycles = (duration_ms * 1000) / (2 * half_period_us);

    for (int i = 0; i < cycles; i++) {
        printf("Buzzer ON\n");  // In real: PORTB |= (1 << BUZZER_PIN);
        delay_us(half_period_us);
        printf("Buzzer OFF\n");  // In real: PORTB &= ~(1 << BUZZER_PIN);
        delay_us(half_period_us);
    }
}

int main(void) {
    // In real: DDRB |= (1 << BUZZER_PIN);
    printf("Playing tones on buzzer.\n");
    int frequencies[] = {500, 1000, 2000};

    while (1) {
        for (int i = 0; i < 3; i++) {
            play_tone(frequencies[i], 1000);
            delay_us(500000);  // 0.5s pause
        }
    }
    return 0;
}
```

Expert Tips

- Use a hardware timer for precise frequency control.
- Ensure the buzzer supports the target frequencies.
- Add a transistor for active buzzers requiring higher current.

Project 74: Create a project to control an LED strip using GPIO outputs.

Project Description: Write a C program to control a 4-LED strip using GPIO pins, displaying a chasing pattern.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Sequential GPIO control.
- **Steps**:
    1. Configure 4 GPIO pins as outputs.
    2. Define a chasing pattern (e.g., one LED on at a time).

3. Set pins according to the pattern with delays.
4. Print the pattern state.

C Code Implementation

```
#include <stdio.h>
#define LED1_PIN 0
#define LED2_PIN 1
#define LED3_PIN 2
#define LED4_PIN 3

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void set_strip(int led1, int led2, int led3, int led4) {
    printf("Strip: %d %d %d %d\n", led1, led2, led3, led4);
    // In real: PORTB = (led1 << LED1_PIN) | (led2 << LED2_PIN) | (led3 << LED3_PIN) | (led4 <<
LED4_PIN);
}


int main(void) {
    // In real: DDRB |= (1 << LED1_PIN) | (1 << LED2_PIN) | (1 << LED3_PIN) | (1 << LED4_PIN);
    printf("Chasing LED strip.\n");

    while (1) {
        set_strip(1, 0, 0, 0); delay_ms(500);
        set_strip(0, 1, 0, 0); delay_ms(500);
        set_strip(0, 0, 1, 0); delay_ms(500);
        set_strip(0, 0, 0, 1); delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- For addressable LED strips (e.g., WS2812), use precise timing protocols like bit-banging or SPI.
- Use PWM for brightness control.
- Ensure sufficient power supply for the LED strip.

Project 75: Implement a project to read a digital temperature sensor via GPIO.

Project Description: Write a C program to read a digital temperature sensor (e.g., DS18B20, simulated) via a GPIO pin and print the temperature.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bit-bang protocol (simulated).
- **Steps**:
    1. Configure GPIO pin as input/output for 1-Wire protocol.
    2. Simulate reading temperature data.
    3. Print the temperature.
    4. Add delay between readings.

C Code Implementation

```c
#include <stdio.h>
#define SENSOR_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_temp(void) {
    return 25 + (rand() % 5);  // Simulate; in real: implement 1-Wire protocol
}

int main(void) {
    // In real: Configure 1-Wire protocol
    printf("Reading temperature sensor on pin %d.\n", SENSOR_PIN);

    while (1) {
        int temp = read_temp();
        printf("Temperature: %d°C\n", temp);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- For DS18B20, implement the 1-Wire protocol with precise timing (~1us).
- Use a pull-up resistor (4.7kΩ) for the 1-Wire bus.
- Validate temperature data with checksums.

Project 76: Configure a GPIO pin to interface with a simple touch sensor.

Project Description: Write a C program to read a digital touch sensor (e.g., TTP223) via a GPIO pin and print its state.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling input.
- **Steps**:
    1. Configure GPIO pin as input.
    2. Read the sensor state (e.g., high = touched).
    3. Print the state.
    4. Add delay for readability.

C Code Implementation

```c
#include <stdio.h>
#define TOUCH_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
int read_touch(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << TOUCH_PIN)) ? 1 : 0
}
```

```
int main(void) {
    // In real: DDRB &= ~(1 << TOUCH_PIN); PORTB |= (1 << TOUCH_PIN);
    printf("Reading touch sensor on pin %d.\n", TOUCH_PIN);

    while (1) {
        int state = read_touch();
        printf("Touch sensor: %s\n", state ? "Touched" : "Not touched");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Check the sensor's output logic (active-high or active-low).
- Use interrupts for faster touch detection.
- Calibrate sensitivity via sensor configuration if available.

Project 77: Create a project to control a DC motor speed using PWM.

Project Description: Write a C program to control a DC motor's speed using software PWM on a GPIO pin with different duty cycles.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Software PWM.
- **Steps**:
    1. Configure GPIO pin as output.
    2. Define PWM period (e.g., 1ms) and duty cycles (e.g., 25%, 50%, 75%).
    3. Generate PWM by toggling the pin with appropriate timing.
    4. Cycle through speeds.

C Code Implementation

```
#include <stdio.h>
#define MOTOR_PIN 0

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

void pwm_motor(int duty_cycle, int period_us) {
    int on_time = (duty_cycle * period_us) / 100;
    int off_time = period_us - on_time;

    printf("Motor ON\n");  // In real: PORTB |= (1 << MOTOR_PIN);
    delay_us(on_time);
    printf("Motor OFF\n");  // In real: PORTB &= ~(1 << MOTOR_PIN);
    delay_us(off_time);
}

int main(void) {
    // In real: DDRB |= (1 << MOTOR_PIN);
    printf("Controlling motor speed on pin %d.\n", MOTOR_PIN);
    int duties[] = {25, 50, 75};
```

```
    while (1) {
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 100; j++) {  // Run PWM for ~100 cycles
                pwm_motor(duties[i], 1000);
            }
            printf("Speed: %d%%\n", duties[i]);
            delay_us(1000000);  // 1s per speed
        }
    }
    return 0;
}
```

Expert Tips

- Use hardware PWM (e.g., Timer1) for efficient motor control.
- Ensure the motor driver (e.g., L298N) supports PWM input.
- Test with a real motor to verify speed changes.

Project 78: Implement a project to read a limit switch and stop a motor.

Project Description: Write a C program to read a limit switch via a GPIO pin and stop a DC motor (via another GPIO pin) when the switch is activated.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling with conditional output.
- **Steps**:
    1. Configure motor pin as output and switch pin as input with pull-up.
    2. Read switch state; stop motor if switch is pressed (low).
    3. Otherwise, keep motor running.
    4. Print the state.

C Code Implementation

```
#include <stdio.h>
#define MOTOR_PIN 0
#define SWITCH_PIN 1

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_switch(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << SWITCH_PIN)) ? 0 : 1
}

int main(void) {
    // In real: DDRB |= (1 << MOTOR_PIN); DDRB &= ~(1 << SWITCH_PIN); PORTB |= (1 << SWITCH_PIN);
    printf("Motor control with limit switch.\n");

    while (1) {
        if (read_switch()) {
            printf("Limit switch activated, motor OFF\n");  // In real: PORTB &= ~(1 << MOTOR_PIN);
        } else {
            printf("Motor ON\n");  // In real: PORTB |= (1 << MOTOR_PIN);
        }
        delay_ms(500);
    }
```

```
    return 0;
}
```

Expert Tips

- Use interrupts for immediate response to limit switch activation.
- Debounce the switch to prevent false triggers.
- Ensure the motor driver circuit is properly wired.

Project 79: Configure a GPIO pin to drive a single-digit 7-segment display.

Project Description: Write a C program to display a single digit (e.g., 5) on a common-cathode 7-segment display using 7 GPIO pins.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Static GPIO output.
- **Steps**:
    1. Configure 7 GPIO pins as outputs.
    2. Define the segment pattern for a specific digit.
    3. Set the pins according to the pattern.
    4. Print the pattern.

C Code Implementation

```
#include <stdio.h>
#define DISPLAY_PORT 0

void print_binary(unsigned char value) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");
}

int main(void) {
    // Common-cathode pattern for digit 5: a,b,c,d,e,f,g
    unsigned char pattern = 0x6D;  // 01101101
    // In real: DDRB |= 0x7F; PORTB = pattern;
    printf("Displaying digit 5 on 7-segment.\n");
    print_binary(pattern);
    return 0;
}
```

Expert Tips

- Verify segment mapping with the display's datasheet.
- Use resistors for each segment to limit current.
- For dynamic displays, extend to multiplexing with additional pins.

Project 80: Create a project to control a traffic light sequence using GPIO pins.

Project Description: Write a C program to control a traffic light sequence (red, green, yellow) using three GPIO pins with appropriate timing.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: State machine with delays.
- **Steps**:
    1. Configure three GPIO pins for red, green, yellow LEDs.
    2. Define a sequence: red (5s), green (5s), yellow (2s).
    3. Set pins according to the current state.
    4. Print the state.

C Code Implementation

```c
#include <stdio.h>
#define RED_PIN 0
#define GREEN_PIN 1
#define YELLOW_PIN 2

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void set_traffic_light(int red, int green, int yellow) {
    printf("Traffic light: R=%d, G=%d, Y=%d\n", red, green, yellow);
    // In real: PORTB = (red << RED_PIN) | (green << GREEN_PIN) | (yellow << YELLOW_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << RED_PIN) | (1 << GREEN_PIN) | (1 << YELLOW_PIN);
    printf("Traffic light sequence.\n");

    while (1) {
        set_traffic_light(1, 0, 0); delay_ms(5000);  // Red
        set_traffic_light(0, 1, 0); delay_ms(5000);  // Green
        set_traffic_light(0, 0, 1); delay_ms(2000);  // Yellow
    }
    return 0;
}
```

Expert Tips

- Use a timer for precise state timing.
- Add pedestrian signals or button inputs for advanced control.
- Ensure LEDs have proper current-limiting resistors.

Project 81: Implement a project to read a PIR motion sensor via GPIO.

Project Description: Write a C program to read a PIR motion sensor via a GPIO pin and print when motion is detected.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling input.
- **Steps**:
    1. Configure GPIO pin as input.
    2. Read the sensor state (e.g., high = motion detected).
    3. Print the state.

4. Add delay for readability.

C Code Implementation

```
#include <stdio.h>
#define PIR_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_pir(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << PIR_PIN)) ? 1 : 0
}

int main(void) {
    // In real: DDRB &= ~(1 << PIR_PIN); PORTB |= (1 << PIR_PIN);
    printf("Reading PIR sensor on pin %d.\n", PIR_PIN);

    while (1) {
        int state = read_pir();
        printf("PIR sensor: %s\n", state ? "Motion detected" : "No motion");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- PIR sensors often have a warm-up time; account for it in initialization.
- Use interrupts for immediate motion detection.
- Adjust sensor sensitivity and delay via hardware settings.

Project 82: Configure a GPIO pin to interface with a simple magnetic sensor.

Project Description: Write a C program to read a magnetic sensor (e.g., hall-effect switch) via a GPIO pin and print its state.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling input.
- **Steps**:
    1. Configure GPIO pin as input with pull-up.
    2. Read the sensor state (e.g., low = magnet detected).
    3. Print the state.
    4. Add delay for readability.

C Code Implementation

```
#include <stdio.h>
#define MAGNETIC_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
int read_magnetic(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << MAGNETIC_PIN)) ? 0 : 1
}

int main(void) {
    // In real: DDRB &= ~(1 << MAGNETIC_PIN); PORTB |= (1 << MAGNETIC_PIN);
    printf("Reading magnetic sensor on pin %d.\n", MAGNETIC_PIN);

    while (1) {
        int state = read_magnetic();
        printf("Magnetic sensor: %s\n", state ? "Magnet detected" : "No magnet");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Check the sensor's output type (open-drain or push-pull).
- Use interrupts for faster response to magnetic field changes.
- Test with different magnet strengths to verify range.

Project 83: Create a project to control a stepper motor using GPIO pins.

Project Description: Write a C program to control a stepper motor using 4 GPIO pins in full-step mode to rotate in one direction.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Sequential stepping.
- **Steps**:
    1. Configure 4 GPIO pins for stepper motor coils.
    2. Define full-step sequence (e.g., 1000, 0100, 0010, 0001).
    3. Cycle through the sequence to rotate the motor.
    4. Add delay for step timing.

C Code Implementation

```
#include <stdio.h>
#define COIL1_PIN 0
#define COIL2_PIN 1
#define COIL3_PIN 2
#define COIL4_PIN 3

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void set_step(int c1, int c2, int c3, int c4) {
    printf("Step: %d %d %d %d\n", c1, c2, c3, c4);
    // In real: PORTB = (c1 << COIL1_PIN) | (c2 << COIL2_PIN) | (c3 << COIL3_PIN) | (c4 << COIL4_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << COIL1_PIN) | (1 << COIL2_PIN) | (1 << COIL3_PIN) | (1 << COIL4_PIN);
    printf("Controlling stepper motor.\n");
    unsigned char steps[] = {0x08, 0x04, 0x02, 0x01};  // Full-step sequence
```

```
    while (1) {
        for (int i = 0; i < 4; i++) {
            set_step((steps[i] >> 3) & 1, (steps[i] >> 2) & 1, (steps[i] >> 1) & 1, steps[i] & 1);
            delay_ms(10);  // Adjust for motor speed
        }
    }
    return 0;
}
```

Expert Tips

- Use a stepper driver (e.g., ULN2003) to handle coil currents.
- Adjust step delay to match motor specs and desired speed.
- Implement half-stepping for smoother rotation.

Project 84: Implement a project to read a tilt sensor and toggle an LED.

Project Description: Write a C program to read a tilt sensor via a GPIO pin and toggle an LED when the sensor is tilted.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling with edge detection.
- **Steps**:
    1. Configure tilt sensor pin as input and LED pin as output.
    2. Read sensor state; toggle LED on state change (tilt detected).
    3. Print the state.
    4. Add debounce delay.

C Code Implementation

```
#include <stdio.h>
#define TILT_PIN 0
#define LED_PIN 1
int prev_state = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_tilt(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << TILT_PIN)) ? 1 : 0
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN); DDRB &= ~(1 << TILT_PIN); PORTB |= (1 << TILT_PIN);
    int led_state = 0;
    printf("Toggling LED on tilt detection.\n");

    while (1) {
        int state = read_tilt();
        if (state && !prev_state) {
            led_state = !led_state;
            printf("Tilt detected, LED %s\n", led_state ? "ON" : "OFF");  // In real: PORTB ^= (1 <<
LED_PIN);
        }
```

```
        prev_state = state;
        delay_ms(50);
    }
    return 0;
}
```

Expert Tips

- Debounce the tilt sensor to avoid false triggers.
- Use interrupts for immediate response to tilt events.
- Verify sensor orientation and logic levels.

Project 85: Configure a GPIO pin to drive a simple LED bar graph.

Project Description: Write a C program to drive a 4-LED bar graph using GPIO pins to display levels (1 to 4 LEDs lit).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Incremental GPIO control.
- **Steps**:
    1. Configure 4 GPIO pins as outputs.
    2. Define levels (e.g., 1 LED, 2 LEDs, etc.).
    3. Cycle through levels, setting pins accordingly.
    4. Print the level.

C Code Implementation

```
#include <stdio.h>
#define LED1_PIN 0
#define LED2_PIN 1
#define LED3_PIN 2
#define LED4_PIN 3

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void set_bar(int level) {
    unsigned char pattern = (1 << level) - 1;  // e.g., level 2: 00000011
    printf("Bar level %d: %04b\n", level, pattern);
    // In real: PORTB = pattern;
}

int main(void) {
    // In real: DDRB |= (1 << LED1_PIN) | (1 << LED2_PIN) | (1 << LED3_PIN) | (1 << LED4_PIN);
    printf("Driving LED bar graph.\n");

    while (1) {
        for (int level = 1; level <= 4; level++) {
            set_bar(level);
            delay_ms(1000);
        }
        for (int level = 3; level >= 0; level--) {
            set_bar(level);
            delay_ms(1000);
        }
    }
```

```
    return 0;
}
```

Expert Tips

- Use a single port for all LEDs to simplify code.
- Add PWM for variable brightness in each LED.
- Test with a real bar graph to confirm visual effect.

Project 86: Create a project to control a fan speed using PWM on a GPIO pin.

Project Description: Write a C program to control a fan's speed using software PWM on a GPIO pin with different duty cycles.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Software PWM.
- **Steps**:
    1. Configure GPIO pin as output.
    2. Define PWM period (e.g., 1ms) and duty cycles (e.g., 25%, 50%, 100%).
    3. Generate PWM for each duty cycle.
    4. Print the speed level.

C Code Implementation

```
#include <stdio.h>
#define FAN_PIN 0

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

void pwm_fan(int duty_cycle, int period_us) {
    int on_time = (duty_cycle * period_us) / 100;
    int off_time = period_us - on_time;

    printf("Fan ON\n");  // In real: PORTB |= (1 << FAN_PIN);
    delay_us(on_time);
    printf("Fan OFF\n");  // In real: PORTB &= ~(1 << FAN_PIN);
    delay_us(off_time);
}

int main(void) {
    // In real: DDRB |= (1 << FAN_PIN);
    printf("Controlling fan speed on pin %d.\n", FAN_PIN);
    int duties[] = {25, 50, 100};

    while (1) {
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 100; j++) {  // Run PWM for ~100 cycles
                pwm_fan(duties[i], 1000);
            }
            printf("Fan speed: %d%%\n", duties[i]);
            delay_us(1000000);  // 1s per speed
        }
    }
    return 0;
}
```

Expert Tips

- Use hardware PWM for smoother fan control.
- Ensure the fan driver supports PWM (e.g., MOSFET circuit).
- Verify fan response to different duty cycles.

Project 87: Implement a project to read a hall-effect sensor via GPIO.

Project Description: Write a C program to read a hall-effect sensor via a GPIO pin and print its state (e.g., magnet detected or not).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling input.
- **Steps**:
    1. Configure GPIO pin as input with pull-up.
    2. Read the sensor state (e.g., low = magnet detected).
    3. Print the state.
    4. Add delay for readability.

C Code Implementation

```c
#include <stdio.h>
#define HALL_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_hall(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << HALL_PIN)) ? 0 : 1
}

int main(void) {
    // In real: DDRB &= ~(1 << HALL_PIN); PORTB |= (1 << HALL_PIN);
    printf("Reading hall-effect sensor on pin %d.\n", HALL_PIN);

    while (1) {
        int state = read_hall();
        printf("Hall sensor: %s\n", state ? "Magnet detected" : "No magnet");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Check the sensor's output type (e.g., open-drain requires pull-up).
- Use interrupts for real-time magnet detection.
- Test with magnets of varying strength to verify sensitivity.

Project 88: Configure a GPIO pin to interface with a simple light sensor.

Project Description: Write a C program to read a digital light sensor (e.g., phototransistor-based) via a GPIO pin and print its state.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling input.
- **Steps**:
    1. Configure GPIO pin as input.
    2. Read the sensor state (e.g., high = light detected).
    3. Print the state.
    4. Add delay for readability.

C Code Implementation

```
#include <stdio.h>
#define LIGHT_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_light(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << LIGHT_PIN)) ? 1 : 0
}

int main(void) {
    // In real: DDRB &= ~(1 << LIGHT_PIN); PORTB |= (1 << LIGHT_PIN);
    printf("Reading light sensor on pin %d.\n", LIGHT_PIN);

    while (1) {
        int state = read_light();
        printf("Light sensor: %s\n", state ? "Light detected" : "No light");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Verify the sensor's logic levels and adjust pull-up/down resistors.
- Use ADC for analog light sensors instead of digital.
- Calibrate thresholds for ambient light conditions.

Project 89: Create a project to control a buzzer with a melody using GPIO (Continued)

Project Description: Write a C program to play a simple melody on a piezo buzzer using a GPIO pin with different frequencies.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Sequential tone generation.

- **Steps**:
    1. Configure the buzzer pin as an output.
    2. Define a melody as an array of frequencies (e.g., 261Hz, 294Hz, 330Hz for C4, D4, E4) and durations.
    3. For each note, generate a square wave by toggling the GPIO pin with the appropriate period.
    4. Add delays between notes for clarity.
    5. Print the current note being played.

C Code Implementation

```c
#include <stdio.h>
#define BUZZER_PIN 0

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);  // Simulate ~1us per 10 loops
}

void play_note(int freq_hz, int duration_ms) {
    int half_period_us = 500000 / freq_hz;  // Half period in microseconds
    int cycles = (duration_ms * 1000) / (2 * half_period_us);

    for (int i = 0; i < cycles; i++) {
        printf("Buzzer ON\n");  // In real: PORTB |= (1 << BUZZER_PIN);
        delay_us(half_period_us);
        printf("Buzzer OFF\n");  // In real: PORTB &= ~(1 << BUZZER_PIN);
        delay_us(half_period_us);
    }
}

int main(void) {
    // In real: DDRB |= (1 << BUZZER_PIN);
    printf("Playing melody on buzzer.\n");

    // Simple melody: C4, D4, E4 (261Hz, 294Hz, 330Hz)
    int melody[] = {261, 294, 330};
    int durations[] = {500, 500, 500};  // Duration in ms

    while (1) {
        for (int i = 0; i < 3; i++) {
            printf("Playing note: %d Hz\n", melody[i]);
            play_note(melody[i], durations[i]);
            delay_us(100000);  // 100ms pause between notes
        }
        delay_us(1000000);  // 1s pause before repeating
    }
    return 0;
}
```

Expert Tips

- Use a hardware timer (e.g., Timer1 in AVR) for precise frequency control to avoid CPU-intensive delays.
- Store melodies in an array or struct for complex sequences.
- Ensure the piezo buzzer supports the desired frequency range; active buzzers may require a driver circuit.
- Test with an oscilloscope to verify square wave accuracy.

Project 90: Implement a project to read a vibration sensor and toggle an LED.

Project Description: Write a C program to read a vibration sensor (e.g., SW-420) via a GPIO pin and toggle an LED when vibration is detected.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling with edge detection.
- **Steps**:
    1. Configure the vibration sensor pin as input with pull-up and the LED pin as output.
    2. Read the sensor state; toggle the LED on a state change (vibration detected).
    3. Print the vibration state and LED status.
    4. Add a debounce delay to avoid false triggers.

C Code Implementation

```
#include <stdio.h>
#define VIBRATION_PIN 0
#define LED_PIN 1
int prev_state = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_vibration(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << VIBRATION_PIN)) ? 1 : 0
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN); DDRB &= ~(1 << VIBRATION_PIN); PORTB |= (1 << VIBRATION_PIN);
    int led_state = 0;
    printf("Toggling LED on vibration detection.\n");

    while (1) {
        int state = read_vibration();
        if (state && !prev_state) {  // Vibration detected (rising edge)
            led_state = !led_state;
            printf("Vibration detected, LED %s\n", led_state ? "ON" : "OFF");  // In real: PORTB ^= (1
<< LED_PIN);
        }
        prev_state = state;
        delay_ms(50);  // Debounce
    }
    return 0;
}
```

Expert Tips

- Use interrupts for faster vibration detection to avoid missing short pulses.
- Adjust the sensor's sensitivity (if adjustable) to match the application.
- Implement software debouncing or use a capacitor for hardware debouncing.
- Verify sensor output logic (active-high or active-low).

Project 91: Configure a GPIO pin to drive a single relay for appliance control.

Project Description: Write a C program to control a single relay connected to a GPIO pin, toggling it every 5 seconds to simulate appliance control.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timed toggle loop.
- **Steps**:
    1. Configure the relay pin as an output.
    2. Toggle the relay state (on/off) every 5 seconds.
    3. Print the relay state.
    4. Add delays to control timing.

C Code Implementation

```
#include <stdio.h>
#define RELAY_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << RELAY_PIN);
    printf("Controlling relay on pin %d.\n", RELAY_PIN);
    int relay_state = 0;

    while (1) {
        relay_state = !relay_state;
        printf("Relay %s\n", relay_state ? "ON" : "OFF");  // In real: PORTB = (relay_state <<
RELAY_PIN);
        delay_ms(5000);  // Toggle every 5 seconds
    }
    return 0;
}
```

Expert Tips

- Use a transistor or driver (e.g., ULN2003) for relays, as GPIO pins typically cannot handle relay coil current.
- Ensure the relay module is compatible with the MCU's logic level (e.g., 5V or 3.3V).
- Add a flyback diode across the relay coil to protect against voltage spikes.
- Test with a low-power appliance to verify operation.

Project 92: Create a project to control a 2x2 LED matrix using GPIO pins.

Project Description: Write a C program to control a 2x2 LED matrix using 4 GPIO pins, displaying a pattern (e.g., diagonal LEDs on).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Matrix scanning with GPIO control.
- **Steps**:

1. Configure 2 row pins as outputs and 2 column pins as outputs for a 2x2 matrix.
2. Define a pattern (e.g., turn on LEDs at (0,0) and (1,1)).
3. Set row and column pins to light specific LEDs.
4. Print the pattern state.

C Code Implementation

```
#include <stdio.h>
#define ROW1_PIN 0
#define ROW2_PIN 1
#define COL1_PIN 2
#define COL2_PIN 3

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void set_matrix(int row1, int row2, int col1, int col2) {
    printf("Matrix: R1=%d, R2=%d, C1=%d, C2=%d\n", row1, row2, col1, col2);
    // In real: PORTB = (row1 << ROW1_PIN) | (row2 << ROW2_PIN) | (col1 << COL1_PIN) | (col2 <<
COL2_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << ROW1_PIN) | (1 << ROW2_PIN) | (1 << COL1_PIN) | (1 << COL2_PIN);
    printf("Controlling 2x2 LED matrix.\n");

    while (1) {
        // Diagonal pattern: (0,0) and (1,1) ON
        set_matrix(1, 0, 1, 0);  // (0,0) ON
        delay_ms(500);
        set_matrix(0, 1, 0, 1);  // (1,1) ON
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- For a real LED matrix, ensure proper row/column polarity (e.g., common-anode or common-cathode).
- Use multiplexing to reduce flickering in larger matrices.
- Add current-limiting resistors for each LED.
- Test with a breadboard to confirm the pattern.

Project 93: Implement a project to read a simple proximity sensor via GPIO.

Project Description: Write a C program to read a digital proximity sensor (e.g., IR-based) via a GPIO pin and print its state.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling input.
- **Steps**:
    1. Configure the proximity sensor pin as input.
    2. Read the sensor state (e.g., high = object detected).
    3. Print the state (object detected or not).

4. Add a delay for readability.

C Code Implementation

```
#include <stdio.h>
#define PROXIMITY_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_proximity(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << PROXIMITY_PIN)) ? 1 : 0
}

int main(void) {
    // In real: DDRB &= ~(1 << PROXIMITY_PIN); PORTB |= (1 << PROXIMITY_PIN);
    printf("Reading proximity sensor on pin %d.\n", PROXIMITY_PIN);

    while (1) {
        int state = read_proximity();
        printf("Proximity sensor: %s\n", state ? "Object detected" : "No object");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Verify the sensor's output logic (active-high or active-low).
- Use interrupts for real-time detection of proximity changes.
- Adjust the sensor's range and sensitivity via hardware (e.g., potentiometer).
- Test in different lighting conditions for IR-based sensors.

Project 94: Configure a GPIO pin to interface with a reed switch.

Project Description: Write a C program to read a reed switch via a GPIO pin and print its state (e.g., magnet detected or not).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling input.
- **Steps**:
    1. Configure the reed switch pin as input with pull-up.
    2. Read the switch state (e.g., low = magnet detected).
    3. Print the state.
    4. Add a delay for readability.

C Code Implementation

```
#include <stdio.h>
#define REED_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_reed(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << REED_PIN)) ? 0 : 1
}

int main(void) {
    // In real: DDRB &= ~(1 << REED_PIN); PORTB |= (1 << REED_PIN);
    printf("Reading reed switch on pin %d.\n", REED_PIN);

    while (1) {
        int state = read_reed();
        printf("Reed switch: %s\n", state ? "Magnet detected" : "No magnet");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Reed switches are mechanical; debounce to avoid false triggers.
- Use interrupts for immediate response to magnet detection.
- Test with magnets of appropriate strength to avoid over-sensitivity.
- Ensure proper pull-up resistor value (e.g., 10kΩ).

Project 95: Create a project to control a 4-bit binary counter using GPIO pins.

Project Description: Write a C program to display a 4-bit binary counter (0 to 15) using 4 GPIO pins connected to LEDs.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Incremental GPIO output.
- **Steps**:
    1. Configure 4 GPIO pins as outputs for LEDs.
    2. Initialize a counter from 0 to 15.
    3. Set the GPIO pins to reflect the binary value of the counter.
    4. Increment the counter with a delay for visibility.
    5. Print the counter value.

C Code Implementation

```
#include <stdio.h>
#define LED1_PIN 0
#define LED2_PIN 1
#define LED3_PIN 2
#define LED4_PIN 3
```

```
void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void set_counter(int value) {
    printf("Counter: %d (Binary: %d%d%d%d)\n", value,
           (value >> 3) & 1, (value >> 2) & 1, (value >> 1) & 1, value & 1);
    // In real: PORTB = ((value & 0x0F) << LED1_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << LED1_PIN) | (1 << LED2_PIN) | (1 << LED3_PIN) | (1 << LED4_PIN);
    printf("4-bit binary counter.\n");

    while (1) {
        for (int count = 0; count <= 15; count++) {
            set_counter(count);
            delay_ms(1000);
        }
    }
    return 0;
}
```

Expert Tips

- Use a single port for all LEDs to simplify bit manipulation.
- Add a reset mechanism (e.g., button) to restart the counter.
- Ensure LEDs have current-limiting resistors.
- Test the binary pattern with a logic analyzer or visually.

Project 96: Implement a project to read a flame sensor and trigger a buzzer.

Project Description: Write a C program to read a flame sensor via a GPIO pin and activate a buzzer when a flame is detected.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling with conditional output.
- **Steps**:
    1. Configure the flame sensor pin as input and the buzzer pin as output.
    2. Read the sensor state (e.g., low = flame detected).
    3. If a flame is detected, activate the buzzer; otherwise, turn it off.
    4. Print the sensor and buzzer state.

C Code Implementation

```
#include <stdio.h>
#define FLAME_PIN 0
#define BUZZER_PIN 1

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
int read_flame(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << FLAME_PIN)) ? 0 : 1
}
```

```
int main(void) {
    // In real: DDRB |= (1 << BUZZER_PIN); DDRB &= ~(1 << FLAME_PIN); PORTB |= (1 << FLAME_PIN);
    printf("Flame sensor controlling buzzer.\n");

    while (1) {
        int state = read_flame();
        if (state) {
            printf("Flame detected, Buzzer ON\n");  // In real: PORTB |= (1 << BUZZER_PIN);
        } else {
            printf("No flame, Buzzer OFF\n");  // In real: PORTB &= ~(1 << BUZZER_PIN);
        }
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Verify the flame sensor's output logic (active-high or active-low).
- Use a timer to generate a specific buzzer frequency when activated.
- Test the sensor's range and sensitivity in a controlled environment.
- Add a delay or interrupt to avoid rapid toggling.

Project 97: Configure a GPIO pin to drive a simple RGB LED with color mixing.

Project Description: Write a C program to control an RGB LED using three GPIO pins, cycling through colors with software PWM for mixing.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Software PWM for color control.
- **Steps**:
    1. Configure three GPIO pins for R, G, B as outputs.
    2. Define PWM duty cycles for different colors (e.g., red, purple, cyan).
    3. Generate PWM signals for each color channel.
    4. Cycle through colors with delays.
    5. Print the current color.

C Code Implementation

```
#include <stdio.h>
#define RED_PIN 0
#define GREEN_PIN 1
#define BLUE_PIN 2

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

void pwm_rgb(int red_duty, int green_duty, int blue_duty, int period_us) {
    int red_on = (red_duty * period_us) / 100;
    int green_on = (green_duty * period_us) / 100;
    int blue_on = (blue_duty * period_us) / 100;

    printf("RGB: R=%d, G=%d, B=%d\n", red_duty, green_duty, blue_duty);
```

```
    // In real: PORTB |= (1 << RED_PIN); delay_us(red_on); PORTB &= ~(1 << RED_PIN);
delay_us(period_us - red_on);
    // Similar for green and blue
    delay_us(period_us);
}

int main(void) {
    // In real: DDRB |= (1 << RED_PIN) | (1 << GREEN_PIN) | (1 << BLUE_PIN);
    printf("RGB LED color mixing.\n");

    int colors[][3] = {{100, 0, 0}, {100, 0, 100}, {0, 100, 100}};  // Red, Purple, Cyan

    while (1) {
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 100; j++) {  // Run PWM for ~100 cycles
                pwm_rgb(colors[i][0], colors[i][1], colors[i][2], 1000);
            }
            delay_us(1000000);  // 1s per color
        }
    }
    return 0;
}
```

Expert Tips

- Use hardware PWM (e.g., Timer1 in AVR) for smoother color transitions.
- Ensure the RGB LED is common-cathode or common-anode and adjust logic.
- Calibrate duty cycles for balanced color mixing.
- Test with a diffuser for better color blending.

Project 98: Create a project to control a motor with forward/reverse using GPIO.

Project Description: Write a C program to control a DC motor's direction (forward, reverse, stop) using two GPIO pins via an H-bridge.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: State-based GPIO control.
- **Steps**:
    1. Configure two GPIO pins for H-bridge inputs (IN1, IN2).
    2. Define states: forward (IN1 high, IN2 low), reverse (IN1 low, IN2 high), stop (both low).
    3. Cycle through states with delays.
    4. Print the motor state.

C Code Implementation

```
#include <stdio.h>
#define IN1_PIN 0
#define IN2_PIN 1

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
void set_motor(int in1, int in2) {
    printf("Motor: IN1=%d, IN2=%d\n", in1, in2);  // In real: PORTB = (in1 << IN1_PIN) | (in2 <<
IN2_PIN);
}
```

```c
int main(void) {
    // In real: DDRB |= (1 << IN1_PIN) | (1 << IN2_PIN);
    printf("Controlling motor direction.\n");

    while (1) {
        set_motor(1, 0);  // Forward
        delay_ms(2000);
        set_motor(0, 0);  // Stop
        delay_ms(1000);
        set_motor(0, 1);  // Reverse
        delay_ms(2000);
        set_motor(0, 0);  // Stop
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use a motor driver (e.g., L298N) to handle high current.
- Add PWM for speed control in addition to direction.
- Include flyback diodes in the H-bridge circuit to protect against back-EMF.
- Test with a low-power motor to verify direction control.

Project 99: Implement a project to read a water level sensor via GPIO.

Project Description: Write a C program to read a digital water level sensor via a GPIO pin and print its state (e.g., water detected or not).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling input.
- **Steps**:
    1. Configure the water level sensor pin as input with pull-up.
    2. Read the sensor state (e.g., low = water detected).
    3. Print the state.
    4. Add a delay for readability.

C Code Implementation

```c
#include <stdio.h>
#define WATER_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_water(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << WATER_PIN)) ? 0 : 1
}

int main(void) {
    // In real: DDRB &= ~(1 << WATER_PIN); PORTB |= (1 << WATER_PIN);
    printf("Reading water level sensor on pin %d.\n", WATER_PIN);
```

```
    while (1) {
        int state = read_water();
        printf("Water level sensor: %s\n", state ? "Water detected" : "No water");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Verify the sensor's output logic (active-high or active-low).
- Use interrupts for immediate detection of water level changes.
- Protect the sensor and MCU from moisture or corrosion.
- Test with varying water levels to ensure reliability.

Project 100: Configure a GPIO pin to interface with a simple pressure sensor.

Project Description: Write a C program to read a digital pressure sensor (e.g., a binary switch type) via a GPIO pin and print its state.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling input.
- **Steps**:
    1. Configure the pressure sensor pin as input with pull-up.
    2. Read the sensor state (e.g., high = pressure detected).
    3. Print the state.
    4. Add a delay for readability.

C Code Implementation

```
#include <stdio.h>
#define PRESSURE_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_pressure(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << PRESSURE_PIN)) ? 1 : 0
}

int main(void) {
    // In real: DDRB &= ~(1 << PRESSURE_PIN); PORTB |= (1 << PRESSURE_PIN);
    printf("Reading pressure sensor on pin %d.\n", PRESSURE_PIN);

    while (1) {
        int state = read_pressure();
        printf("Pressure sensor: %s\n", state ? "Pressure detected" : "No pressure");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Check the pressure sensor's output type (digital or analog; use ADC for analog sensors).
- Use interrupts for real-time pressure detection.
- Calibrate the sensor's threshold if adjustable.
- Test under controlled pressure conditions to verify accuracy.
- **Debounce the Input Signal**
  Even digital pressure sensors (especially mechanical switch-type) can produce **bouncing** signals.
  Implement a **software debounce** (e.g., sample multiple times or use a small time filter) or use the MCU's **hardware input filter** if available to prevent false triggers.
- **Use Interrupts Instead of Polling**
  Polling wastes CPU cycles and may miss **fast pressure changes**.
  Configure the GPIO for **edge-triggered interrupts** so the MCU responds immediately when the sensor changes state.
  This reduces **power consumption** and ensures **real-time detection**.
- **Protect Against Noise and Floating Inputs**
  Enable **internal pull-ups or pull-downs** to ensure a defined logic level when the sensor is disconnected.
  If the environment is noisy (industrial settings), add **external RC filters** or **Schmitt trigger inputs** for cleaner signals.
- **Implement Error Handling & Diagnostics**
  Add checks for **stuck-at** conditions (e.g., sensor always high or always low) to detect wiring faults or sensor failure.
  Provide diagnostic messages or set error flags for higher-level software.
- **Abstract the Hardware Layer**
  Create a small **sensor driver API** (e.g., `pressure_init()`, `pressure_read()`).
  This makes the code **portable across MCUs** and allows future upgrades (switching to an **analog pressure sensor** using ADC) with minimal changes.

Timers and Interrupts

Project 101: Configure a timer to blink an LED every 1 second without a loop.

Project Description: Write a C program to configure a timer to toggle an LED every 1 second using a timer interrupt, without using a delay loop in the main function.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt-driven toggle.
- **Steps**:
    1. Configure a GPIO pin as output for the LED.
    2. Set up a timer (e.g., Timer0 in AVR) to generate an interrupt every 1 second.
    3. In the interrupt service routine (ISR), toggle the LED state.
    4. Keep the main loop empty or idle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define LED_PIN 0

volatile int led_state = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);  // Simulate delay
}

ISR(TIMER0_OVF_vect) {  // Simulated ISR; in real: Timer0 overflow
    led_state = !led_state;
    printf("LED %s\n", led_state ? "ON" : "OFF");  // In real: PORTB ^= (1 << LED_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN); // Set LED pin as output
    // In real: TCCR0B |= (1 << CS02) | (1 << CS00); // Prescaler 1024
    // In real: TIMSK0 |= (1 << TOIE0); // Enable overflow interrupt
    // In real: sei(); // Enable global interrupts
    printf("Timer blinking LED every 1s.\n");

    while (1) {
        // Simulate timer interrupt every 1s
        delay_ms(1000);
        ISR(TIMER0_OVF_vect);
    }
    return 0;
}
```

Expert Tips

- For a 1-second interval, calculate the timer prescaler and overflow count based on the MCU clock (e.g., for 16MHz AVR, prescaler 1024, Timer0 overflows every 16ms, so count ~62 overflows).
- Use CTC mode for precise timing instead of overflow.
- Ensure the LED has a current-limiting resistor.

- Minimize ISR code to avoid blocking other interrupts.

Project 102: Create a project to generate a 1 kHz PWM signal using a timer.

Project Description: Write a C program to generate a 1 kHz PWM signal on a GPIO pin using a timer in fast PWM mode.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM generation.
- **Steps**:
    1. Configure a GPIO pin as output.
    2. Set up a timer (e.g., Timer1 in AVR) in fast PWM mode with a 1 kHz frequency.
    3. Set a fixed duty cycle (e.g., 50%) for the PWM signal.
    4. Print the PWM status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); // Set PWM pin as output
    // In real: TCCR1A |= (1 << WGM11) | (1 << COM1A1); // Fast PWM, non-inverting
    // In real: TCCR1B |= (1 << WGM13) | (1 << WGM12) | (1 << CS11); // Prescaler 8
    // In real: ICR1 = 1999; // For 1kHz at 16MHz
    // In real: OCR1A = 999; // 50% duty cycle
    printf("Generating 1kHz PWM on pin %d.\n", PWM_PIN);

    while (1) {
        // Simulate PWM output
        printf("PWM HIGH\n");
        delay_ms(1);  // 1ms period (50% duty cycle)
        printf("PWM LOW\n");
        delay_ms(1);
    }
    return 0;
}
```

Expert Tips

- For a 1 kHz PWM with a 16MHz AVR, set the timer top (ICR1) to 1999 with a prescaler of 8.
- Use fast PWM mode for high-frequency signals.
- Verify the PWM signal with an oscilloscope.
- Adjust OCR1A for different duty cycles.

Project 103: Implement a project to count timer overflows and toggle an LED.

Project Description: Write a C program to count timer overflows and toggle an LED every 10 overflows using a timer interrupt.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer overflow interrupt with counter.
- **Steps**:
    1. Configure a GPIO pin as output for the LED.
    2. Set up a timer (e.g., Timer0) to generate overflow interrupts.
    3. In the ISR, increment a counter and toggle the LED after 10 overflows.
    4. Print the counter and LED state.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define LED_PIN 0
volatile int overflow_count = 0;
volatile int led_state = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER0_OVF_vect) {
    overflow_count++;
    if (overflow_count >= 10) {
        led_state = !led_state;
        printf("LED %s (Overflows: %d)\n", led_state ? "ON" : "OFF", overflow_count);
        overflow_count = 0;  // Reset counter
        // In real: PORTB ^= (1 << LED_PIN);
    }
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN); TCCR0B |= (1 << CS02); TIMSK0 |= (1 << TOIE0); sei();
    printf("Counting timer overflows to toggle LED.\n");

    while (1) {
        // Simulate timer overflow every 16ms
        delay_ms(16);
        ISR(TIMER0_OVF_vect);
    }
    return 0;
}
```

Expert Tips

- Calculate overflow frequency based on MCU clock and prescaler (e.g., 16MHz, prescaler 256 = ~61Hz for 8-bit timer).
- Use a volatile variable for overflow_count to ensure correct ISR behavior.
- Test with different overflow thresholds for varying toggle rates.
- Minimize ISR processing to avoid interrupt conflicts.

Project 104: Configure a timer to generate a 50% duty cycle PWM signal.

Project Description: Write a C program to generate a 50% duty cycle PWM signal on a GPIO pin using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM generation.
- **Steps**:
    1. Configure a GPIO pin as output.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Set the duty cycle to 50% by adjusting the compare register.
    4. Print the PWM status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1);
    // In real: TCCR1B |= (1 << WGM13) | (1 << WGM12) | (1 << CS11); ICR1 = 1999; OCR1A = 999;
    printf("Generating 50%% duty cycle PWM on pin %d.\n", PWM_PIN);

    while (1) {
        // Simulate 50% duty cycle PWM
        printf("PWM HIGH\n");
        delay_ms(1);
        printf("PWM LOW\n");
        delay_ms(1);
    }
    return 0;
}
```

Expert Tips

- For a 1 kHz PWM at 16MHz, use prescaler 8 and set ICR1 to 1999, OCR1A to 999 for 50% duty.
- Use phase-correct PWM for smoother transitions in some applications.
- Verify the signal with an oscilloscope or logic analyzer.
- Ensure the PWM pin is hardware-supported (e.g., OC1A in AVR).

Project 105: Create a project to use a timer interrupt to toggle a GPIO pin.

Project Description: Write a C program to toggle a GPIO pin every 500ms using a timer interrupt.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt-driven toggle.
- **Steps**:
    1. Configure a GPIO pin as output.
    2. Set up a timer to generate an interrupt every 500ms.
    3. In the ISR, toggle the GPIO pin.
    4. Print the pin state.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define GPIO_PIN 0
volatile int pin_state = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER0_COMPA_vect) {
    pin_state = !pin_state;
    printf("GPIO %s\n", pin_state ? "HIGH" : "LOW");  // In real: PORTB ^= (1 << GPIO_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << GPIO_PIN); TCCR0A |= (1 << WGM01); OCR0A = 124; TCCR0B |= (1 << CS02) |
(1 << CS00); TIMSK0 |= (1 << OCIE0A); sei();
    printf("Toggling GPIO every 500ms.\n");

    while (1) {
        // Simulate interrupt every 500ms
        delay_ms(500);
        ISR(TIMER0_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- Use CTC mode with OCR0A for precise 500ms interrupts (e.g., at 16MHz, prescaler 1024, OCR0A = 124 for ~500ms).
- Keep ISR code minimal to avoid delays in interrupt handling.
- Test toggle frequency with a scope or LED.
- Ensure the GPIO pin can source/sink sufficient current.

Project 106: Implement a project to measure the frequency of a square wave using a timer.

Project Description: Write a C program to measure the frequency of an external square wave using a timer in capture mode.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer input capture.
- **Steps**:
    1. Configure a GPIO pin as input for the square wave.
    2. Set up a timer (e.g., Timer1) in input capture mode to detect rising edges.
    3. In the ISR, record the capture time and calculate the period.
    4. Compute and print the frequency (1/period).

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define SIGNAL_PIN 0
volatile unsigned long last_capture = 0;
volatile unsigned long period = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_CAPT_vect) {
    unsigned long current = rand() % 10000;  // Simulate capture; in real: ICR1
    if (last_capture != 0) {
        period = current - last_capture;
        printf("Frequency: %.2f Hz\n", 1000000.0 / period);  // Assuming timer ticks in us
    }
    last_capture = current;
}

int main(void) {
    // In real: DDRB &= ~(1 << SIGNAL_PIN); TCCR1B |= (1 << ICES1) | (1 << CS11); TIMSK1 |= (1 <<
ICIE1); sei();
    printf("Measuring square wave frequency.\n");

    while (1) {
        // Simulate capture every 1ms (1kHz signal)
        delay_ms(1);
        ISR(TIMER1_CAPT_vect);
    }
    return 0;
}
```

Expert Tips

- Use input capture mode (e.g., Timer1 ICP1 pin in AVR) for accurate edge detection.
- Account for timer overflow when calculating long periods.
- Test with a known signal frequency (e.g., from a function generator).
- Ensure the input signal matches the MCU's logic levels.

Project 107: Configure a timer to generate a 1 ms periodic interrupt.

Project Description: Write a C program to configure a timer to generate an interrupt every 1 ms and print a message.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer compare interrupt.
- **Steps**:
    1. Set up a timer (e.g., Timer0) in CTC mode with a 1ms period.
    2. Enable the compare match interrupt.
    3. In the ISR, print a message.
    4. Keep the main loop idle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER0_COMPA_vect) {
    printf("1ms interrupt occurred\n");
}

int main(void) {
    // In real: TCCR0A |= (1 << WGM01); OCR0A = 15; TCCR0B |= (1 << CS01) | (1 << CS00); TIMSK0 |= (1
<< OCIE0A); sei();
    printf("Generating 1ms periodic interrupt.\n");

    while (1) {
        // Simulate 1ms interrupt
        delay_ms(1);
        ISR(TIMER0_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 1ms at 16MHz, use prescaler 64 and OCR0A = 15 for Timer0 in CTC mode.
- Use a 16-bit timer (e.g., Timer1) for longer periods or higher resolution.
- Avoid heavy processing in the ISR to maintain timing accuracy.
- Verify interrupt frequency with a logic analyzer.

Project 108: Create a project to fade an LED using PWM with a timer.

Project Description: Write a C program to fade an LED in and out using a timer's PWM output.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM with dynamic duty cycle.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Gradually increase/decrease the PWM duty cycle in the main loop.
    4. Print the duty cycle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 << WGM12)
| (1 << CS11); OCR1A = 0;
    printf("Fading LED with PWM.\n");

    int duty = 0;
    int step = 10;

    while (1) {
        for (duty = 0; duty <= 100; duty += step) {
            printf("Duty cycle: %d%%\n", duty);  // In real: OCR1A = (duty * 255) / 100;
            delay_ms(100);
        }
        for (duty = 100; duty >= 0; duty -= step) {
            printf("Duty cycle: %d%%\n", duty);  // In real: OCR1A = (duty * 255) / 100;
            delay_ms(100);
        }
    }
    return 0;
}
```

Expert Tips

- Use fast PWM mode for smooth LED fading (e.g., 8-bit resolution, OCR1A = 0-255).
- Set PWM frequency above 100Hz to avoid flicker (e.g., 490Hz with prescaler 64 at 16MHz).
- Test with an LED and resistor to verify fading effect.
- Use a lookup table for non-linear brightness curves.

Project 109: Implement a project to use a timer to create a 1-second delay.

Project Description: Write a C program to use a timer interrupt to create a 1-second delay and toggle an LED after each delay.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt with counter.
- **Steps**:
    1. Configure a GPIO pin as output for the LED.
    2. Set up a timer to generate interrupts (e.g., every 1ms).
    3. Count interrupts until 1000ms is reached, then toggle the LED.
    4. Print the LED state.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define LED_PIN 0
volatile int ms_count = 0;
volatile int led_state = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
ISR(TIMER0_COMPA_vect) {
    ms_count++;
    if (ms_count >= 1000) {  // 1 second
        led_state = !led_state;
        printf("LED %s\n", led_state ? "ON" : "OFF");  // In real: PORTB ^= (1 << LED_PIN);
        ms_count = 0;
    }
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN); TCCR0A |= (1 << WGM01); OCR0A = 15; TCCR0B |= (1 << CS01) | (1
<< CS00); TIMSK0 |= (1 << OCIE0A); sei();
    printf("1s delay with timer interrupt.\n");

    while (1) {
        // Simulate 1ms interrupt
        delay_ms(1);
        ISR(TIMER0_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- Use CTC mode for precise 1ms interrupts (e.g., OCR0A = 15, prescaler 64 at 16MHz).
- Avoid polling in the main loop; let interrupts handle timing.
- Test the delay accuracy with a stopwatch or scope.
- Consider using a 16-bit timer for longer delays.

Project 110: Configure a timer to generate a 10 kHz square wave.

Project Description: Write a C program to generate a 10 kHz square wave on a GPIO pin using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer toggle mode.
- **Steps**:
    1. Configure a GPIO pin as output.
    2. Set up a timer (e.g., Timer1) in CTC mode to toggle the pin every 50us (half period for 10kHz).
    3. Enable the timer's output compare to toggle the pin.
    4. Print the signal status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define OUTPUT_PIN 0

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << OUTPUT_PIN); TCCR1A |= (1 << COM1A0); TCCR1B |= (1 << WGM12) | (1 <<
CS10); OCR1A = 799; // 10kHz at 16MHz
    printf("Generating 10kHz square wave on pin %d.\n", OUTPUT_PIN);
```

```
    while (1) {
        // Simulate 10kHz square wave
        printf("HIGH\n");
        delay_us(50);
        printf("LOW\n");
        delay_us(50);
    }
    return 0;
}
```

Expert Tips

- For 10kHz at 16MHz, use no prescaler and set OCR1A = 799 in CTC mode.
- Use toggle mode (COM1A0) for automatic pin toggling.
- Verify the signal frequency with an oscilloscope.
- Ensure the pin supports timer output (e.g., OC1A in AVR).

Project 111: Create a project to control a servo motor using a timer's PWM.

Project Description: Write a C program to control a servo motor using a timer's PWM to set angles (0°, 90°, 180°).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM with variable pulse width.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode with a 20ms period.
    3. Adjust the PWM pulse width for 0° (1ms), 90° (1.5ms), and 180° (2ms).
    4. Cycle through angles and print the current angle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define SERVO_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << SERVO_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 <<
WGM13) | (1 << WGM12) | (1 << CS11); ICR1 = 39999; // 20ms at 16MHz
    printf("Controlling servo with PWM.\n");

    int pulses[] = {2000, 3000, 4000};  // 1ms, 1.5ms, 2ms (0°, 90°, 180°)

    while (1) {
        for (int i = 0; i < 3; i++) {
            printf("Servo angle: %d°\n", i * 90);  // In real: OCR1A = pulses[i];
            delay_ms(1000);  // Hold angle for 1s
        }
    }
    return 0;
}
```

Expert Tips

- For a 20ms period at 16MHz, use prescaler 8 and ICR1 = 39999.
- Map pulse widths (1ms to 2ms) to OCR1A values (2000 to 4000).
- Test with a real servo to verify angle accuracy.
- Ensure the servo's voltage and current requirements are met.

Project 112: Implement a project to use a timer interrupt to count button presses.

Project Description: Write a C program to count button presses using a timer interrupt to debounce and print the count.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt for debouncing, external interrupt for button.
- **Steps**:
    1. Configure a button pin for external interrupt and an LED pin as output.
    2. Set up a timer for 10ms interrupts to check button state (debouncing).
    3. In the timer ISR, check the button state and increment a counter if pressed.
    4. Print the button press count.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define BUTTON_PIN 2
volatile int button_count = 0;
volatile int button_state = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_button(void) {
    return (rand() % 2);  // Simulate; in real: (PIND & (1 << BUTTON_PIN)) ? 0 : 1
}

ISR(TIMER0_COMPA_vect) {
    static int last_state = 0;
    int current_state = read_button();
    if (current_state && !last_state) {  // Rising edge
        button_count++;
        printf("Button presses: %d\n", button_count);
    }
    last_state = current_state;
}

int main(void) {
    // In real: DDRD &= ~(1 << BUTTON_PIN); PORTD |= (1 << BUTTON_PIN); TCCR0A |= (1 << WGM01); OCR0A
= 124; TCCR0B |= (1 << CS02); TIMSK0 |= (1 << OCIE0A); sei();
    printf("Counting button presses with timer debounce.\n");

    while (1) {
        // Simulate 10ms interrupt
        delay_ms(10);
        ISR(TIMER0_COMPA_vect);
    }
    return 0;      }
```

Expert Tips

- Use a 10ms timer interrupt for reliable debouncing (e.g., OCR0A = 124, prescaler 1024 at 16MHz).
- Combine with an external interrupt (e.g., INT0) for initial button detection.
- Test with a real button to confirm debouncing.
- Use a volatile variable for button_count to ensure ISR safety.

Project 113: Configure a timer to generate a variable duty cycle PWM signal.

Project Description: Write a C program to generate a PWM signal with variable duty cycles (25%, 50%, 75%) using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM with dynamic duty cycle.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Cycle through duty cycles (25%, 50%, 75%) by updating the compare register.
    4. Print the duty cycle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 << WGM13)
| (1 << WGM12) | (1 << CS11); ICR1 = 1999;
    printf("Generating variable duty cycle PWM.\n");

    int duties[] = {25, 50, 75};

    while (1) {
        for (int i = 0; i < 3; i++) {
            printf("Duty cycle: %d%%\n", duties[i]);  // In real: OCR1A = (duties[i] * 1999) / 100;
            delay_ms(1000);  // Hold each duty cycle for 1s
        }
    }
    return 0;
}
```

Expert Tips

- For a 1 kHz PWM at 16MHz, use prescaler 8 and ICR1 = 1999; set OCR1A = 499, 999, 1499 for 25%, 50%, 75%.
- Use fast PWM mode for high-frequency signals.
- Test the PWM signal with an oscilloscope or LED.
- Ensure the PWM pin is hardware-supported (e.g., OC1A).

Project 114: Create a project to use a timer to measure pulse width.

Project Description: Write a C program to measure the pulse width of an external signal using a timer in capture mode.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer input capture for pulse width.
- **Steps**:
    1. Configure a GPIO pin as input for the signal.
    2. Set up a timer (e.g., Timer1) in input capture mode to detect rising and falling edges.
    3. In the ISR, record capture times and calculate pulse width.
    4. Print the pulse width.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define SIGNAL_PIN 0
volatile unsigned long rising_time = 0;
volatile unsigned long pulse_width = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_CAPT_vect) {
    static int edge = 0;
    unsigned long current = rand() % 10000;  // Simulate; in real: ICR1
    if (edge == 0) {  // Rising edge
        rising_time = current;
        edge = 1;
        // In real: TCCR1B &= ~(1 << ICES1); // Set to falling edge
    } else {  // Falling edge
        pulse_width = current - rising_time;
        printf("Pulse width: %lu us\n", pulse_width);
        edge = 0;
        // In real: TCCR1B |= (1 << ICES1); // Set to rising edge
    }
}

int main(void) {
    // In real: DDRB &= ~(1 << SIGNAL_PIN); TCCR1B |= (1 << ICES1) | (1 << CS11); TIMSK1 |= (1 <<
ICIE1); sei();
    printf("Measuring pulse width.\n");

    while (1) {
        // Simulate capture every 1ms
        delay_ms(1);
        ISR(TIMER1_CAPT_vect);
    }
    return 0;
}
```

Expert Tips

- Use input capture mode with a 16-bit timer for high resolution.
- Handle timer overflows for long pulses.

- Test with a known pulse width signal (e.g., from a function generator).
- Ensure the signal's voltage matches the MCU's logic levels.

Project 115: Implement a project to generate a 1 Hz signal using a timer.

Project Description: Write a C program to generate a 1 Hz square wave on a GPIO pin using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer toggle mode.
- **Steps**:
    1. Configure a GPIO pin as output.
    2. Set up a timer (e.g., Timer1) in CTC mode to toggle the pin every 500ms (half period for 1Hz).
    3. Enable the timer's output compare to toggle the pin.
    4. Print the signal status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define OUTPUT_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << OUTPUT_PIN); TCCR1A |= (1 << COM1A0); TCCR1B |= (1 << WGM12) | (1 <<
CS12) | (1 << CS10); OCR1A = 7812; // 1Hz at 16MHz
    printf("Generating 1Hz square wave on pin %d.\n", OUTPUT_PIN);

    while (1) {
        // Simulate 1Hz square wave
        printf("HIGH\n");
        delay_ms(500);
        printf("LOW\n");
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- For 1Hz at 16MHz, use prescaler 1024 and OCR1A = 7812 in CTC mode.
- Use toggle mode (COM1A0) for automatic pin toggling.
- Verify the signal with a scope or LED.
- Use a 16-bit timer for low-frequency signals.

Project 116: Configure a timer to trigger an interrupt every 500 ms.

Project Description: Write a C program to configure a timer to generate an interrupt every 500 ms and print a message.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer compare interrupt.
- **Steps**:
    1. Set up a timer (e.g., Timer1) in CTC mode with a 500ms period.
    2. Enable the compare match interrupt.
    3. In the ISR, print a message.
    4. Keep the main loop idle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    printf("500ms interrupt occurred\n");
}

int main(void) {
    // In real: TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS12) | (1 << CS10); OCR1A = 7812; TIMSK1 |=
(1 << OCIE1A); sei();
    printf("Generating 500ms periodic interrupt.\n");

    while (1) {
        // Simulate 500ms interrupt
        delay_ms(500);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 500ms at 16MHz, use prescaler 1024 and OCR1A = 7812 in CTC mode.
- Use a 16-bit timer for precise long intervals.
- Verify interrupt timing with a logic analyzer.
- Keep ISR code minimal for reliability.

Project 117: Create a project to control LED brightness with a timer's PWM.

Project Description: Write a C program to control an LED's brightness using a timer's PWM, cycling through different brightness levels.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM with dynamic duty cycle.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Cycle through duty cycles (e.g., 10%, 50%, 90%).

4. Print the brightness level.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 << WGM13)
| (1 << WGM12) | (1 << CS11); ICR1 = 1999;
    printf("Controlling LED brightness with PWM.\n");

    int duties[] = {10, 50, 90};

    while (1) {
        for (int i = 0; i < 3; i++) {
            printf("Brightness: %d%%\n", duties[i]);  // In real: OCR1A = (duties[i] * 1999) / 100;
            delay_ms(1000);
        }
    }
    return 0;
}
```

Expert Tips

- Use a PWM frequency above 100Hz to avoid flicker (e.g., 1kHz with ICR1 = 1999, prescaler 8).
- Map duty cycles to OCR1A values (e.g., 10% = 199, 50% = 999).
- Test with an LED and resistor to verify brightness changes.
- Consider non-linear duty cycle steps for perceptually uniform brightness.

Project 118: Implement a project to use a timer to create a blinking pattern.

Project Description: Write a C program to use a timer interrupt to create a blinking pattern (e.g., short-long-short) on an LED.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt with state machine.
- **Steps**:
    1. Configure a GPIO pin as output for the LED.
    2. Set up a timer to generate interrupts every 100ms.
    3. Use a state machine in the ISR to control the LED pattern (e.g., 100ms ON, 200ms OFF, 300ms ON).
    4. Print the pattern state.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define LED_PIN 0
volatile int state = 0;
volatile int led_state = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER0_COMPA_vect) {
    static int count = 0;
    count++;
    if (state == 0 && count >= 1) {  // 100ms ON
        led_state = 1; state = 1; count = 0;
        printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
    } else if (state == 1 && count >= 2) {  // 200ms OFF
        led_state = 0; state = 2; count = 0;
        printf("LED OFF\n");  // In real: PORTB &= ~(1 << LED_PIN);
    } else if (state == 2 && count >= 3) {  // 300ms ON
        led_state = 1; state = 0; count = 0;
        printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
    }
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN); TCCR0A |= (1 << WGM01); OCR0A = 124; TCCR0B |= (1 << CS02);
TIMSK0 |= (1 << OCIE0A); sei();
    printf("Blinking pattern with timer.\n");

    while (1) {
        // Simulate 100ms interrupt
        delay_ms(100);
        ISR(TIMER0_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- Use CTC mode for precise timing (e.g., OCR0A = 124, prescaler 1024 for 100ms at 16MHz).
- Implement the pattern as a state machine for flexibility.
- Test the pattern visually or with a scope.
- Add more states for complex patterns.

Project 119: Configure a timer to generate a 100 Hz PWM signal for a buzzer.

Project Description: Write a C program to generate a 100 Hz PWM signal on a GPIO pin using a timer to drive a buzzer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM generation.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode with a 100 Hz frequency.

3. Set a 50% duty cycle for the buzzer.
4. Print the PWM status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define BUZZER_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << BUZZER_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 <<
WGM13) | (1 << WGM12) | (1 << CS12); ICR1 = 6249; OCR1A = 3124;
    printf("Generating 100Hz PWM for buzzer.\n");

    while (1) {
        // Simulate 100Hz PWM
        printf("Buzzer ON\n");
        delay_ms(5);
        printf("Buzzer OFF\n");
        delay_ms(5);
    }
    return 0;
}
```

Expert Tips

- For 100Hz at 16MHz, use prescaler 256 and ICR1 = 6249, OCR1A = 3124 for 50% duty.
- Ensure the buzzer supports 100Hz; adjust frequency if needed.
- Verify the signal with an oscilloscope.
- Use a transistor if the buzzer requires high current.

Project 120: Create a project to use a timer interrupt to toggle multiple LEDs.

Project Description: Write a C program to toggle four LEDs in sequence every 500ms using a timer interrupt.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt with state machine.
- **Steps**:
    1. Configure four GPIO pins as outputs for LEDs.
    2. Set up a timer to generate interrupts every 500ms.
    3. In the ISR, advance to the next LED in the sequence and toggle it.
    4. Print the LED states.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define LED1_PIN 0
#define LED2_PIN 1
#define LED3_PIN 2
#define LED4_PIN 3
```

```
volatile int current_led = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    current_led = (current_led + 1) % 4;
    printf("LED %d ON\n", current_led + 1);
    // In real: PORTB = (1 << (LED1_PIN + current_led));
}

int main(void) {
    // In real: DDRB |= (1 << LED1_PIN) | (1 << LED2_PIN) | (1 << LED3_PIN) | (1 << LED4_PIN); TCCR1A
|= (1 << WGM12); TCCR1B |= (1 << CS12) | (1 << CS10); OCR1A = 7812; TIMSK1 |= (1 << OCIE1A); sei();
    printf("Toggling multiple LEDs with timer.\n");

    while (1) {
        // Simulate 500ms interrupt
        delay_ms(500);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- Use CTC mode for 500ms interrupts (e.g., OCR1A = 7812, prescaler 1024 at 16MHz).
- Use a single port for LEDs to simplify bit manipulation.
- Test the sequence visually or with a logic analyzer.
- Add patterns (e.g., reverse sequence) for variety.

Project 121: Implement a project to measure the period of an external signal using a timer.

Project Description: Write a C program to measure the period of an external signal using a timer in capture mode.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer input capture.
- **Steps**:
    1. Configure a GPIO pin as input for the signal.
    2. Set up a timer (e.g., Timer1) in input capture mode to detect rising edges.
    3. In the ISR, record capture times and calculate the period.
    4. Print the period.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>

#define SIGNAL_PIN 0

volatile unsigned long last_capture = 0;
volatile unsigned long period = 0;
```

```
void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_CAPT_vect) {
    unsigned long current = rand() % 10000;  // Simulate; in real: ICR1
    if (last_capture != 0) {
        period = current - last_capture;
        printf("Period: %lu us\n", period);
    }
    last_capture = current;
}

int main(void) {
    // In real: DDRB &= ~(1 << SIGNAL_PIN); TCCR1B |= (1 << ICES1) | (1 << CS11); TIMSK1 |= (1 <<
ICIE1); sei();
    printf("Measuring signal period.\n");

    while (1) {
        // Simulate capture every 1ms
        delay_ms(1);
        ISR(TIMER1_CAPT_vect);
    }
    return 0;
}
```

Expert Tips

- Use input capture mode for precise edge detection.
- Handle timer overflows for long periods.
- Test with a known signal period (e.g., from a function generator).
- Ensure the signal matches the MCU's logic levels.

Project 122: Configure a timer to generate a 25% duty cycle PWM signal.

Project Description: Write a C program to generate a 25% duty cycle PWM signal on a GPIO pin using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM generation.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Set the duty cycle to 25% by adjusting the compare register.
    4. Print the PWM status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 << WGM13)
| (1 << WGM12) | (1 << CS11); ICR1 = 1999; OCR1A = 499;
    printf("Generating 25%% duty cycle PWM on pin %d.\n", PWM_PIN);

    while (1) {
        // Simulate 25% duty cycle PWM (1kHz)
        printf("PWM HIGH\n");
        delay_ms(0.25);
        printf("PWM LOW\n");
        delay_ms(0.75);
    }
    return 0;
}
```

Expert Tips

- For 1kHz at 16MHz, use prescaler 8, ICR1 = 1999, OCR1A = 499 for 25% duty.
- Use fast PWM mode for high-frequency signals.
- Verify the duty cycle with an oscilloscope.
- Ensure the PWM pin is hardware-supported.

Project 123: Create a project to use a timer to control a fan speed.

Project Description: Write a C program to control a fan's speed using a timer's PWM, cycling through different speeds.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM with dynamic duty cycle.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Cycle through duty cycles (e.g., 25%, 50%, 75%).
    4. Print the fan speed.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define FAN_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << FAN_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 << WGM13)
| (1 << WGM12) | (1 << CS11); ICR1 = 1999;
    printf("Controlling fan speed with PWM.\n");

    int duties[] = {25, 50, 75};
```

```
    while (1) {
        for (int i = 0; i < 3; i++) {
            printf("Fan speed: %d%%\n", duties[i]);  // In real: OCR1A = (duties[i] * 1999) / 100;
            delay_ms(2000);  // Hold speed for 2s
        }
    }
    return 0;
}
```

Expert Tips

- Use a PWM frequency suitable for the fan (e.g., 25kHz for many DC fans).
- Ensure the fan driver (e.g., MOSFET) supports PWM.
- Test with a real fan to verify speed changes.
- Map duty cycles to OCR1A values for precise control.

Project 124: Implement a project to use a timer interrupt to update a counter.

Project Description: Write a C program to use a timer interrupt to increment a counter every 100ms and print the value.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer compare interrupt.
- **Steps**:
    1. Set up a timer (e.g., Timer0) in CTC mode to interrupt every 100ms.
    2. In the ISR, increment a counter.
    3. Print the counter value.
    4. Keep the main loop idle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
volatile unsigned long counter = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER0_COMPA_vect) {
    counter++;
    printf("Counter: %lu\n", counter);
}

int main(void) {
    // In real: TCCR0A |= (1 << WGM01); OCR0A = 124; TCCR0B |= (1 << CS02); TIMSK0 |= (1 << OCIE0A);
sei();
    printf("Updating counter every 100ms.\n");

    while (1) {
        // Simulate 100ms interrupt
        delay_ms(100);
        ISR(TIMER0_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 100ms at 16MHz, use prescaler 1024 and OCR0A = 124 in CTC mode.
- Use a volatile variable for the counter to ensure ISR safety.
- Test counter accuracy with a stopwatch or logic analyzer.
- Reset the counter if it overflows for long-running applications.

Project 125: Configure a timer to generate a 2 kHz square wave for a buzzer.

Project Description: Write a C program to generate a 2 kHz square wave on a GPIO pin using a timer to drive a buzzer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer toggle mode.
- **Steps**:
    1. Configure a GPIO pin as output.
    2. Set up a timer (e.g., Timer1) in CTC mode to toggle the pin every 250us (half period for 2kHz).
    3. Enable the timer's output compare to toggle the pin.
    4. Print the signal status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define BUZZER_PIN 0

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << BUZZER_PIN); TCCR1A |= (1 << COM1A0); TCCR1B |= (1 << WGM12) | (1 <<
CS11); OCR1A = 999; // 2kHz at 16MHz
    printf("Generating 2kHz square wave for buzzer.\n");

    while (1) {
        // Simulate 2kHz square wave
        printf("Buzzer ON\n");
        delay_us(250);
        printf("Buzzer OFF\n");
        delay_us(250);
    }
    return 0;
}
```

Expert Tips

- For 2kHz at 16MHz, use prescaler 8 and OCR1A = 999 in CTC mode.
- Use toggle mode (COM1A0) for automatic pin toggling.
- Verify the frequency with an oscilloscope.
- Ensure the buzzer supports 2kHz and has a proper driver circuit.

Project 126: Create a project to use a timer to create a 1-minute delay.

Project Description: Write a C program to use a timer interrupt to create a 1-minute delay and toggle an LED after each delay.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt with counter.
- **Steps**:
    1. Configure a GPIO pin as output for the LED.
    2. Set up a timer to generate interrupts every 1ms.
    3. Count 60,000 interrupts (60s) in the ISR, then toggle the LED.
    4. Print the LED state.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define LED_PIN 0
volatile unsigned long ms_count = 0;
volatile int led_state = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    ms_count++;
    if (ms_count >= 60000) {  // 1 minute
        led_state = !led_state;
        printf("LED %s\n", led_state ? "ON" : "OFF");  // In real: PORTB ^= (1 << LED_PIN);
        ms_count = 0;
    }
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN); TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS11) | (1 << CS10);
OCR1A = 249; TIMSK1 |= (1 << OCIE1A); sei();
    printf("1-minute delay with timer interrupt.\n");

    while (1) {
        // Simulate 1ms interrupt
        delay_ms(1);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 1ms at 16MHz, use prescaler 64 and OCR1A = 249 in CTC mode.
- Use a 16-bit timer for long delays to avoid frequent overflows.
- Test delay accuracy with a stopwatch.
- Consider using a real-time clock (RTC) for very long delays.

Project 127: Implement a project to use a timer interrupt to read a sensor.

Project Description: Write a C program to use a timer interrupt to read a digital sensor (e.g., PIR) every 500ms and print its state.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt for periodic polling.
- **Steps**:
    1. Configure a GPIO pin as input for the sensor.
    2. Set up a timer to generate interrupts every 500ms.
    3. In the ISR, read the sensor state and print it.
    4. Keep the main loop idle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define SENSOR_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_sensor(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << SENSOR_PIN)) ? 1 : 0
}

ISR(TIMER1_COMPA_vect) {
    int state = read_sensor();
    printf("Sensor: %s\n", state ? "Detected" : "Not detected");
}

int main(void) {
    // In real: DDRB &= ~(1 << SENSOR_PIN); TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS12) | (1 <<
CS10); OCR1A = 7812; TIMSK1 |= (1 << OCIE1A); sei();
    printf("Reading sensor every 500ms.\n");

    while (1) {
        // Simulate 500ms interrupt
        delay_ms(500);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 500ms at 16MHz, use prescaler 1024 and OCR1A = 7812 in CTC mode.
- Use interrupts for real-time sensor monitoring instead of polling.
- Test with a real sensor to verify detection.
- Ensure the sensor's output matches the MCU's logic levels.

Project 128: Configure a timer to generate a 75% duty cycle PWM signal.

Project Description: Write a C program to generate a 75% duty cycle PWM signal on a GPIO pin using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM generation.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Set the duty cycle to 75% by adjusting the compare register.
    4. Print the PWM status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 << WGM13)
| (1 << WGM12) | (1 << CS11); ICR1 = 1999; OCR1A = 1499;
    printf("Generating 75%% duty cycle PWM on pin %d.\n", PWM_PIN);

    while (1) {
        // Simulate 75% duty cycle PWM (1kHz)
        printf("PWM HIGH\n");
        delay_ms(0.75);
        printf("PWM LOW\n");
        delay_ms(0.25);
    }
    return 0;
}
```

Expert Tips

- For 1kHz at 16MHz, use prescaler 8, ICR1 = 1999, OCR1A = 1499 for 75% duty.
- Use fast PWM mode for high-frequency signals.
- Verify the duty cycle with an oscilloscope.
- Ensure the PWM pin is hardware-supported.

Project 129: Create a project to use a timer to control a motor speed.

Project Description: Write a C program to control a motor's speed using a timer's PWM, cycling through different speeds.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM with dynamic duty cycle.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Cycle through duty cycles (e.g., 25%, 50%, 75%).
    4. Print the motor speed.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define MOTOR_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << MOTOR_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 <<
WGM13) | (1 << WGM12) | (1 << CS11); ICR1 = 1999;
    printf("Controlling motor speed with PWM.\n");

    int duties[] = {25, 50, 75};

    while (1) {
        for (int i = 0; i < 3; i++) {
            printf("Motor speed: %d%%\n", duties[i]);  // In real: OCR1A = (duties[i] * 1999) / 100;
            delay_ms(2000);  // Hold speed for 2s
        }
    }
    return 0;
}
```

Expert Tips

- Use a PWM frequency suitable for the motor (e.g., 25kHz for DC motors).
- Ensure the motor driver supports PWM.
- Test with a real motor to verify speed changes.
- Map duty cycles to OCR1A for precise control.

Project 130: Implement a project to use a timer interrupt to toggle a relay.

Project Description: Write a C program to toggle a relay every 5 seconds using a timer interrupt.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt-driven toggle.
- **Steps**:
    1. Configure a GPIO pin as output for the relay.
    2. Set up a timer to generate interrupts every 1s.
    3. Count 5 interrupts in the ISR, then toggle the relay.
    4. Print the relay state.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define RELAY_PIN 0
volatile int seconds = 0;
volatile int relay_state = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++); }
```

```
ISR(TIMER1_COMPA_vect) {
    seconds++;
    if (seconds >= 5) {
        relay_state = !relay_state;
        printf("Relay %s\n", relay_state ? "ON" : "OFF");  // In real: PORTB ^= (1 << RELAY_PIN);
        seconds = 0;
    }
}

int main(void) {
    // In real: DDRB |= (1 << RELAY_PIN); TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS12) | (1 << CS10);
OCR1A = 15624; TIMSK1 |= (1 << OCIE1A); sei();
    printf("Toggling relay every 5s.\n");

    while (1) {
        // Simulate 1s interrupt
        delay_ms(1000);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 1s at 16MHz, use prescaler 1024 and OCR1A = 15624 in CTC mode.
- Use a transistor or driver for the relay due to high current.
- Test with a low-power appliance to verify relay operation.
- Add a flyback diode to protect against voltage spikes.

Project 131: Configure a timer to generate a 500 Hz PWM signal.

Project Description: Write a C program to generate a 500 Hz PWM signal on a GPIO pin using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM generation.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode with a 500 Hz frequency.
    3. Set a 50% duty cycle.
    4. Print the PWM status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 << WGM13)
| (1 << WGM12) | (1 << CS11); ICR1 = 3999; OCR1A = 1999;
    printf("Generating 500Hz PWM on pin %d.\n", PWM_PIN);
```

```
    while (1) {
        // Simulate 500Hz PWM
        printf("PWM HIGH\n");
        delay_ms(1);
        printf("PWM LOW\n");
        delay_ms(1);
    }
    return 0;
}
```

Expert Tips

- For 500Hz at 16MHz, use prescaler 8 and ICR1 = 3999, OCR1A = 1999 for 50% duty.
- Use fast PWM mode for high-frequency signals.
- Verify the signal with an oscilloscope.
- Ensure the PWM pin is hardware-supported.

Project 132: Create a project to use a timer to create a fading LED effect.

Project Description: Write a C program to create a fading LED effect using a timer's PWM, with smooth brightness transitions.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM with dynamic duty cycle.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Use a timer interrupt to gradually increase/decrease the duty cycle.
    4. Print the brightness level.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define PWM_PIN 0
volatile int duty = 0;
volatile int step = 1;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    duty += step;
    if (duty >= 100 || duty <= 0) step = -step;
    printf("Duty cycle: %d%%\n", duty);  // In real: OCR1A = (duty * 255) / 100;
}

int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 << WGM13)
| (1 << WGM12) | (1 << CS11); ICR1 = 255; TIMSK1 |= (1 << OCIE1A); sei();
    printf("Fading LED with timer PWM.\n");
```

```
    while (1) {
        // Simulate 50ms interrupt
        delay_ms(50);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- Use a PWM frequency above 100Hz to avoid flicker (e.g., 490Hz with prescaler 64).
- Update the duty cycle every 50ms for smooth fading.
- Test with an LED and resistor to verify the effect.
- Use a non-linear duty cycle curve for perceptual smoothness.

Project 133: Implement a project to measure the duration of a button press using a timer.

Project Description: Write a C program to measure the duration of a button press using a timer and print the duration.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based duration measurement.
- **Steps**:
    1. Configure a button pin as input with pull-up.
    2. Set up a timer to count in microseconds or milliseconds.
    3. Detect button press (falling edge) and start the timer; stop on release (rising edge).
    4. Print the duration.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define BUTTON_PIN 2
volatile unsigned long start_time = 0;
volatile unsigned long duration = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_button(void) {
    return (rand() % 2);  // Simulate; in real: (PIND & (1 << BUTTON_PIN)) ? 0 : 1
}

ISR(TIMER1_OVF_vect) {
    // Timer counts in background
}

int main(void) {
    // In real: DDRD &= ~(1 << BUTTON_PIN); PORTD |= (1 << BUTTON_PIN); TCCR1B |= (1 << CS11); //
Prescaler 8 for us resolution
    printf("Measuring button press duration.\n");

    int last_state = 1;
```

```
    while (1) {
        int state = read_button();
        if (!state && last_state) {  // Falling edge
            start_time = rand() % 10000;  // Simulate; in real: TCNT1
        } else if (state && !last_state) {  // Rising edge
            duration = (rand() % 10000) - start_time;  // Simulate; in real: TCNT1 - start_time
            printf("Button press duration: %lu us\n", duration);
        }
        last_state = state;
        delay_ms(10);  // Debounce
    }
    return 0;
}
```

Expert Tips

- Use a 16-bit timer for high-resolution measurements.
- Handle timer overflows for long button presses.
- Test with a real button to verify duration accuracy.
- Use interrupts for edge detection to improve responsiveness.

Project 134: Configure a timer to trigger an interrupt every 100 ms.

Project Description: Write a C program to configure a timer to generate an interrupt every 100 ms and print a message.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer compare interrupt.
- **Steps**:
    1. Set up a timer (e.g., Timer1) in CTC mode with a 100ms period.
    2. Enable the compare match interrupt.
    3. In the ISR, print a message.
    4. Keep the main loop idle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    printf("100ms interrupt occurred\n");
}

int main(void) {
    // In real: TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS12) | (1 << CS10); OCR1A = 1562; TIMSK1 |=
(1 << OCIE1A); sei();
    printf("Generating 100ms periodic interrupt.\n");
```

```
    while (1) {
        // Simulate 100ms interrupt
        delay_ms(100);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 100ms at 16MHz, use prescaler 1024 and OCR1A = 1562 in CTC mode.
- Use a 16-bit timer for precise intervals.
- Verify interrupt timing with a logic analyzer.
- Keep ISR code minimal for reliability.

Project 135: Create a project to use a timer to generate a simple tone on a buzzer.

Project Description: Write a C program to generate a 500 Hz tone on a buzzer using a timer's PWM.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM generation.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode with a 500 Hz frequency.
    3. Set a 50% duty cycle for the tone.
    4. Print the tone status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define BUZZER_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << BUZZER_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 <<
WGM13) | (1 << WGM12) | (1 << CS11); ICR1 = 3999; OCR1A = 1999;
    printf("Generating 500Hz tone on buzzer.\n");

    while (1) {
        // Simulate 500Hz PWM
        printf("Buzzer ON\n");
        delay_ms(1);
        printf("Buzzer OFF\n");
        delay_ms(1);
    }
    return 0;
}
```

Expert Tips

- For 500Hz at 16MHz, use prescaler 8 and ICR1 = 3999, OCR1A = 1999 for 50% duty.

- Ensure the buzzer supports 500Hz.
- Verify the tone with an oscilloscope or by listening.
- Use a transistor for active buzzers requiring

Project 136: Implement a project to use a timer interrupt to control a sequence of LEDs.

Project Description: Write a C program to use a timer interrupt to control a sequence of four LEDs (e.g., light each LED one at a time in a circular pattern) every 250ms.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt with state machine.
- **Steps**:
    1. Configure four GPIO pins as outputs for LEDs.
    2. Set up a timer (e.g., Timer1) to generate interrupts every 250ms.
    3. In the ISR, advance to the next LED in the sequence and turn it on (others off).
    4. Print the current LED state.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define LED1_PIN 0
#define LED2_PIN 1
#define LED3_PIN 2
#define LED4_PIN 3
volatile int current_led = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    current_led = (current_led + 1) % 4;
    printf("LED %d ON\n", current_led + 1);
    // In real: PORTB = (1 << (LED1_PIN + current_led));
}

int main(void) {
    // In real: DDRB |= (1 << LED1_PIN) | (1 << LED2_PIN) | (1 << LED3_PIN) | (1 << LED4_PIN);
    // In real: TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS12) | (1 << CS10); OCR1A = 3906; TIMSK1 |=
(1 << OCIE1A); sei();
    printf("Controlling LED sequence every 250ms.\n");

    while (1) {
        // Simulate 250ms interrupt
        delay_ms(250);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 250ms at 16MHz, use prescaler 1024 and OCR1A = 3906 in CTC mode.

- Use a single port for all LEDs to simplify bit manipulation.
- Test the sequence visually or with a logic analyzer.
- Add complex patterns (e.g., forward-backward) by modifying the state machine.

Project 137: Configure a timer to generate a 1 MHz square wave.

Project Description: Write a C program to generate a 1 MHz square wave on a GPIO pin using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer toggle mode.
- **Steps**:
    1. Configure a GPIO pin as output.
    2. Set up a timer (e.g., Timer1) in CTC mode to toggle the pin every 0.5us (half period for 1MHz).
    3. Enable the timer's output compare to toggle the pin.
    4. Print the signal status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define OUTPUT_PIN 0

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << OUTPUT_PIN); TCCR1A |= (1 << COM1A0); TCCR1B |= (1 << WGM12) | (1 <<
CS10); OCR1A = 7; // 1MHz at 16MHz
    printf("Generating 1MHz square wave on pin %d.\n", OUTPUT_PIN);

    while (1) {
        // Simulate 1MHz square wave
        printf("HIGH\n");
        delay_us(0.5);
        printf("LOW\n");
        delay_us(0.5);
    }
    return 0;
}
```

Expert Tips

- For 1MHz at 16MHz, use no prescaler and OCR1A = 7 in CTC mode.
- Use toggle mode (COM1A0) for automatic pin toggling.
- Verify the signal with a high-frequency oscilloscope.
- Ensure the MCU and pin can handle 1MHz switching.

Project 138: Create a project to use a timer to control a stepper motor step rate.

Project Description: Write a C program to control a stepper motor's step rate using a timer interrupt to pulse a GPIO pin at 100 steps per second.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt-driven pulsing.
- **Steps**:
    1. Configure a GPIO pin as output for the stepper motor step signal.
    2. Set up a timer to generate interrupts every 10ms (100Hz).
    3. In the ISR, pulse the step pin to advance the motor.
    4. Print the step count.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define STEP_PIN 0
volatile unsigned long step_count = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    printf("Step %lu\n", ++step_count);
    // In real: PORTB |= (1 << STEP_PIN); PORTB &= ~(1 << STEP_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << STEP_PIN); TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS12) | (1 << CS10);
OCR1A = 1562; TIMSK1 |= (1 << OCIE1A); sei();
    printf("Controlling stepper motor at 100 steps/s.\n");

    while (1) {
        // Simulate 10ms interrupt
        delay_ms(10);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 100Hz at 16MHz, use prescaler 1024 and OCR1A = 1562 in CTC mode.
- Use a stepper driver (e.g., A4988) to handle motor current.
- Test with a real stepper motor to verify step rate.
- Add direction control with another GPIO pin.

Project 139: Implement a project to use a timer interrupt to update a display.

Project Description: Write a C program to use a timer interrupt to update a simulated 7-segment display (or print numbers) every 1 second.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt-driven counter update.
- **Steps**:
    1. Set up a timer to generate interrupts every 1s.

2. In the ISR, increment a counter (0-9) and print it (simulating a 7-segment display).
3. Keep the main loop idle.
4. Print the display value.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
volatile int display_value = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    display_value = (display_value + 1) % 10;
    printf("Display: %d\n", display_value);
    // In real: Update 7-segment display pins
}

int main(void) {
    // In real: Configure display pins; TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS12) | (1 << CS10);
OCR1A = 15624; TIMSK1 |= (1 << OCIE1A); sei();
    printf("Updating display every 1s.\n");

    while (1) {
        // Simulate 1s interrupt
        delay_ms(1000);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 1s at 16MHz, use prescaler 1024 and OCR1A = 15624 in CTC mode.
- For real displays, use a lookup table for 7-segment patterns.
- Test with a real 7-segment display or simulator.
- Minimize ISR code to avoid delays.

Project 140: Configure a timer to generate a 10% duty cycle PWM signal.

Project Description: Write a C program to generate a 10% duty cycle PWM signal on a GPIO pin using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM generation.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Set the duty cycle to 10% by adjusting the compare register.
    4. Print the PWM status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 << WGM13)
| (1 << WGM12) | (1 << CS11); ICR1 = 1999; OCR1A = 199;
    printf("Generating 10%% duty cycle PWM on pin %d.\n", PWM_PIN);

    while (1) {
        // Simulate 10% duty cycle PWM (1kHz)
        printf("PWM HIGH\n");
        delay_ms(0.1);
        printf("PWM LOW\n");
        delay_ms(0.9);
    }
    return 0;
}
```

Expert Tips

- For 1kHz at 16MHz, use prescaler 8, ICR1 = 1999, OCR1A = 199 for 10% duty.
- Use fast PWM mode for high-frequency signals.
- Verify the duty cycle with an oscilloscope.
- Ensure the PWM pin is hardware-supported.

Project 141: Create a project to use a timer to create a heartbeat LED pattern.

Project Description: Write a C program to use a timer interrupt to create a heartbeat LED pattern (e.g., two quick flashes every 2 seconds).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt with state machine.
- **Steps**:
    1. Configure a GPIO pin as output for the LED.
    2. Set up a timer to generate interrupts every 100ms.
    3. In the ISR, implement a state machine for two quick flashes (100ms ON, 100ms OFF) followed by a 1.8s pause.
    4. Print the LED state.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define LED_PIN 0
volatile int state = 0;
volatile int led_state = 0;
```

```
void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    static int count = 0;
    count++;
    if (state == 0 && count >= 1) {  // 100ms ON
        led_state = 1; state = 1; count = 0;
        printf("LED ON\n");  // In real: PORTB |= (1 << LED_PIN);
    } else if (state == 1 && count >= 1) {  // 100ms OFF
        led_state = 0; state = 2; count = 0;
        printf("LED OFF\n");  // In real: PORTB &= ~(1 << LED_PIN);
    } else if (state == 2 && count >= 1) {  // 100ms ON
        led_state = 1; state = 3; count = 0;
        printf("LED ON\n");
    } else if (state == 3 && count >= 18) {  // 1.8s OFF
        led_state = 0; state = 0; count = 0;
        printf("LED OFF\n");
    }
}

int main(void) {
    // In real: DDRB |= (1 << LED_PIN); TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS12) | (1 << CS10);
OCR1A = 1562; TIMSK1 |= (1 << OCIE1A); sei();
    printf("Heartbeat LED pattern.\n");

    while (1) {
        // Simulate 100ms interrupt
        delay_ms(100);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 100ms at 16MHz, use prescaler 1024 and OCR1A = 1562 in CTC mode.
- Adjust timing for different heartbeat patterns.
- Test the pattern visually or with a scope.
- Use a lookup table for complex patterns.

Project 142: Implement a project to use a timer interrupt to monitor a sensor.

Project Description: Write a C program to use a timer interrupt to monitor a digital sensor (e.g., motion sensor) every 200ms and print its state.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt for periodic polling.
- **Steps**:
    1. Configure a GPIO pin as input for the sensor.
    2. Set up a timer to generate interrupts every 200ms.
    3. In the ISR, read the sensor state and print it.
    4. Keep the main loop idle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define SENSOR_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_sensor(void) {
    return (rand() % 2);  // Simulate; in real: (PINB & (1 << SENSOR_PIN)) ? 1 : 0
}

ISR(TIMER1_COMPA_vect) {
    int state = read_sensor();
    printf("Sensor: %s\n", state ? "Detected" : "Not detected");
}

int main(void) {
    // In real: DDRB &= ~(1 << SENSOR_PIN); TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS12) | (1 <<
CS10); OCR1A = 3124; TIMSK1 |= (1 << OCIE1A); sei();
    printf("Monitoring sensor every 200ms.\n");

    while (1) {
        // Simulate 200ms interrupt
        delay_ms(200);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 200ms at 16MHz, use prescaler 1024 and OCR1A = 3124 in CTC mode.
- Use interrupts for real-time sensor monitoring.
- Test with a real sensor to verify detection.
- Ensure the sensor's output matches the MCU's logic levels.

Project 143: Configure a timer to trigger an interrupt every 10 ms.

Project Description: Write a C program to configure a timer to generate an interrupt every 10 ms and print a message.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer compare interrupt.
- **Steps**:
    1. Set up a timer (e.g., Timer1) in CTC mode with a 10ms period.
    2. Enable the compare match interrupt.
    3. In the ISR, print a message.
    4. Keep the main loop idle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    printf("10ms interrupt occurred\n");
}

int main(void) {
    // In real: TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS12); OCR1A = 624; TIMSK1 |= (1 << OCIE1A);
sei();
    printf("Generating 10ms periodic interrupt.\n");

    while (1) {
        // Simulate 10ms interrupt
        delay_ms(10);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 10ms at 16MHz, use prescaler 256 and OCR1A = 624 in CTC mode.
- Use a 16-bit timer for precise intervals.
- Verify interrupt timing with a logic analyzer.
- Keep ISR code minimal for reliability.

Project 144: Create a project to use a timer to control a servo position.

Project Description: Write a C program to control a servo motor's position (0°, 45°, 90°) using a timer's PWM.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM with variable pulse width.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode with a 20ms period.
    3. Adjust the PWM pulse width for 0° (1ms), 45° (1.25ms), and 90° (1.5ms).
    4. Print the servo angle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define SERVO_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
int main(void) {
    // In real: DDRB |= (1 << SERVO_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 <<
WGM13) | (1 << WGM12) | (1 << CS11); ICR1 = 39999;
    printf("Controlling servo position.\n");

    int pulses[] = {2000, 2500, 3000};  // 1ms, 1.25ms, 1.5ms (0°, 45°, 90°)

    while (1) {
        for (int i = 0; i < 3; i++) {
            printf("Servo angle: %d°\n", i * 45);  // In real: OCR1A = pulses[i];
            delay_ms(1000);  // Hold angle for 1s
        }
    }
    return 0;
}
```

Expert Tips

- For a 20ms period at 16MHz, use prescaler 8 and ICR1 = 39999.
- Map pulse widths (1ms to 1.5ms) to OCR1A values (2000 to 3000).
- Test with a real servo to verify angle accuracy.
- Ensure the servo's voltage and current requirements are met.

Project 145: Implement a project to use a timer to generate a dual PWM signal.

Project Description: Write a C program to generate two PWM signals (e.g., 50% and 75% duty cycles) on two GPIO pins using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based dual PWM generation.
- **Steps**:
    1. Configure two GPIO pins as outputs for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Set one channel to 50% duty cycle and another to 75%.
    4. Print the PWM status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM1_PIN 0
#define PWM2_PIN 1

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << PWM1_PIN) | (1 << PWM2_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1) | (1
<< COM1B1); TCCR1B |= (1 << WGM13) | (1 << WGM12) | (1 << CS11); ICR1 = 1999; OCR1A = 999; OCR1B =
1499;
    printf("Generating dual PWM: 50%% on pin %d, 75%% on pin %d.\n", PWM1_PIN, PWM2_PIN);
```

```
    while (1) {
        // Simulate dual PWM
        printf("PWM1 HIGH, PWM2 HIGH\n");
        delay_ms(0.75);
        printf("PWM1 HIGH, PWM2 LOW\n");
        delay_ms(0.25);
        printf("PWM1 LOW, PWM2 LOW\n");
        delay_ms(0.75);
        printf("PWM1 LOW, PWM2 HIGH\n");
        delay_ms(0.25);
    }
    return 0;
}
```

Expert Tips

- For 1kHz at 16MHz, use prescaler 8, ICR1 = 1999, OCR1A = 999 (50%), OCR1B = 1499 (75%).
- Use Timer1 channels A and B for dual PWM outputs.
- Verify both signals with an oscilloscope.
- Ensure both pins support PWM (e.g., OC1A, OC1B).

Project 146: Configure a timer to generate a 200 Hz PWM signal.

Project Description: Write a C program to generate a 200 Hz PWM signal on a GPIO pin using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM generation.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode with a 200 Hz frequency.
    3. Set a 50% duty cycle.
    4. Print the PWM status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 << WGM13)
| (1 << WGM12) | (1 << CS11); ICR1 = 9999; OCR1A = 4999;
    printf("Generating 200Hz PWM on pin %d.\n", PWM_PIN);

    while (1) {
        // Simulate 200Hz PWM
        printf("PWM HIGH\n");
        delay_ms(2.5);
        printf("PWM LOW\n");
        delay_ms(2.5);
    }
    return 0;
}
```

Expert Tips

- For 200Hz at 16MHz, use prescaler 8 and ICR1 = 9999, OCR1A = 4999 for 50% duty.
- Use fast PWM mode for high-frequency signals.
- Verify the signal with an oscilloscope.
- Ensure the PWM pin is hardware-supported.

Project 147: Create a project to use a timer interrupt to toggle a buzzer.

Project Description: Write a C program to toggle a buzzer on and off every 1 second using a timer interrupt.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt-driven toggle.
- **Steps**:
    1. Configure a GPIO pin as output for the buzzer.
    2. Set up a timer to generate interrupts every 1s.
    3. In the ISR, toggle the buzzer state.
    4. Print the buzzer state.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define BUZZER_PIN 0
volatile int buzzer_state = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    buzzer_state = !buzzer_state;
    printf("Buzzer %s\n", buzzer_state ? "ON" : "OFF");  // In real: PORTB ^= (1 << BUZZER_PIN);
}

int main(void) {
    // In real: DDRB |= (1 << BUZZER_PIN); TCCR1A |= (1 << WGM12); TCCR1B |= (1 << CS12) | (1 <<
CS10); OCR1A = 15624; TIMSK1 |= (1 << OCIE1A); sei();
    printf("Toggling buzzer every 1s.\n");

    while (1) {
        // Simulate 1s interrupt
        delay_ms(1000);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 1s at 16MHz, use prescaler 1024 and OCR1A = 15624 in CTC mode.
- Use a transistor or driver for active buzzers.
- Test with a real buzzer to verify toggling.
- Ensure the buzzer's voltage requirements are met.

Project 148: Implement a project to measure the frequency of a sensor signal using a timer.

Project Description: Write a C program to measure the frequency of a sensor's square wave signal using a timer in capture mode.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer input capture.
- **Steps**:
    1. Configure a GPIO pin as input for the sensor signal.
    2. Set up a timer (e.g., Timer1) in input capture mode to detect rising edges.
    3. In the ISR, record capture times and calculate the frequency.
    4. Print the frequency.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define SENSOR_PIN 0
volatile unsigned long last_capture = 0;
volatile unsigned long period = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_CAPT_vect) {
    unsigned long current = rand() % 10000;  // Simulate; in real: ICR1
    if (last_capture != 0) {
        period = current - last_capture;
        printf("Sensor frequency: %.2f Hz\n", 1000000.0 / period);  // Assuming timer ticks in us
    }
    last_capture = current;
}

int main(void) {
    // In real: DDRB &= ~(1 << SENSOR_PIN); TCCR1B |= (1 << ICES1) | (1 << CS11); TIMSK1 |= (1 <<
ICIE1); sei();
    printf("Measuring sensor signal frequency.\n");

    while (1) {
        // Simulate capture every 1ms
        delay_ms(1);
        ISR(TIMER1_CAPT_vect);
    }
    return 0;
}
```

Expert Tips

- Use input capture mode for precise edge detection.
- Handle timer overflows for low-frequency signals.
- Test with a known signal frequency (e.g., from a function generator).
- Ensure the sensor signal matches the MCU's logic levels.

Project 149: Configure a timer to generate a 90% duty cycle PWM signal.

Project Description: Write a C program to generate a 90% duty cycle PWM signal on a GPIO pin using a timer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer-based PWM generation.
- **Steps**:
    1. Configure a GPIO pin as output for PWM.
    2. Set up a timer (e.g., Timer1) in fast PWM mode.
    3. Set the duty cycle to 90% by adjusting the compare register.
    4. Print the PWM status.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>
#define PWM_PIN 0

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    // In real: DDRB |= (1 << PWM_PIN); TCCR1A |= (1 << WGM11) | (1 << COM1A1); TCCR1B |= (1 << WGM13)
| (1 << WGM12) | (1 << CS11); ICR1 = 1999; OCR1A = 1799;
    printf("Generating 90%% duty cycle PWM on pin %d.\n", PWM_PIN);

    while (1) {
        // Simulate 90% duty cycle PWM (1kHz)
        printf("PWM HIGH\n");
        delay_ms(0.9);
        printf("PWM LOW\n");
        delay_ms(0.1);
    }
    return 0;
}
```

Expert Tips

- For 1kHz at 16MHz, use prescaler 8, ICR1 = 1999, OCR1A = 1799 for 90% duty.
- Use fast PWM mode for high-frequency signals.
- Verify the duty cycle with an oscilloscope.
- Ensure the PWM pin is hardware-supported.

Project 150: Create a project to use a timer to create a multi-stage LED sequence.

Project Description: Write a C program to use a timer interrupt to create a multi-stage LED sequence (e.g., binary counter from 0 to 7) on three LEDs every 500ms.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timer interrupt with counter.
- **Steps**:

1. Configure three GPIO pins as outputs for LEDs.
2. Set up a timer to generate interrupts every 500ms.
3. In the ISR, increment a counter (0-7) and set the LEDs to reflect its binary value.
4. Print the LED states.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>
#define LED1_PIN 0
#define LED2_PIN 1
#define LED3_PIN 2
volatile int counter = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

ISR(TIMER1_COMPA_vect) {
    counter = (counter + 1) % 8;
    printf("LEDs: %d%d%d (Counter: %d)\n",
           (counter >> 2) & 1, (counter >> 1) & 1, counter & 1, counter);
    // In real: PORTB = (counter & 0x07) << LED1_PIN;
}

int main(void) {
    // In real: DDRB |= (1 << LED1_PIN) | (1 << LED2_PIN) | (1 << LED3_PIN); TCCR1A |= (1 << WGM12);
TCCR1B |= (1 << CS12) | (1 << CS10); OCR1A = 7812; TIMSK1 |= (1 << OCIE1A); sei();
    printf("Multi-stage LED sequence (binary counter).\n");

    while (1) {
        // Simulate 500ms interrupt
        delay_ms(500);
        ISR(TIMER1_COMPA_vect);
    }
    return 0;
}
```

Expert Tips

- For 500ms at 16MHz, use prescaler 1024 and OCR1A = 7812 in CTC mode.
- Use a single port for LEDs to simplify bit manipulation.
- Test the binary pattern visually or with a logic analyzer.
- Add more LEDs or complex sequences (e.g., Gray code) for variation.

Analog-to-Digital Converter (ADC) and Sensors

Project 151: Configure an ADC to read a potentiometer value and print it to the serial console.

Project Description: Write a C program to configure an ADC channel to read the analog value from a potentiometer connected to an ADC pin and print the 10-bit value (0-1023) to the serial console.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polling-based ADC reading.
- **Steps**:
    1. Configure the ADC module (e.g., select channel, reference voltage, prescaler).
    2. In a loop, start ADC conversion, wait for completion, and read the result.
    3. Print the ADC value.
    4. Add a delay to avoid flooding the console.

C Code Implementation

```
#include <stdio.h>  // For simulation; use UART in real
// In real: #include <avr/io.h>, #include <util/delay.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    // Simulate ADC reading (0-1023)
    return (rand() % 1024);  // In real: ADMUX = (1 << REFS0) | channel; ADCSRA |= (1 << ADSC); while
(ADCSRA & (1 << ADSC)); return ADC;
}

int main(void) {
    // In real: ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0); // Enable ADC,
prescaler 128
    printf("Reading potentiometer via ADC.\n");

    while (1) {
        unsigned int value = read_adc(0);  // Channel 0
        printf("Potentiometer value: %u (0-1023)\n", value);
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- For AVR, use AVCC as reference (REFS0) and prescaler 128 for 16MHz clock.
- Connect potentiometer wiper to ADC pin (e.g., PC0), ends to VCC/GND.

- Average multiple readings to reduce noise.
- Use free-running mode for continuous conversions.

Project 152: Create a project to read a temperature sensor using an ADC.

Project Description: Write a C program to read an analog temperature sensor (e.g., LM35) using an ADC and convert the voltage to Celsius temperature.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: ADC reading with linear conversion.
- **Steps**:
    1. Read ADC value from the sensor channel.
    2. Convert ADC to voltage (V = ADC * Vref / 1024).
    3. Convert voltage to temperature (e.g., for LM35: T = V * 100).
    4. Print the temperature.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <util/delay.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC configuration as in Project 151
    printf("Reading temperature sensor via ADC.\n");

    while (1) {
        unsigned int adc = read_adc(1);  // Channel 1
        float voltage = (adc * 5.0) / 1024.0;  // 5V reference
        float temp_c = voltage * 100.0;  // LM35: 10mV/°        printf("Temperature: %.2f°C\n",
temp_c);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- LM35 outputs 10mV/°C; calibrate for your sensor.
- Use internal 1.1V reference for higher resolution if voltage < 1.1V.
- Average 8-16 readings for accuracy.
- Protect sensor from overvoltage.

Project 153: Implement a project to control an LED brightness based on an ADC reading.

Project Description: Write a C program to read a potentiometer via ADC and use the value to control LED brightness via PWM.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: ADC to PWM mapping.
- **Steps**:
    1. Read ADC value (0-1023).
    2. Map ADC to PWM duty cycle (0-255 for 8-bit PWM).
    3. Set PWM compare value to control brightness.
    4. Print the duty cycle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <util/delay.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; TCCR0A = (1 << WGM01) | (1 << COM0A1); TCCR0B = (1 << CS01); // Fast PWM,
prescaler 8
    printf("ADC controlling LED brightness.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        unsigned char pwm = (adc * 255) / 1023;  // Map to 0-255
        printf("ADC: %u, PWM duty: %u\n", adc, pwm);  // In real: OCR0A = pwm;
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Use Timer0 for 8-bit PWM on OC0A pin.
- PWM frequency ~3.9kHz with prescaler 8 at 16MHz.
- Add low-pass filter for smoother brightness.
- Test with potentiometer to verify linear control.

Project 154: Configure an ADC to read a light sensor and toggle an LED.

Project Description: Write a C program to read a light sensor (e.g., LDR) via ADC and toggle an LED if light level is below a threshold.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold-based control.
- **Steps**:
    1. Read ADC value from light sensor.
    2. If ADC < threshold (dark), turn LED on; else off.
    3. Print the light level and LED state.
    4. Add delay for stability.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; DDRB |= (1 << 0); // LED pin
    int threshold = 512;  // Adjust based on sensor
    printf("Light sensor controlling LED.\n");

    while (1) {
        unsigned int adc = read_adc(2);
        int led_on = (adc < threshold) ? 1 : 0;
        printf("Light level: %u, LED %s\n", adc, led_on ? "ON" : "OFF");  // In real: PORTB = (led_on
<< 0);
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- LDR + voltage divider: ADC low in light, high in dark.
- Calibrate threshold empirically.
- Use hysteresis to avoid toggling at threshold.
- Average readings to reduce noise.

Project 155: Create a project to read a voltage level using an ADC and display it.

Project Description: Write a C program to read an external voltage (0-5V) via ADC and display the voltage value.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Voltage scaling.
- **Steps**:
    1. Read ADC value.
    2. Convert to voltage: V = (ADC * Vref) / 1024.
    3. Print the voltage.

4. Delay for readability.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float vref = 5.0;
    printf("Reading voltage via ADC.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float voltage = (adc * vref) / 1023.0;
        printf("Voltage: %.2f V\n", voltage);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use voltage divider if input > Vref.
- Internal 1.1V reference for low voltages.
- Round to 2 decimal places for display.
- Protect ADC pin from overvoltage.

Project 156: Implement a project to read an analog joystick and print X/Y values.

Project Description: Write a C program to read an analog joystick (2-axis) using two ADC channels and print X/Y positions.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Multi-channel ADC reading.
- **Steps**:
    1. Read ADC from X and Y channels.
    2. Map to position (-512 to 512 or 0-1023).
    3. Print X and Y values.
    4. Delay for readability.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    printf("Reading analog joystick X/Y.\n");

    while (1) {
        unsigned int x = read_adc(0);  // X channel
        unsigned int y = read_adc(1);  // Y channel
        int x_pos = x - 512;  // Center at 0
        int y_pos = y - 512;
        printf("X: %d, Y: %d\n", x_pos, y_pos);
        delay_ms(200);
    }
    return 0;
}
```

Expert Tips

- Joystick center ~512; calibrate if needed.
- Use multiplexing for more channels.
- Add button reading if joystick has switch.
- Filter readings for smooth movement.

Project 157: Configure an ADC to read a thermistor and calculate temperature.

Project Description: Write a C program to read a thermistor via ADC, calculate resistance, and convert to temperature using Steinhart-Hart equation (simplified).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Resistance to temperature conversion.
- **Steps**:
    1. Read ADC from thermistor voltage divider.
    2. Calculate resistance R = (Vcc * R_fixed / V_therm) - R_fixed.
    3. Use beta equation or lookup for temperature.
    4. Print temperature.

C Code Implementation

```c
#include <stdio.h>
#include <math.h>  // For simulation
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float vcc = 5.0;
    float r_fixed = 10000.0;  // 10k fixed resistor
    float beta = 3950.0;
    float r25 = 10000.0;
    float t25 = 298.15;  // 25°C in Kelvin
    printf("Reading thermistor temperature.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float v_therm = (adc * vcc) / 1023.0;
        float r_therm = (vcc / v_therm - 1.0) * r_fixed;
        float temp_k = 1.0 / (1.0 / t25 + (1.0 / beta) * log(r_therm / r25));
        float temp_c = temp_k - 273.15;
        printf("Temperature: %.2f°C\n", temp_c);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use voltage divider: thermistor to ground, fixed to VCC.
- Calibrate beta and R25 for your thermistor.
- For precision, use Steinhart-Hart coefficients.
- Average readings and use lookup table for speed.

Project 158: Create a project to control a motor speed based on an ADC reading.

Project Description: Write a C program to read a potentiometer via ADC and control DC motor speed via PWM.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: ADC to PWM mapping.
- **Steps**:
    1. Read ADC value.
    2. Map to PWM duty cycle (0-255).
    3. Set PWM for motor pin.
    4. Print speed.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; TCCR1A = (1 << WGM11) | (1 << COM1A1); TCCR1B = (1 << WGM13) | (1 <<
CS11); ICR1 = 1999;
    printf("ADC controlling motor speed.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        unsigned int pwm = (adc * 2000) / 1023;  // Map to 0-1999
        printf("ADC: %u, Motor PWM: %u\n", adc, pwm);  // In real: OCR1A = pwm;
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Use motor driver (e.g., L298N) for PWM input.
- PWM frequency 1kHz suitable for motors.
- Add direction control with another pin.
- Test with potentiometer for smooth speed change.

Project 159: Implement a project to read a humidity sensor using an ADC.

Project Description: Write a C program to read an analog humidity sensor (e.g., resistive) via ADC and convert to percentage humidity.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Linear or lookup conversion.
- **Steps**:
    1. Read ADC from humidity sensor.
    2. Map ADC to humidity % (calibrate for sensor).
    3. Print humidity.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    printf("Reading humidity sensor via ADC.\n");

    while (1) {
        unsigned int adc = read_adc(1);
        float humidity = (adc / 1023.0) * 100.0;  // Simplified linear mapping
        printf("Humidity: %.1f%%\n", humidity);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- For resistive sensors, use voltage divider and calibrate mapping.
- Digital sensors (e.g., DHT22) preferred for accuracy.
- Average readings and apply hysteresis.
- Protect sensor from condensation.

Project 160: Configure an ADC to read a pressure sensor and print the value.

Project Description: Write a C program to read an analog pressure sensor (e.g., MPX5700AP) via ADC and convert to pressure (kPa).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Sensor-specific conversion.
- **Steps**:
    1. Read ADC.
    2. Convert to voltage.
    3. Apply sensor formula (e.g., P = (Vout - Voff) / sensitivity).
    4. Print pressure.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float vref = 5.0;
    float voff = 0.2;  // Offset voltage
    float sensitivity = 0.018;  // V/kPa for example sensor
    printf("Reading pressure sensor via ADC.\n");
```

```
    while (1) {
        unsigned int adc = read_adc(0);
        float vout = (adc * vref) / 1023.0;
        float pressure = (vout - voff) / sensitivity;
        printf("Pressure: %.1f kPa\n", pressure);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Check datasheet for offset and sensitivity.
- Use op-amp for signal conditioning if needed.
- Calibrate with known pressures.
- Average for noise reduction.

Project 161: Create a project to use an ADC to monitor battery voltage.

Project Description: Write a C program to monitor battery voltage via ADC (using voltage divider) and print the value.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Scaled voltage reading.
- **Steps**:
    1. Read ADC from divided battery voltage.
    2. Scale by divider ratio to get full voltage.
    3. Print battery voltage.
    4. Alert if low.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float vref = 5.0;
    float divider_ratio = 5.0;  // e.g., 4:1 divider
    float low_threshold = 3.3;
    printf("Monitoring battery voltage.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float v_div = (adc * vref) / 1023.0;
        float v_batt = v_div * divider_ratio;
        printf("Battery: %.2f V", v_batt);
        if (v_batt < low_threshold) printf(" (LOW!)");
```

```
        printf("\n");
        delay_ms(5000);
    }
    return 0;
}
```

Expert Tips

- Voltage divider: high resistors to minimize current draw.
- Calibrate divider ratio accurately.
- Use internal reference for stability.
- Add low-battery interrupt if needed.

Project 162: Implement a project to read an analog microphone and detect sound levels.

Project Description: Write a C program to read an analog microphone (electret) via ADC and detect sound levels (e.g., threshold for loudness).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Peak detection or RMS.
- **Steps**:
    1. Read multiple ADC samples.
    2. Calculate average or peak value.
    3. Compare to threshold for sound detection.
    4. Print level.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    int threshold = 600;
    printf("Detecting sound levels from microphone.\n");

    while (1) {
        unsigned long sum = 0;
        for (int i = 0; i < 100; i++) {
            sum += read_adc(0);
            delay_ms(1);
        }
        unsigned int avg = sum / 100;
        printf("Sound level: %u ", avg);
        if (avg > threshold) printf("(LOUD!)");
        printf("\n");
        delay_ms(100);
```

```
    }
    return 0;
}
```

Expert Tips

- Use pre-amplified electret mic module.
- ADC in free-running mode for continuous sampling.
- Calculate RMS for true sound level.
- Adjust threshold based on environment.

Project 163: Configure an ADC to read a soil moisture sensor and toggle an LED.

Project Description: Write a C program to read a soil moisture sensor via ADC and toggle an LED if soil is dry (high ADC).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold-based control.
- **Steps**:
    1. Read ADC from moisture sensor.
    2. If ADC > dry_threshold, turn LED on.
    3. Print moisture level.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; DDRB |= (1 << 0);
    int dry_threshold = 700;
    printf("Soil moisture controlling LED.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        int led_on = (adc > dry_threshold) ? 1 : 0;
        printf("Moisture: %u, LED %s\n", adc, led_on ? "ON (DRY)" : "OFF (WET)");
        // In real: PORTB = (led_on << 0);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Sensor: high ADC dry, low wet.

- Calibrate in air and water.
- Use averaging and hysteresis.
- Waterproof connections.

Project 164: Create a project to use an ADC to read a current sensor.

Project Description: Write a C program to read a current sensor (e.g., ACS712) via ADC and calculate current (A).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Offset and sensitivity scaling.
- **Steps**:
    1. Read ADC.
    2. Convert to voltage.
    3. Apply sensor formula: I = (V - Voff) / sensitivity.
    4. Print current.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float vref = 5.0;
    float voff = 2.5;  // Midpoint for bidirectional
    float sensitivity = 0.185;  // V/A for 5A sensor
    printf("Reading current sensor.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float v = (adc * vref) / 1023.0;
        float current = (v - voff) / sensitivity;
        printf("Current: %.2f A\n", current);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- ACS712: 185mV/A sensitivity, 2.5V offset.
- Average for accuracy, filter noise.
- Protect from high currents.
- Calibrate with known load.

Project 165: Implement a project to read an analog accelerometer and print values.

Project Description: Write a C program to read a 3-axis analog accelerometer (e.g., ADXL335) using three ADC channels and print X/Y/Z g values.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Multi-channel scaling.
- **Steps**:
    1. Read ADC from X, Y, Z channels.
    2. Convert each to g: g = (V - Voff) / sensitivity.
    3. Print values.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float vref = 5.0;
    float voff = 1.65;  // 3.3V supply / 2
    float sensitivity = 0.33;  // V/g
    printf("Reading accelerometer X/Y/Z.\n");

    while (1) {
        unsigned int x_adc = read_adc(0);
        unsigned int y_adc = read_adc(1);
        unsigned int z_adc = read_adc(2);
        float x_g = ( (x_adc * vref / 1023.0) - voff ) / sensitivity;
        float y_g = ( (y_adc * vref / 1023.0) - voff ) / sensitivity;
        float z_g = ( (z_adc * vref / 1023.0) - voff ) / sensitivity;
        printf("X: %.2f g, Y: %.2f g, Z: %.2f g\n", x_g, y_g, z_g);
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- ADXL335: 3.3V supply, 330mV/g sensitivity.
- Calibrate offsets for each axis.
- Average multiple readings.
- Use level shifter if MCU is 5V.

Project 166: Configure an ADC to read a gas sensor and trigger an alarm.

Project Description: Write a C program to read a gas sensor (e.g., MQ-2) via ADC and trigger a buzzer if gas level exceeds threshold.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold-based alarm.
- **Steps**:
    1. Read ADC from gas sensor.
    2. If ADC > threshold, activate buzzer.
    3. Print level and alarm state.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; DDRB |= (1 << 1); // Buzzer pin
    int threshold = 600;
    printf("Gas sensor with alarm.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        int alarm = (adc > threshold) ? 1 : 0;
        printf("Gas level: %u, Alarm %s\n", adc, alarm ? "ON" : "OFF");
        // In real: PORTB = (alarm << 1);
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- MQ-2: preheat 48h, calibrate in clean air.
- Use load resistor in voltage divider.
- Hysteresis for stable alarm.
- Ventilate during testing.

Project 167: Create a project to use an ADC to read a light-dependent resistor (LDR).

Project Description: Write a C program to read an LDR via ADC in a voltage divider and print light level (%).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Inverse mapping for light.
- **Steps**:
    1. Read ADC (high in dark, low in light).
    2. Map to light %: light = 100 - (adc / 10.23).
    3. Print level.
    4. Delay.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    printf("Reading LDR light level.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float light_percent = 100.0 * (1023.0 - adc) / 1023.0;  // Inverted
        printf("Light level: %.1f%%\n", light_percent);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Voltage divider: LDR to VCC, fixed to GND for light-sensitive.
- Calibrate with lux meter.
- Average and smooth readings.
- Shield from ambient light if needed.

Project 168: Implement a project to read an analog temperature sensor and control a fan.

Project Description: Write a C program to read temperature via ADC and control fan PWM if above threshold.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold PWM control.
- **Steps**:
    1. Read temperature from ADC.
    2. If T > 30°C, set PWM based on T; else 0.
    3. Print temp and PWM.
    4. Delay.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC and PWM config
    float threshold = 30.0;
    printf("Temperature controlling fan.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float temp = (adc * 5.0 / 1023.0) * 100.0;  // LM35
        unsigned char pwm = 0;
        if (temp > threshold) {
            pwm = (temp - threshold) * 10;  // Scale
        }
        printf("Temp: %.1f°C, Fan PWM: %u\n", temp, pwm);  // In real: OCR1A = pwm;
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- PWM for fan speed control.
- Hysteresis for threshold.
- Calibrate sensor.
- Use heat sink for fan.

Project 169: Configure an ADC to read a force sensor and print the value.

Project Description: Write a C program to read a force-sensitive resistor (FSR) via ADC and print force (arbitrary units).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Resistance to force mapping.
- **Steps**:
    1. Read ADC from FSR divider.
    2. Map to force level.
    3. Print value.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    printf("Reading force sensor.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float force = 1023.0 - adc;  // Inverse
        printf("Force: %.0f units\n", force);
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- FSR: low resistance under force.
- Use op-amp for better linearity.
- Calibrate with known weights.
- Debounce for stable readings.

Project 170: Create a project to use an ADC to monitor a solar panel voltage.

Project Description: Write a C program to monitor solar panel voltage via ADC (with divider) and print the value.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Scaled voltage reading.
- **Steps**:
    1. Read divided voltage.
    2. Scale to full panel voltage.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float vref = 5.0;
    float divider = 20.0;  // For 100V panel
    printf("Monitoring solar panel voltage.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float v_panel = (adc * vref / 1023.0) * divider;
        printf("Solar voltage: %.1f V\n", v_panel);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- High divider for high voltage.
- Use protection diodes.
- Average for stability.
- Log max power point.

Project 171: Implement a project to read an analog IR sensor and detect proximity.

Project Description: Write a C program to read an analog IR proximity sensor via ADC and detect object distance (arbitrary).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold or linear mapping.
- **Steps**:
    1. Read ADC (higher closer).
    2. Map to distance or detect if > threshold.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}
```

```
int main(void) {
    // In real: ADC config
    int close_threshold = 700;
    printf("IR proximity detection.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        int distance = 1023 - adc;  // Inverse
        printf("Distance: %u units", distance);
        if (adc > close_threshold) printf(" (CLOSE)");
        printf("\n");
        delay_ms(200);
    }
    return 0;
}
```

Expert Tips

- IR sensor: higher ADC closer.
- Calibrate distance.
- Average to reduce noise.
- Adjust LED current for range.

Project 172: Configure an ADC to read a pH sensor and print the value.

Project Description: Write a C program to read a pH sensor via ADC (with op-amp) and convert to pH (0-14).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Linear pH conversion.
- **Steps**:
    1. Read ADC.
    2. Convert to voltage.
    3. pH = 7 - ((V - 2.5) / 0.18) // Nernst equation approx.
    4. Print pH.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float vref = 5.0;
    float slope = 0.059;  // 59mV/pH at 25°    printf("Reading pH sensor.\n");




    while (1) {
```

```
        unsigned int adc = read_adc(0);
        float v = (adc * vref) / 1023.0;
        float ph = 7.0 - ((v - 2.5) / (slope / 2.0));  // Simplified
        printf("pH: %.2f\n", ph);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use high-impedance op-amp for buffering.
- Calibrate with pH 4/7/10 buffers.
- Temperature compensation needed.
- Clean probe regularly.

Project 173: Create a project to use an ADC to read a flex sensor.

Project Description: Write a C program to read a flex sensor via ADC and print bend angle or level.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Linear mapping.
- **Steps**:
    1. Read ADC (changes with flex).
    2. Map to angle or % bend.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    printf("Reading flex sensor.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float bend = (adc / 1023.0) * 90.0;  // 0-90 degrees
        printf("Bend angle: %.1f°\n", bend);
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Flex sensor in voltage divider.
- Calibrate straight and max bend.
- Nonlinear; use lookup table.
- Avoid over-flexing sensor.

Project 174: Implement a project to read an analog heart rate sensor.

Project Description: Write a C program to read an analog heart rate sensor (e.g., pulse oximeter module) via ADC and detect beats.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Peak detection.
- **Steps**:
    1. Read multiple ADC samples.
    2. Detect peaks above threshold.
    3. Calculate BPM from peak intervals.
    4. Print BPM.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return 512 + (rand() % 100 - 50);  // Simulate pulse
}

int main(void) {
    // In real: ADC config
    int threshold = 550;
    printf("Reading heart rate.\n");

    while (1) {
        unsigned long sum = 0;
        for (int i = 0; i < 100; i++) {
            sum += read_adc(0);
            delay_ms(10);
        }
        unsigned int avg = sum / 100;
        if (avg > threshold) {
            printf("Beat detected! BPM: ~80 (simulated)\n");
        }
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Use MAX30100 module for better accuracy.
- Implement proper peak detection algorithm.
- Filter DC offset.
- Calibrate with known BPM.

Project 175: Configure an ADC to read a strain gauge and calculate force.

Project Description: Write a C program to read a strain gauge (with Wheatstone bridge and amp) via ADC and calculate force.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bridge imbalance to strain.
- **Steps**:
    1. Read ADC from amplified bridge.
    2. Calculate strain from voltage difference.
    3. Convert to force.
    4. Print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float gain = 1000.0;  // Amplifier gain
    float v_excite = 5.0;
    printf("Reading strain gauge.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float delta_v = ((adc / 1023.0) * v_excite - v_excite / 2.0) * gain;
        float strain = delta_v / (v_excite * 2.0);  // Simplified
        float force = strain * 1000.0;  // Arbitrary
        printf("Force: %.2f units\n", force);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use HX711 for digital strain gauge.
- Balance Wheatstone bridge.
- Calibrate with known weights.

- High resolution ADC preferred.

Project 176: Create a project to use an ADC to read a color sensor.

Project Description: Write a C program to read a color sensor (e.g., TCS3200 analog output) via ADC and print RGB values.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Multi-channel color reading.
- **Steps**:
    1. Read ADC from R, G, B channels.
    2. Map to 0-255.
    3. Print RGB.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    printf("Reading color sensor RGB.\n");

    while (1) {
        unsigned int r = read_adc(0);
        unsigned int g = read_adc(1);
        unsigned int b = read_adc(2);
        printf("RGB: (%u, %u, %u)\n", r, g, b);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- TCS230/320: digital, but analog version possible.
- Calibrate with white/black reference.
- Average for stability.
- Use LED illumination.

Project 177: Implement a project to read an analog tilt sensor and toggle an LED.

Project Description: Write a C program to read an analog tilt sensor (e.g., ball switch) via ADC and toggle LED on tilt.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold detection.
- **Steps**:
    1. Read ADC.
    2. If > threshold, toggle LED.
    3. Print tilt state.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; DDRB |= (1 << 0);
    int tilt_threshold = 600;
    int led_state = 0;
    printf("Tilt sensor controlling LED.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        if (adc > tilt_threshold) {
            led_state = !led_state;
            printf("Tilt detected, LED %s\n", led_state ? "ON" : "OFF");
            // In real: PORTB ^= (1 << 0);
        }
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Analog tilt: mercury or ball, ADC changes with angle.
- Calibrate thresholds.
- Debounce with delay.
- Use digital tilt for simplicity.

Project 178: Configure an ADC to read a water level sensor and print the value.

Project Description: Write a C program to read an analog water level sensor via ADC and print level (%).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Linear mapping.
- **Steps**:
    1. Read ADC.

2. Map to % level.
3. Print.
4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    printf("Reading water level.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float level = (adc / 1023.0) * 100.0;
        printf("Water level: %.1f%%\n", level);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Capacitive or resistive sensor.
- Calibrate empty/full.
- Waterproof ADC pin.
- Average readings.

Project 179: Create a project to use an ADC to monitor a battery charge level.

Project Description: Write a C program to monitor battery charge via ADC (voltage to SOC mapping) and print % charge.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Voltage to SOC curve.
- **Steps**:
    1. Read battery voltage.
    2. Map to % SOC (simplified linear).
    3. Print %.
    4. Delay.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float v_min = 3.0;
    float v_max = 4.2;  // Li-ion
    printf("Battery charge level.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float v = (adc * 5.0 / 1023.0) * 5.0;  // Assume divider
        float soc = ((v - v_min) / (v_max - v_min)) * 100.0;
        soc = (soc < 0) ? 0 : (soc > 100 ? 100 : soc);
        printf("Charge: %.1f%%\n", soc);
        delay_ms(5000);
    }
    return 0;
}
```

Expert Tips

- Use lookup table for accurate SOC curve.
- Divider for high voltage batteries.
- Temperature compensation.
- Low-power sampling.

Project 180: Implement a project to read an analog UV sensor and print the value.

Project Description: Write a C program to read an analog UV sensor (e.g., VEML6070 analog) via ADC and print UV index.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Sensor calibration mapping.
- **Steps**:
    1. Read ADC.
    2. Map to UV index.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    printf("Reading UV sensor.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float uv_index = adc / 100.0;  // Simplified
        printf("UV Index: %.1f\n", uv_index);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Calibrate with known UV levels.
- Digital UV sensors preferred.
- Filter ambient light.
- Warn for high UV.

Project 181: Configure an ADC to read a temperature sensor with calibration.

Project Description: Write a C program to read temperature sensor with calibration curve and print calibrated value.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Polynomial calibration.
- **Steps**:
    1. Read raw ADC.
    2. Apply calibration: T = a*ADC^2 + b*ADC + c.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float a = 0.0001;  // Calibration coefficients
    float b = 0.5;
    float c = -10.0;
    printf("Calibrated temperature reading.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float temp = a * adc * adc + b * adc + c;
        printf("Calibrated Temp: %.2f°C\n", temp);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Determine coefficients from calibration points.
- Use 2-point linear for simple sensors.
- Store in EEPROM.
- Recalibrate periodically.

Project 182: Create a project to use an ADC to control a servo based on a potentiometer.

Project Description: Write a C program to read potentiometer via ADC and map to servo angle via PWM.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: ADC to PWM mapping.
- **Steps**:
    1. Read ADC (0-1023).
    2. Map to pulse width (1-2ms).
    3. Set PWM for servo.
    4. Print angle.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC and Timer1 PWM config for servo
    printf("Potentiometer controlling servo.\n");
```

```
    while (1) {
        unsigned int adc = read_adc(0);
        int angle = (adc * 180) / 1023;
        unsigned int pulse = 1000 + (angle * 1000 / 180);  // 1-2ms
        printf("Angle: %d°, Pulse: %u us\n", angle, pulse);  // In real: OCR1A = pulse * scale;
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Servo PWM: 20ms period, 1-2ms pulse.
- Smooth mapping to avoid jitter.
- Calibrate pot to servo range.
- Use interrupt for stable PWM.

Project 183: Implement a project to read an analog pressure sensor and trigger a buzzer.

Project Description: Write a C program to read pressure sensor via ADC and trigger buzzer if above threshold.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold alarm.
- **Steps**:
    1. Read pressure from ADC.
    2. If > threshold, buzzer on.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; DDRB |= (1 << 1);
    float threshold = 100.0;  // kPa
    printf("Pressure sensor with buzzer alarm.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float pressure = adc * 0.1;  // Simplified
        int alarm = (pressure > threshold) ? 1 : 0;
        printf("Pressure: %.1f kPa, Alarm %s\n", pressure, alarm ? "ON" : "OFF");
        // In real: PORTB = (alarm << 1);
        delay_ms(500);
```

```
    }
    return 0;
}
```

Expert Tips

- Calibrate sensor threshold.
- Hysteresis for alarm.
- Use tone for buzzer.
- Log high pressure events.

Project 184: Configure an ADC to read a vibration sensor and print the value.

Project Description: Write a C program to read an analog vibration sensor via ADC and print intensity.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Peak or RMS calculation.
- **Steps**:
    1. Read multiple samples.
    2. Calculate RMS or peak.
    3. Print vibration level.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
#include <math.h>  // For simulation
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return 512 + (rand() % 200 - 100);  // Simulate vibration
}

int main(void) {
    // In real: ADC config
    printf("Reading vibration sensor.\n");

    while (1) {
        float rms = 0.0;
        for (int i = 0; i < 100; i++) {
            int adc = read_adc(0);
            rms += (adc - 512) * (adc - 512);
            delay_ms(1);
        }
        rms = sqrt(rms / 100.0);
        printf("Vibration RMS: %.1f\n", rms);
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Use accelerometer for true vibration.
- High sampling rate for frequency analysis.
- Filter DC offset.
- Threshold for detection.

Project 185: Create a project to use an ADC to read a light sensor and control a relay.

Project Description: Write a C program to read light sensor via ADC and control relay for light on/off.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold control.
- **Steps**:
    1. Read light ADC.
    2. If low light, relay on.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; DDRB |= (1 << 1); // Relay
    int low_light = 400;
    printf("Light sensor controlling relay.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        int relay_on = (adc < low_light) ? 1 : 0;
        printf("Light: %u, Relay %s\n", adc, relay_on ? "ON" : "OFF");
        // In real: PORTB = (relay_on << 1);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Hysteresis for stability.
- Use optocoupler for relay.
- Calibrate light threshold.
- Power relay properly.

Project 186: Implement a project to read an analog temperature sensor and log data.

Project Description: Write a C program to read temperature via ADC and log values to serial (simulated array).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Data logging.
- **Steps**:
    1. Read temp.
    2. Store in array or print with timestamp.
    3. Print log.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float log[10];
    int index = 0;
    printf("Logging temperature data.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float temp = (adc * 5.0 / 1023.0) * 100.0;
        log[index % 10] = temp;
        index++;
        printf("Log[%d]: %.2f°C\n", index-1, temp);
        if (index % 10 == 0) {
            printf("Last 10 readings: ");
            for (int i = 0; i < 10; i++) printf("%.1f ", log[i]);
            printf("\n");
        }
        delay_ms(5000);
    }
    return 0;
}
```

Expert Tips

- Use EEPROM or SD card for persistent log.
- Add timestamp with RTC.
- Calculate average/min/max.
- Overflow protection for array.

Project 187: Configure an ADC to read a current sensor and calculate power.

Project Description: Write a C program to read current via ADC, read voltage, calculate power (P = V*I).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Power calculation.
- **Steps**:
    1. Read current ADC.
    2. Read voltage ADC.
    3. Convert to I and V.
    4. P = V * I; print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    printf("Calculating power from current and voltage.\n");

    while (1) {
        unsigned int i_adc = read_adc(0);
        unsigned int v_adc = read_adc(1);
        float current = (i_adc - 512) * 5.0 / 1023.0 / 0.185;  // ACS712
        float voltage = v_adc * 5.0 / 1023.0 * 10.0;  // Divided
        float power = voltage * current;
        printf("Power: %.2f W (V=%.1f, I=%.2f)\n", power, voltage, current);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Separate ADCs for V and I.
- Calibrate sensors.
- RMS for AC power.
- Safety for high voltage.

Project 188: Create a project to use an ADC to read a gas sensor and display levels.

Project Description: Write a C program to read gas sensor via ADC and display level on LEDs or serial bar.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Level mapping to display.
- **Steps**:
    1. Read gas ADC.
    2. Map to level 0-5.
    3. Print bar or set LEDs.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; DDRB |= 0x1F; // 5 LEDs
    printf("Gas level display.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        int level = (adc * 5) / 1024;
        printf("Gas level: ");
        for (int i = 0; i < level; i++) printf("*** ");
        printf("\n");  // In real: PORTB = (1 << level) - 1;
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- 5 LEDs for levels.
- Exponential mapping for sensitivity.
- Calibrate safe/danger levels.
- Add buzzer for high.

Project 189: Implement a project to read an analog accelerometer and detect motion.

Project Description: Write a C program to read accelerometer via ADC and detect motion if change > threshold.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Change detection.
- **Steps**:
    1. Read X/Y/Z.
    2. Calculate magnitude or delta.

3. If > threshold, motion detected.
4. Print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float threshold = 0.5;
    float prev_mag = 0;
    printf("Motion detection from accelerometer.\n");

    while (1) {
        unsigned int x = read_adc(0);
        unsigned int y = read_adc(1);
        unsigned int z = read_adc(2);
        float mag = sqrt( (x-512)*(x-512) + (y-512)*(y-512) + (z-512)*(z-512) ) / 512.0;
        float delta = fabs(mag - prev_mag);
        printf("Motion: %.2f g, Delta: %.2f", mag, delta);
        if (delta > threshold) printf(" (DETECTED!)");
        printf("\n");
        prev_mag = mag;
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Normalize to 1g at rest.
- Use low-pass filter for baseline.
- Adjustable threshold.
- Digital accel for 3-axis ease.

Project 190: Configure an ADC to read a humidity sensor and control a fan.

Project Description: Write a C program to read humidity via ADC and control fan PWM if high.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold PWM.
- **Steps**:
    1. Read humidity.
    2. If > 70%, PWM = (H - 70)*10.
    3. Print.
    4. Delay.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC and PWM config
    float threshold = 70.0;
    printf("Humidity controlling fan.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float humidity = (adc / 1023.0) * 100.0;
        unsigned char pwm = 0;
        if (humidity > threshold) {
            pwm = (humidity - threshold) * 3;  // Scale
        }
        printf("Humidity: %.1f%%, Fan PWM: %u\n", humidity, pwm);  // In real: OCR1A = pwm;
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Hysteresis for fan on/off.
- Calibrate sensor.
- PWM frequency for fan.
- Integrate with dehumidifier.

Project 191: Create a project to use an ADC to read a force sensor and toggle an LED.

Project Description: Write a C program to read force sensor via ADC and toggle LED if force > threshold.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold toggle.
- **Steps**:
    1. Read force.
    2. If > threshold, toggle LED.
    3. Print.
    4. Delay.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; DDRB |= (1 << 0);
    int force_threshold = 600;
    int led_state = 0;
    printf("Force sensor toggling LED.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        if (adc > force_threshold) {
            led_state = !led_state;
            printf("Force detected, LED %s\n", led_state ? "ON" : "OFF");
            // In real: PORTB ^= (1 << 0);
        }
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Debounce force detection.
- Calibrate threshold.
- Use for switch or alarm.
- FSR nonlinearity.

Project 192: Implement a project to read an analog IR sensor and control a motor.

Project Description: Write a C program to read IR sensor via ADC and control motor speed based on proximity.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Inverse distance to speed.
- **Steps**:
    1. Read IR ADC (higher closer).
    2. Map to speed PWM.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC and PWM config
    printf("IR proximity controlling motor.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        unsigned char pwm = adc / 4;  // Map 0-1023 to 0-255
        printf("Proximity: %u, Motor speed: %u\n", adc, pwm);  // In real: OCR1A = pwm;
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Higher ADC = closer = higher speed.
- Nonlinear distance; calibrate.
- Add direction.
- Test range.

Project 193: Configure an ADC to read a temperature sensor and trigger an alarm.

Project Description: Write a C program to read temperature via ADC and trigger alarm buzzer if > threshold.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold alarm.
- **Steps**:
    1. Read temp.
    2. If > 50°C, buzzer on.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; DDRB |= (1 << 1);
    float threshold = 50.0;
    printf("Temperature alarm.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float temp = (adc * 5.0 / 1023.0) * 100.0;
        int alarm = (temp > threshold) ? 1 : 0;
        printf("Temp: %.1f°C, Alarm %s\n", temp, alarm ? "ON" : "OFF");
        // In real: PORTB = (alarm << 1);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Hysteresis to prevent chatter.
- Tone for alarm.
- Log high temps.
- Sensor placement.

Project 194: Create a project to use an ADC to read a light sensor and adjust PWM.

Project Description: Write a C program to read light sensor via ADC and adjust LED PWM inversely (dim in light).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Inverse mapping.
- **Steps**:
    1. Read light ADC.
    2. PWM = 255 - (adc / 4).
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC and PWM config
    printf("Light sensor adjusting PWM.\n");
```

```
    while (1) {
        unsigned int adc = read_adc(0);
        unsigned char pwm = 255 - (adc / 4);
        printf("Light: %u, PWM: %u\n", adc, pwm);  // In real: OCR0A = pwm;
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Inverse for auto-dimming.
- Smooth changes.
- Calibrate for ambient.
- PWM frequency >100Hz.

Project 195: Implement a project to read an analog pressure sensor and log data.

Project Description: Write a C program to read pressure via ADC and log values to serial.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Data logging.
- **Steps**:
    1. Read pressure.
    2. Store/log.
    3. Print log.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float log[10];
    int index = 0;
    printf("Logging pressure data.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float pressure = adc * 0.1;
        log[index % 10] = pressure;
        index++;
        printf("Log[%d]: %.1f kPa\n", index-1, pressure);
        if (index % 10 == 0) {
            printf("Last 10: ");
            for (int i = 0; i < 10; i++) printf("%.1f ", log[i]);
            printf("\n");
```

```
        }
        delay_ms(5000);
    }
    return 0;
}
```

Expert Tips

- Persistent storage for log.
- Timestamp.
- Average/peak in log.
- Overflow handling.

Project 196: Configure an ADC to read a soil moisture sensor and control a pump.

Project Description: Write a C program to read soil moisture via ADC and control water pump relay if dry.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold control.
- **Steps**:
    1. Read moisture.
    2. If < wet_threshold, pump on for time.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; DDRB |= (1 << 1); // Pump relay
    int dry_threshold = 300;
    printf("Soil moisture controlling pump.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        int pump_on = (adc < dry_threshold) ? 1 : 0;
        printf("Moisture: %u, Pump %s\n", adc, pump_on ? "ON" : "OFF");
        // In real: PORTB = (pump_on << 1); if (pump_on) delay_ms(5000); // Run 5s
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Hysteresis.
- Timed pump run.
- Calibrate thresholds.
- Water level safety.

Project 197: Create a project to use an ADC to read a flex sensor and print values.

Project Description: Write a C program to read flex sensor via ADC and print bend % or angle.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Linear mapping.
- **Steps**:
    1. Read ADC.
    2. Map to bend.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    printf("Flex sensor values.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float bend_percent = (adc / 1023.0) * 100.0;
        printf("Bend: %.1f%%\n", bend_percent);
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Calibrate straight/max.
- Nonlinear; lookup.
- Smooth readings.
- Avoid damage.

Project 198: Implement a project to read an analog heart rate sensor and display beats.

Project Description: Write a C program to read heart rate sensor via ADC, detect beats, calculate BPM.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Peak detection and timing.
- **Steps**:
    1. Sample ADC.
    2. Detect peaks.
    3. Time between peaks for BPM.
    4. Print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    static int beat_count = 0;
    static unsigned long last_beat = 0;
    int adc = 512 + (rand() % 100 - 50);  // Simulate
    if (adc > 550 && last_beat == 0) {
        beat_count++;
        last_beat = ms;
        if (beat_count % 10 == 0) {
            float bpm = 60000.0 / (ms - last_beat + 600);  // Simulated
            printf("BPM: %.0f\n", bpm);
        }
    } else if (adc < 450) {
        last_beat = 0;
    }
    return adc;
}

int main(void) {
    // In real: ADC config
    printf("Heart rate display.\n");

    while (1) {
        read_adc(0);
        delay_ms(10);
    }
    return 0;
}
```

Expert Tips

- Use finger clip sensor.
- Advanced peak detection (e.g., derivative).
- Filter noise.
- Display on LCD.

Project 199: Configure an ADC to read a pH sensor and log data.

Project Description: Write a C program to read pH via ADC and log values.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Logging.
- **Steps**:
    1. Read pH.
    2. Store/log.
    3. Print log.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config
    float log[10];
    int index = 0;
    printf("Logging pH data.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float ph = 7.0 - ((adc * 5.0 / 1023.0 - 2.5) / 0.059);
        log[index % 10] = ph;
        index++;
        printf("Log[%d]: %.2f pH\n", index-1, ph);
        if (index % 10 == 0) {
            printf("Last 10: ");
            for (int i = 0; i < 10; i++) printf("%.1f ", log[i]);
            printf("\n");
        }
        delay_ms(5000);
    }
    return 0;
}
```

Expert Tips

- Calibration buffers.
- Temperature compensation.
- Persistent log.
- Alert for extremes.

Project 200: Create a project to use an ADC to monitor a solar panel and control a relay.

Project Description: Write a C program to monitor solar voltage via ADC and control charge relay if sufficient.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Threshold relay control.
- **Steps**:
    1. Read solar voltage.
    2. If > threshold, relay on.
    3. Print.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

int main(void) {
    // In real: ADC config; DDRB |= (1 << 1); // Relay
    float threshold = 12.0;
    printf("Solar panel monitoring with relay.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        float v_solar = (adc * 5.0 / 1023.0) * 20.0;  // Divided
        int relay_on = (v_solar > threshold) ? 1 : 0;
        printf("Solar V: %.1f, Relay %s\n", v_solar, relay_on ? "ON" : "OFF");
        // In real: PORTB = (relay_on << 1);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- MPPT for efficiency.
- Hysteresis.
- Overvoltage protection.
- Battery monitoring too.

Basic Communication Protocols

Project 201: Configure UART to send "Hello, World!" to a serial console.

Project Description: Write a C program to configure UART to transmit the string "Hello, World!" to a serial console at 9600 baud.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: UART initialization and transmission.
- **Steps**:
    1. Configure UART baud rate, frame format, and enable TX.
    2. Send each character of the string via UART data register.
    3. Add delay between transmissions if needed.
    4. Print confirmation.

C Code Implementation

```
#include <stdio.h>  // For simulation; use <avr/io.h> in real
// In real: #include <avr/io.h>, #include <util/setbaud.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void uart_send_char(char c) {
    printf("%c", c);  // In real: while (!(UCSRA & (1 << UDRE))); UDR = c;
}

void uart_send_string(const char* str) {
    while (*str) {
        uart_send_char(*str++);
    }
}

int main(void) {
    // In real: UBRRH = UBRRH_VALUE; UBRRL = UBRRL_VALUE; UCSRB = (1 << TXEN); UCSRC = (1 << URSEL) |
(1 << UCSZ1) | (1 << UCSZ0); // 9600 baud, 8N1
    printf("UART configured at 9600 baud.\n");
    uart_send_string("Hello, World!\n");
    return 0;
}
```

Expert Tips

- For AVR at 16MHz, UBRR = 103 for 9600 baud (use setbaud.h for calculation).
- Connect TX to RX of USB-serial adapter (e.g., FTDI).
- Use a terminal like PuTTY for viewing.
- Enable RX if bidirectional needed.

Project 202: Create a project to receive UART data and echo it back.

Project Description: Write a C program to configure UART to receive data and immediately echo it back to the sender.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: UART polling for reception.
- **Steps**:
    1. Configure UART for TX and RX.
    2. In a loop, check if data received (RX complete flag).
    3. Read data and transmit it back.
    4. Print received char.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

char uart_receive_char(void) {
    char c = getchar();  // Simulate; in real: while (!(UCSRA & (1 << RXC))); return UDR;
    return c;
}

void uart_send_char(char c) {
    putchar(c);  // In real: while (!(UCSRA & (1 << UDRE))); UDR = c;
}

int main(void) {
    // In real: UART config as in 201, plus RXEN
    printf("UART echo ready. Send characters:\n");

    while (1) {
        char received = uart_receive_char();
        printf("Received: %c\n", received);
        uart_send_char(received);  // Echo back
    }
    return 0;
}
```

Expert Tips

- Use interrupts for RX to avoid blocking.
- Add buffer for multiple chars.
- Handle errors (frame error, overrun).
- Test with terminal sending keys.

Project 203: Implement a project to send a counter value over UART every second.

Project Description: Write a C program to send an incrementing counter (0-255) over UART every second.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timed transmission with delay.
- **Steps**:
    1. Initialize UART.
    2. In a loop, increment counter, convert to string, send via UART.
    3. Delay 1 second.
    4. Wrap counter at 255.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <util/delay.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void uart_send_string(const char* str) {
    while (*str) putchar(*str++);  // In real: send chars
}

int main(void) {
    // In real: UART config
    unsigned char counter = 0;
    printf("Sending counter every second.\n");

    while (1) {
        char buf[10];
        sprintf(buf, "Counter: %d\n", counter);
        uart_send_string(buf);
        counter = (counter + 1) % 256;
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use sprintf for formatting.
- For real delay, use timer instead of busy-wait.
- Add checksum for reliability.
- View in terminal.

Project 204: Configure UART to receive a command and toggle an LED.

Project Description: Write a C program to receive UART commands ("ON"/"OFF") and toggle an LED accordingly.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Command parsing.
- **Steps**:
    1. Configure UART RX/TX.
    2. Receive chars until newline.
    3. Compare string to "ON"/"OFF".

4. Toggle LED and confirm.

C Code Implementation

```c
#include <stdio.h>
#include <string.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

char uart_receive_char(void) {
    return getchar();  // Simulate
}

void uart_send_string(const char* str) {
    printf("%s", str);  // In real: send
}

int main(void) {
    // In real: UART config; DDRB |= (1 << 0); // LED
    char cmd[10];
    int led_state = 0;
    printf("Send 'ON' or 'OFF' to toggle LED.\n");

    while (1) {
        // Simulate receiving command
        strcpy(cmd, "ON\n");  // Replace with real receive loop
        if (strcmp(cmd, "ON\n") == 0) {
            led_state = 1;
            uart_send_string("LED ON\n");
        } else if (strcmp(cmd, "OFF\n") == 0) {
            led_state = 0;
            uart_send_string("LED OFF\n");
        }
        // In real: PORTB = (led_state << 0);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use buffer and newline termination.
- Case-insensitive parsing.
- Error handling for invalid commands.
- Use interrupts for RX.

Project 205: Create a project to send ADC readings over UART.

Project Description: Write a C program to read ADC and send the value over UART every 500ms.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Periodic ADC and transmission.
- **Steps**:
    1. Configure ADC and UART.

2. Read ADC.
3. Format and send value.
4. Delay 500ms.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate
}

void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: ADC and UART config
    printf("Sending ADC readings over UART.\n");

    while (1) {
        unsigned int adc = read_adc(0);
        char buf[20];
        sprintf(buf, "ADC: %u\n", adc);
        uart_send_string(buf);
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Average ADC readings.
- Send voltage instead of raw.
- Binary transmission for efficiency.
- Check UART buffer.

Project 206: Implement a project to parse a simple UART command (e.g., "ON"/"OFF").

Project Description: Write a C program to receive UART string commands and parse "ON"/"OFF" to control LED.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: String parsing.
- **Steps**:
    1. Receive chars into buffer until newline.
    2. Parse buffer for "ON" or "OFF".
    3. Control LED.
    4. Send confirmation.

C Code Implementation

```
#include <stdio.h>
#include <string.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

char uart_receive_char(void) {
    return getchar();
}

void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: UART config; DDRB |= (1 << 0);
    char buffer[10];
    int index = 0;
    int led_state = 0;
    printf("Send 'ON' or 'OFF'.\n");

    while (1) {
        char c = uart_receive_char();
        if (c == '\n') {
            buffer[index] = '\0';
            if (strncmp(buffer, "ON", 2) == 0) {
                led_state = 1;
                uart_send_string("LED ON\n");
            } else if (strncmp(buffer, "OFF", 3) == 0) {
                led_state = 0;
                uart_send_string("LED OFF\n");
            }
            index = 0;
        } else if (index < 9) {
            buffer[index++] = c;
        }
        // In real: PORTB = (led_state << 0);
    }
    return 0;
}
```

Expert Tips

- Null-terminate buffer.
- Handle overflow.
- Trim whitespace.
- Use state machine for parsing.

Project 207: Configure SPI to send a byte to an external device.

Project Description: Write a C program to configure SPI as master and send a single byte to an external SPI device.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI master transmission.

- **Steps**:
    1. Configure SPI as master (clock, mode, speed).
    2. Assert slave select (SS) low.
    3. Send byte via SPDR.
    4. Wait for transmission complete, deassert SS.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void spi_send_byte(unsigned char data) {
    printf("SPI sending: 0x%02X\n", data);  // In real: SPDR = data; while (!(SPSR & (1 << SPIF)));
}

int main(void) {
    // In real: DDRB |= (1 << 4) | (1 << 5) | (1 << 7); SPCR = (1 << SPE) | (1 << MSTR) | (1 << SPR0);
// Master, fck/16
    // In real: PORTB |= (1 << 2); // SS high; PORTB &= ~(1 << 2); spi_send_byte(0x55); PORTB |= (1 <<
2); // SS high
    printf("SPI master sending byte.\n");
    spi_send_byte(0x55);
    return 0;
}
```

Expert Tips

- SS pin manual control for single slave.
- SPI mode 0 (CPOL=0, CPHA=0) common.
- Clock speed <= slave max.
- Check MISO for response.

Project 208: Create a project to read data from an SPI-based sensor.

Project Description: Write a C program to configure SPI master to read data from an SPI sensor (e.g., temperature).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI read transaction.
- **Steps**:
    1. Configure SPI master.
    2. Send command byte to sensor.
    3. Read response byte(s).
    4. Print data.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned char spi_exchange_byte(unsigned char data) {
    printf("SPI TX: 0x%02X\n", data);  // In real: SPDR = data; while (!(SPSR & (1 << SPIF))); return
SPDR;
    return (rand() % 256);  // Simulate RX
}

int main(void) {
    // In real: SPI config as in 207
    printf("Reading SPI sensor.\n");
    // In real: SS low; spi_exchange_byte(0x01); unsigned char data = spi_exchange_byte(0x00); SS high
    unsigned char data = spi_exchange_byte(0x01);  // Command
    data = spi_exchange_byte(0x00);  // Read
    printf("Sensor data: 0x%02X\n", data);
    return 0;
}
```

Expert Tips

- Consult sensor datasheet for command format.
- Dummy byte for read (clock out data).
- Multiple bytes for longer data.
- Error checking.

Project 209: Implement a project to control an SPI-based LED driver.

Project Description: Write a C program to control an SPI LED driver (e.g., MAX7219) to set LED pattern.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI command transmission.
- **Steps**:
    1. Configure SPI master.
    2. Send command bytes to set intensity, scan limit, then row data.
    3. Print pattern.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void spi_send_byte(unsigned char data) {
    printf("SPI: 0x%02X\n", data);  // In real: SPDR = data; while (!(SPSR & (1 << SPIF)));
}
```

```c
int main(void) {
    // In real: SPI config
    printf("Controlling SPI LED driver.\n");
    // In real: SS low
    spi_send_byte(0x09); spi_send_byte(0x00);  // No decode
    spi_send_byte(0x0A); spi_send_byte(0x03);  // Intensity
    spi_send_byte(0x0B); spi_send_byte(0x07);  // Scan 0-7
    spi_send_byte(0x0F); spi_send_byte(0x00);  // Off
    spi_send_byte(0x08); spi_send_byte(0x01);  // Row 0 pattern
    // In real: SS high
    return 0;
}
```

Expert Tips

- MAX7219: address/data pairs.
- Chain multiple drivers.
- Test with LED matrix.
- Power supply for LEDs.

Project 210: Configure SPI to send a string to an SPI display.

Project Description: Write a C program to send a string to an SPI display (e.g., Nokia 5110).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI command and data transmission.
- **Steps**:
    1. Configure SPI.
    2. Send init commands.
    3. Send string chars as data.
    4. Print sent string.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void spi_send_byte(unsigned char data) {
    printf("SPI data: 0x%02X\n", data);
}

void spi_send_command(unsigned char cmd) {
    // In real: DC low; spi_send_byte(cmd);
    printf("Command: 0x%02X\n", cmd);
}

int main(void) {
    // In real: SPI config; DDR for DC, Reset, etc.
    printf("Sending string to SPI display.\n");
    spi_send_command(0x21);  // Extended commands
    spi_send_command(0xB0);  // Bias
    spi_send_command(0x04);  // Temp coefficient
    spi_send_command(0x13);  // Inverse
```

```
    spi_send_command(0x20);  // Basic commands
    spi_send_command(0x0C);  // Display on
    // Send "Hello"
    for (char c = 'H'; c <= 'o'; c++) {
        spi_send_byte(c);  // Simplified, real needs font lookup
    }
    return 0;
}
```

Expert Tips

- Nokia 5110: DC pin for cmd/data.
- Use font table for chars.
- Contrast adjustment.
- Clear screen first.

Project 211: Create a project to read an SPI temperature sensor.

Project Description: Write a C program to read temperature from an SPI sensor (e.g., MCP9808).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI read sequence.
- **Steps**:
    1. Configure SPI.
    2. Send read command.
    3. Read data bytes.
    4. Convert to temperature.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned char spi_exchange_byte(unsigned char data) {
    return (rand() % 256);  // Simulate
}

int main(void) {
    // In real: SPI config
    printf("Reading SPI temperature sensor.\n");
    // In real: SS low
    spi_exchange_byte(0x01);  // Read temp register
    unsigned char high = spi_exchange_byte(0x00);
    unsigned char low = spi_exchange_byte(0x00);
    // In real: SS high
    int temp_raw = (high << 8) | low;
    float temp_c = (temp_raw / 256.0) * 0.0625;  // MCP9808 resolution
    printf("Temperature: %.2f°C\n", temp_c);
    return 0;
}
```

Expert Tips

- Check datasheet for register map.
- Sign extension for negative temps.
- Average readings.
- Power cycle if needed.

Project 212: Implement a project to use SPI to control a shift register.

Project Description: Write a C program to control an 8-bit shift register (e.g., 74HC595) via SPI to set output pattern.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI data shift.
- **Steps**:
    1. Configure SPI master.
    2. Send byte to shift data.
    3. Pulse latch to output.
    4. Print pattern.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void spi_send_byte(unsigned char data) {
    printf("Shifting: 0x%02X\n", data);
}

int main(void) {
    // In real: SPI config; latch pin setup
    printf("Controlling SPI shift register.\n");
    spi_send_byte(0xAA);  // Pattern
    // In real: latch low; spi_send_byte(data); latch high
    return 0;
}
```

Expert Tips

- 74HC595: SER to MOSI, SRCLK to SCK, RCLK latch.
- Chain multiple for more bits.
- Test with LEDs.
- Clock speed < 25MHz.

Project 213: Configure I2C to read data from an I2C temperature sensor.

Project Description: Write a C program to configure I2C master to read temperature from sensor (e.g., LM75).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C read transaction.
- **Steps**:
    1. Configure I2C (speed, address).
    2. Start, send device address + read pointer.
    3. Read data byte.
    4. Stop, convert to temp.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned char i2c_read_byte(unsigned char addr, unsigned char reg) {
    printf("I2C read from 0x%02X reg 0x%02X\n", addr, reg);
    return (rand() % 256);  // Simulate
}

int main(void) {
    // In real: TWBR = 72; // 100kHz; TWCR = (1 << TWEN);
    unsigned char temp_raw = i2c_read_byte(0x48, 0x00);  // LM75 address, temp reg
    float temp_c = (signed char)temp_raw;  // 1°C resolution
    printf("Temperature: %.0f°C\n", temp_c);
    return 0;
}
```

Expert Tips

- Pull-up resistors on SDA/SCL (4.7kΩ).
- LM75 address 0x48-0x4F.
- Handle NACK.
- Repeated start for efficiency.

Project 214: Create a project to write data to an I2C EEPROM.

Project Description: Write a C program to write a byte to an I2C EEPROM (e.g., 24C02) at a specific address.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C write transaction.
- **Steps**:
    1. Configure I2C.
    2. Start, send address + write + memory address + data.
    3. Stop, delay for write cycle.
    4. Confirm write.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void i2c_write_byte(unsigned char addr, unsigned char mem, unsigned char data) {
    printf("I2C write to 0x%02X mem 0x%02X data 0x%02X\n", addr, mem, data);
}

int main(void) {
    // In real: I2C config
    i2c_write_byte(0x50, 0x00, 0xAA);  // 24C02 address, location 0
    delay_ms(5);  // Write cycle
    printf("Data written to EEPROM.\n");
    return 0;
}
```

Expert Tips

- Write cycle 5ms.
- Page write for multiple bytes.
- Address rollover.
- Read back to verify.

Project 215: Implement a project to read an I2C accelerometer and print values.

Project Description: Write a C program to read 3-axis accelerometer (e.g., ADXL345) via I2C and print X/Y/Z.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C multi-byte read.
- **Steps**:
    1. Configure I2C and sensor.
    2. Read data register for X/Y/Z.
    3. Convert to g.
    4. Print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void i2c_read_multi(unsigned char addr, unsigned char reg, unsigned char* data, int len) {
    printf("I2C read %d bytes from 0x%02X reg 0x%02X\n", len, addr, reg);
    for (int i = 0; i < len; i++) data[i] = (rand() % 256);  // Simulate
}
```

```
int main(void) {
    // In real: I2C config; init ADXL345
    unsigned char data[6];
    i2c_read_multi(0x53, 0x32, data, 6);  // X/Y/Z low/high
    int x = (data[1] << 8) | data[0];
    int y = (data[3] << 8) | data[2];
    int z = (data[5] << 8) | data[4];
    float scale = 256.0 / 9.81;  // ±2g
    printf("Accel X: %.2fg, Y: %.2fg, Z: %.2fg\n", x / scale, y / scale, z / scale);
    return 0;
}
```

Expert Tips

- ADXL345 address 0x53.
- Configure range/resolution.
- Sign extension for negative.
- Filter data.

Project 216: Configure I2C to control an I2C-based OLED display.

Project Description: Write a C program to initialize and write text to an I2C OLED (e.g., SSD1306).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C command and data.
- **Steps**:
    1. Configure I2C.
    2. Send init commands.
    3. Set page/column, send font data.
    4. Print text.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void i2c_send_command(unsigned char addr, unsigned char cmd) {
    printf("OLED command 0x%02X\n", cmd);
}

void i2c_send_data(unsigned char addr, unsigned char data) {
    printf("OLED data 0x%02X\n", data);
}

int main(void) {
    // In real: I2C config
    printf("Initializing I2C OLED.\n");
    i2c_send_command(0x3C, 0x00);  // Co=0, D/C=0
    i2c_send_command(0x3C, 0xAE);  // Display off
    i2c_send_command(0x3C, 0xD5); i2c_send_command(0x3C, 0x80);  // Clock
    // Send "Hello" font data (simplified)
    i2c_send_data(0x3C, 0x48);  // H pixel data
    i2c_send_command(0x3C, 0xAF);  // Display on
```

```
    return 0;
}
```

Expert Tips

- SSD1306 address 0x3C/0x3D.
- Use SSD1306 library for ease.
- Font 8x8.
- Contrast adjustment.

Project 217: Create a project to read an I2C pressure sensor.

Project Description: Write a C program to read pressure from I2C sensor (e.g., BMP280).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C read with compensation.
- **Steps**:
    1. Configure I2C and sensor.
    2. Read pressure raw.
    3. Apply compensation formula.
    4. Print hPa.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned short i2c_read_word(unsigned char addr, unsigned char reg) {
    printf("I2C read word from 0x%02X reg 0x%02X\n", addr, reg);
    return (rand() % 65536);  // Simulate
}

int main(void) {
    // In real: I2C config; init BMP280
    unsigned short pressure_raw = i2c_read_word(0x76, 0xF7);  // Pressure MSB/LSB
    float pressure_hpa = pressure_raw / 256.0;  // Simplified
    printf("Pressure: %.2f hPa\n", pressure_hpa);
    return 0;
}
```

Expert Tips

- BMP280 address 0x76/0x77.
- Read calibration data.
- Complex compensation.
- Temperature too.

Project 218: Implement a project to use I2C to communicate with a real-time clock (RTC).

Project Description: Write a C program to read time from I2C RTC (e.g., DS1307).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C BCD read.
- **Steps**:
    1. Configure I2C.
    2. Read seconds, minutes, hours registers.
    3. Convert BCD to decimal.
    4. Print time.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned char i2c_read_byte(unsigned char addr, unsigned char reg) {
    return (rand() % 256);  // Simulate
}

unsigned char bcd_to_dec(unsigned char bcd) {
    return ((bcd >> 4) * 10) + (bcd & 0x0F);
}

int main(void) {
    // In real: I2C config
    unsigned char sec = i2c_read_byte(0x68, 0x00);
    unsigned char min = i2c_read_byte(0x68, 0x01);
    unsigned char hour = i2c_read_byte(0x68, 0x02);
    int s = bcd_to_dec(sec & 0x7F);
    int m = bcd_to_dec(min);
    int h = bcd_to_dec(hour & 0x3F);
    printf("Time: %02d:%02d:%02d\n", h, m, s);
    return 0;
}
```

Expert Tips

- DS1307 address 0x68.
- Stop oscillator for write.
- Battery backup.
- Set time first.

Project 219: Configure UART to send sensor data in a formatted string.

Project Description: Write a C program to send formatted sensor data (e.g., temp, humidity) over UART.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Formatted transmission.
- **Steps**:
    1. Read sensors (simulate).
    2. Format string with sprintf.
    3. Send via UART.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: UART config
    float temp = 25.5;
    float hum = 60.2;
    char buf[50];
    sprintf(buf, "Temp: %.1fC, Hum: %.1f%%\n", temp, hum);
    uart_send_string(buf);
    printf("Formatted sensor data sent.\n");
    return 0;
}
```

Expert Tips

- CSV or JSON for parsing.
- Check buffer size.
- Timestamp.
- Error codes.

Project 220: Create a project to use UART to receive and parse a number.

Project Description: Write a C program to receive a number over UART and use it to set PWM duty.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Numeric parsing.
- **Steps**:
    1. Receive string.
    2. Parse to int with atoi.
    3. Set PWM.
    4. Confirm.

C Code Implementation

```
#include <stdio.h>
#include <stdlib.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

char uart_receive_string(void) {
    return "128\n";  // Simulate
}

void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: UART config
    char buf[10];
    strcpy(buf, uart_receive_string());  // Real receive
    int duty = atoi(buf);
    printf("Received number: %d, PWM set\n", duty);  // In real: OCR0A = duty;
    return 0;
}
```

Expert Tips

- Validate range (0-255).
- Handle non-numeric.
- Buffer overflow protection.
- Units in command.

Project 221: Implement a project to use SPI to read a digital potentiometer.

Project Description: Write a C program to read wiper position from SPI digital pot (e.g., MCP41xxx).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI command read.
- **Steps**:
    1. Configure SPI.
    2. Send read command.
    3. Read wiper value.
    4. Print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
unsigned char spi_exchange_byte(unsigned char data) {
    return (rand() % 256);
}

int main(void) {
    // In real: SPI config
    printf("Reading SPI digital pot.\n");
    spi_exchange_byte(0x0C);  // Read command
    unsigned char wiper = spi_exchange_byte(0x00);
    printf("Wiper position: %d (0-256)\n", wiper);
    return 0;
}
```

Expert Tips

- MCP41010: command 0x0C for read.
- 8-bit value.
- Set before read if needed.
- Chain pots.

Project 222: Configure I2C to read a humidity sensor and print values.

Project Description: Write a C program to read humidity from I2C sensor (e.g., SHT21).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C read.
- **Steps**:
    1. Configure I2C.
    2. Send trigger command.
    3. Read humidity data.
    4. Convert and print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned short i2c_read_word(unsigned char addr, unsigned char cmd) {
    printf("I2C trigger 0x%02X\n", cmd);
    delay_ms(100);  // Measurement time
    return (rand() % 65536);  // Simulate
}

int main(void) {
    // In real: I2C config
    unsigned short hum_raw = i2c_read_word(0x40, 0xF5);  // Hold master, humidity
    float humidity = -6.0 + 125.0 * (hum_raw / 65536.0);  // SHT21 formula
    printf("Humidity: %.1f%%\n", humidity);
    return 0;
}
```

Expert Tips

- SHT21 address 0x40.
- Soft reset if needed.
- CRC check.
- Temperature too.

Project 223: Create a project to use I2C to control a digital-to-analog converter (DAC).

Project Description: Write a C program to set voltage on I2C DAC (e.g., MCP4725).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C write.
- **Steps**:
    1. Configure I2C.
    2. Send DAC register and value.
    3. Print voltage.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void i2c_write_word(unsigned char addr, unsigned short value) {
    printf("I2C DAC set to 0x%04X\n", value);
}

int main(void) {
    // In real: I2C config
    unsigned short dac_value = 2048;  // Half scale for 12-bit
    i2c_write_word(0x60, dac_value);  // MCP4725
    float voltage = (dac_value / 4095.0) * 5.0;
    printf("DAC voltage: %.2f V\n", voltage);
    return 0;
}
```

Expert Tips

- MCP4725 address 0x60.
- 12-bit value.
- Fast mode write.
- Verify with multimeter.

Project 224: Implement a project to send a heartbeat signal over UART.

Project Description: Write a C program to send "Heartbeat" over UART every 5 seconds.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Periodic transmission.
- **Steps**:
    1. Configure UART.
    2. Send string every 5s.
    3. Use timer for delay.
    4. Print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: UART config
    printf("Sending heartbeat every 5s.\n");

    while (1) {
        uart_send_string("Heartbeat\n");
        delay_ms(5000);
    }
    return 0;
}
```

Expert Tips

- Use timer interrupt for precision.
- Add timestamp or counter.
- For monitoring systems.
- Error if missed.

Project 225: Configure SPI to interface with an SPI flash memory chip.

Project Description: Write a C program to read ID from SPI flash (e.g., W25Q32).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI command sequence.
- **Steps**:
    1. Configure SPI.
    2. Send JEDEC ID command.
    3. Read 3 bytes.
    4. Print manufacturer/ID.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned char spi_exchange_byte(unsigned char data) {
    return (rand() % 256);
}

int main(void) {
    // In real: SPI config
    printf("Reading SPI flash ID.\n");
    // In real: SS low
    spi_exchange_byte(0x9F);  // JEDEC ID
    unsigned char mfg = spi_exchange_byte(0x00);
    unsigned char id1 = spi_exchange_byte(0x00);
    unsigned char id2 = spi_exchange_byte(0x00);
    // In real: SS high
    printf("Flash ID: Mfg 0x%02X, ID 0x%02X%02X\n", mfg, id1, id2);
    return 0;
}
```

Expert Tips

- W25Q: address 0x9F for ID.
- Fast read for data.
- Erase before write.
- Page program 256 bytes.

Project 226: Create a project to use I2C to read a light sensor.

Project Description: Write a C program to read light level from I2C sensor (e.g., BH1750).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C read.
- **Steps**:
    1. Configure I2C.
    2. Send measurement command.
    3. Read lux data.
    4. Convert and print.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
unsigned short i2c_read_word(unsigned char addr) {
    return (rand() % 65536);  // Simulate
}

int main(void) {
    // In real: I2C config
    i2c_write_byte(0x23, 0x10);  // One time H-res
    delay_ms(120);
    unsigned short lux_raw = i2c_read_word(0x23);
    float lux = (lux_raw / 1.2);  // BH1750 formula
    printf("Light level: %.1f lux\n", lux);
    return 0;
}
```

Expert Tips

- BH1750 address 0x23.
- Delay for measurement.
- Resolution modes.
- Calibration.

Project 227: Implement a project to use UART to send button press events.

Project Description: Write a C program to detect button press and send event over UART.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Edge detection and transmission.
- **Steps**:
    1. Configure UART and button input.
    2. Poll button for press (debounce).
    3. Send "BUTTON_PRESSED".
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_button(void) {
    return (rand() % 2);  // Simulate
}

void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: UART config; DDRB &= ~(1 << 1); PORTB |= (1 << 1);
    int prev = 1;
    printf("Sending button events.\n");
```

```
    while (1) {
        int curr = read_button();
        if (curr == 0 && prev == 1) {
            uart_send_string("BUTTON_PRESSED\n");
        }
        prev = curr;
        delay_ms(50);  // Debounce
    }
    return 0;
}
```

Expert Tips

- Edge detection for single event.
- Debounce 50ms.
- Send timestamp.
- Interrupt for button.

Project 228: Configure I2C to communicate with a magnetometer.

Project Description: Write a C program to read magnetic field from I2C magnetometer (e.g., HMC5883L).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C multi-byte read.
- **Steps**:
    1. Configure I2C and sensor.
    2. Read X/Y/Z registers.
    3. Convert to uT.
    4. Print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void i2c_read_multi(unsigned char addr, unsigned char reg, unsigned char* data, int len) {
    for (int i = 0; i < len; i++) data[i] = (rand() % 256);
}

int main(void) {
    // In real: I2C config; init HMC5883L
    unsigned char data[6];
    i2c_read_multi(0x1E, 0x03, data, 6);  // X/Y/Z
    int x = (data[0] << 8) | data[1];
    int y = (data[2] << 8) | data[3];
    int z = (data[4] << 8) | data[5];
    printf("Magnetic field X: %d, Y: %d, Z: %d\n", x, y, z);
    return 0;
}
```

Expert Tips

- HMC5883L address 0x1E.
- Configure gain/ mode.
- Calibration for heading.
- Fuse with accel/gyro.

Project 229: Create a project to use SPI to control a 7-segment display driver.

Project Description: Write a C program to display a digit on SPI 7-segment driver (e.g., MAX7219).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI command.
- **Steps**:
    1. Configure SPI.
    2. Send digit to row.
    3. Print digit.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void spi_send_byte(unsigned char addr, unsigned char data) {
    printf("7-seg: addr 0x%02X data 0x%02X\n", addr, data);
}

int main(void) {
    // In real: SPI config
    unsigned char digit5 = 0x6D;  // Pattern for 5
    spi_send_byte(0x01, digit5);  // Digit 1
    printf("Displaying digit 5 on 7-segment.\n");
    return 0;
}
```

Expert Tips

- MAX7219 for 7-seg.
- Decode mode on.
- Multiple digits.
- Brightness control.

Project 230: Implement a project to use UART to send temperature data.

Project Description: Write a C program to read temp sensor and send formatted temp over UART.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Sensor read and format.
- **Steps**:
    1. Read temp (ADC or I2C).
    2. Format string.
    3. Send UART.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

float read_temp(void) {
    return 25.5 + (rand() % 10 - 5);  // Simulate
}

void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: UART config
    printf("Sending temperature data.\n");

    while (1) {
        float temp = read_temp();
        char buf[20];
        sprintf(buf, "Temp: %.2f C\n", temp);
        uart_send_string(buf);
        delay_ms(2000);
    }
    return 0;
}
```

Expert Tips

- High precision format.
- Include timestamp.
- Error if invalid.
- Binary for efficiency.

Project 231: Configure I2C to read a gyroscope and print values.

Project Description: Write a C program to read gyro from I2C sensor (e.g., L3G4200D).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C read.
- **Steps**:
    1. Configure I2C and gyro.

2. Read X/Y/Z registers.
3. Convert to dps.
4. Print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void i2c_read_multi(unsigned char addr, unsigned char reg, short* data) {
    data[0] = (rand() % 65536 - 32768);  // Simulate signed
    data[1] = (rand() % 65536 - 32768);
    data[2] = (rand() % 65536 - 32768);
}

int main(void) {
    // In real: I2C config; init L3G
    short gyro[3];
    i2c_read_multi(0x69, 0x28, gyro);  // X/Y/Z
    float scale = 8.75 / 32768.0;  // ±250 dps
    printf("Gyro X: %.2fdps, Y: %.2fdps, Z: %.2fdps\n", gyro[0]*scale, gyro[1]*scale, gyro[2]*scale);
    return 0;
}
```

Expert Tips

- L3G address 0x69.
- Configure range.
- Offset calibration.
- Fuse with accel.

Project 232: Create a project to use SPI to read an external ADC.

Project Description: Write a C program to read value from SPI external ADC (e.g., MCP3008).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI read.
- **Steps**:
    1. Configure SPI.
    2. Send channel select.
    3. Read high/low bytes.
    4. Combine to 10-bit.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned char spi_exchange_byte(unsigned char data) {
    return (rand() % 256);
}

int main(void) {
    // In real: SPI config
    printf("Reading external SPI ADC.\n");
    // In real: SS low
    spi_exchange_byte(0x01 | (0 << 6));  // Channel 0, single-ended
    unsigned char low = spi_exchange_byte(0x00);
    unsigned char high = spi_exchange_byte(0x00);
    // In real: SS high
    unsigned int adc = ((high & 0x03) << 8) | low;
    printf("ADC value: %u (0-1023)\n", adc);
    return 0;
}
```

Expert Tips

- MCP3008: start bit, SGL/DIFF, channel.
- Clock speed <1MHz.
- Reference voltage.
- Average readings.

Project 233: Implement a project to use UART to receive and execute commands.

Project Description: Write a C program to receive UART commands (e.g., "LED_ON", "READ_TEMP") and execute.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Command parser.
- **Steps**:
    1. Receive full command.
    2. strcmp to match.
    3. Execute action.
    4. Respond.

C Code Implementation

```c
#include <stdio.h>
#include <string.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
char uart_receive_string(void) {
    return "LED_ON\n";  // Simulate
}

void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: UART config
    char cmd[20];
    strcpy(cmd, uart_receive_string());
    if (strcmp(cmd, "LED_ON\n") == 0) {
        printf("Turning LED ON\n");  // Action
        uart_send_string("OK\n");
    } else if (strcmp(cmd, "READ_TEMP\n") == 0) {
        uart_send_string("25.5C\n");
    }
    return 0;
}
```

Expert Tips

- Buffer and terminate.
- Case insensitive.
- Parameter parsing.
- Error responses.

Project 234: Configure I2C to control an I2C-based motor driver.

Project Description: Write a C program to set speed on I2C motor driver (e.g., PCA9685 PWM).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C PWM set.
- **Steps**:
    1. Configure I2C.
    2. Set channel PWM value.
    3. Print speed.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void i2c_set_pwm(unsigned char addr, unsigned char channel, unsigned short on, unsigned short off) {
    printf("PWM channel %d: on %d off %d\n", channel, on, off);
}

int main(void) {
    // In real: I2C config; init PCA9685
    i2c_set_pwm(0x40, 0, 0, 307);  // 50% duty for channel 0
    printf("Motor speed set to 50%.\n");
    return 0;      }
```

Expert Tips

- PCA9685 address 0x40.
- 12-bit PWM (0-4095).
- Prescaler for frequency.
- Multiple channels.

Project 235: Create a project to use SPI to interface with an SD card.

Project Description: Write a C program to initialize SD card via SPI and read card type.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SD card protocol.
- **Steps**:
    1. Configure SPI low speed.
    2. Send CMD0 (reset).
    3. Send CMD8, CMD58 for type.
    4. Print type.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned char spi_exchange_byte(unsigned char data) {
    if (data == 0x40) return 0x01;  // Simulate CMD0 response
    return 0x00;
}

int main(void) {
    // In real: SPI config slow
    printf("Initializing SD card.\n");
    spi_exchange_byte(0x40);  // CMD0
    spi_exchange_byte(0x00); spi_exchange_byte(0x00); spi_exchange_byte(0x01);
spi_exchange_byte(0xAA);  // CMD8 args
    unsigned char resp = spi_exchange_byte(0x48);  // CMD8
    printf("SD response: 0x%02X\n", resp);
    return 0;
}
```

Expert Tips

- SD init complex, use library.
- CMD0 R1 response 0x01.
- SPI mode 0.
- Level shifter for 3.3V.

Project 236: Implement a project to use UART to log sensor data.

Project Description: Write a C program to log multiple sensor data over UART in CSV format.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Formatted logging.
- **Steps**:
    1. Read sensors.
    2. Format CSV line.
    3. Send UART.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: UART config
    float temp = 25.5;
    float hum = 60.0;
    int light = 500;
    char buf[50];
    sprintf(buf, "Temp,%.1f,Hum,%.1f,Light,%d\n", temp, hum, light);
    uart_send_string(buf);
    printf("Sensor log sent.\n");
    return 0;
}
```

Expert Tips

- CSV for easy parsing.
- Header first.
- Timestamp column.
- Buffer flush.

Project 237: Configure I2C to read a temperature and humidity sensor.

Project Description: Write a C program to read temp/hum from I2C sensor (e.g., SHT31).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C read.
- **Steps**:
    1. Configure I2C.

2. Trigger measurement.
3. Read 6 bytes (temp/hum).
4. Convert and print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void i2c_read_multi(unsigned char addr, unsigned char* data, int len) {
    for (int i = 0; i < len; i++) data[i] = (rand() % 256);
}

int main(void) {
    // In real: I2C config
    i2c_write_byte(0x44, 0x24);  // High precision
    delay_ms(15);
    unsigned char data[6];
    i2c_read_multi(0x44, data, 6);
    unsigned int temp_raw = (data[0] << 12) | (data[1] << 4) | (data[2] >> 4);
    unsigned int hum_raw = (data[3] << 12) | (data[4] << 4) | (data[5] >> 4);
    float temp = -45 + 175 * (temp_raw / 65535.0);
    float hum = -6 + 125 * (hum_raw / 65535.0);
    printf("Temp: %.2f°C, Hum: %.2f%%\n", temp, hum);
    return 0;
}
```

Expert Tips

- SHT31 address 0x44.
- CRC on data.
- Repeatability modes.
- Alert pin.

Project 238: Create a project to use SPI to control an RGB LED driver.

Project Description: Write a C program to set color on SPI RGB driver (e.g., WS2801).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI data stream.
- **Steps**:
    1. Configure SPI.
    2. Send 24-bit RGB data.
    3. Latch if needed.
    4. Print color.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void spi_send_byte(unsigned char data) {
    printf("RGB byte: 0x%02X\n", data);
}

int main(void) {
    // In real: SPI config
    printf("Setting RGB LED to red.\n");
    spi_send_byte(0xFF);  // Red
    spi_send_byte(0x00);  // Green
    spi_send_byte(0x00);  // Blue
    // In real: latch pulse
    return 0;
}
```

Expert Tips

- WS2801: RGB order, 24 bits per LED.
- Chain multiple.
- Timing critical.
- Power injection.

Project 239: Implement a project to use UART to send a timestamp.

Project Description: Write a C program to send current timestamp over UART every 10s (simulate time).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timed formatted send.
- **Steps**:
    1. Simulate or read RTC time.
    2. Format HH:MM:SS.
    3. Send UART.
    4. Delay.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void uart_send_string(const char* str) {
    printf("%s", str);
}
```

```
int main(void) {
    // In real: UART and RTC config
    int seconds = 0;
    printf("Sending timestamp every 10s.\n");

    while (1) {
        int h = seconds / 3600;
        int m = (seconds % 3600) / 60;
        int s = seconds % 60;
        char buf[20];
        sprintf(buf, "Time: %02d:%02d:%02d\n", h, m, s);
        uart_send_string(buf);
        seconds += 10;
        delay_ms(10000);
    }
    return 0;
}
```

Expert Tips

- Use RTC for real time.
- Sync with NTP if networked.
- ISO format.
- Leap seconds.

Project 240: Configure I2C to communicate with a color sensor.

Project Description: Write a C program to read RGB from I2C color sensor (e.g., TCS34725).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C read registers.
- **Steps**:
    1. Configure I2C and sensor.
    2. Read RGBC registers.
    3. Convert to color.
    4. Print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned short i2c_read_word(unsigned char addr, unsigned char reg) {
    return (rand() % 65536);
}

int main(void) {
    // In real: I2C config; init TCS34725
    unsigned short r = i2c_read_word(0x29, 0x14);  // Red
    unsigned short g = i2c_read_word(0x29, 0x16);  // Green
    unsigned short b = i2c_read_word(0x29, 0x18);  // Blue
    unsigned short c = i2c_read_word(0x29, 0x1C);  // Clear
    printf("Color R:%u G:%u B:%u C:%u\n", r, g, b, c);
```

```
    return 0;
}
```

Expert Tips

- TCS34725 address 0x29.
- Integration time.
- Calibration with white.
- Lux calculation.

Project 241: Create a project to use SPI to read a pressure sensor.

Project Description: Write a C program to read pressure from SPI sensor (e.g., MS5611).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI command read.
- **Steps**:
    1. Configure SPI.
    2. Send conversion command.
    3. Read ADC.
    4. Convert to pressure.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned short spi_exchange_word(unsigned short cmd) {
    return (rand() % 65536);
}

int main(void) {
    // In real: SPI config
    spi_exchange_word(0x40);  // D1 conversion
    delay_ms(10);
    unsigned short adc = spi_exchange_word(0x00);  // Read
    float pressure = adc / 240.0;  // Simplified
    printf("Pressure: %.2f mbar\n", pressure);
    return 0;
}
```

Expert Tips

- MS5611: command/response.
- Calibration coefficients.
- Temperature compensation.
- Oversampling.

Project 242: Implement a project to use UART to send ADC data in CSV format.

Project Description: Write a C program to send multiple ADC readings in CSV over UART.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: CSV formatting.
- **Steps**:
    1. Read ADCs.
    2. sprintf CSV line.
    3. Send UART.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int ch) {
    return (rand() % 1024);
}

void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: UART and ADC config
    printf("ADC CSV data.\n");

    while (1) {
        unsigned int adc1 = read_adc(0);
        unsigned int adc2 = read_adc(1);
        char buf[30];
        sprintf(buf, "%u,%u\n", adc1, adc2);
        uart_send_string(buf);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Header: "ADC1,ADC2\n".
- Escaping commas.
- Timestamp column.
- Binary alternative.

Project 243: Configure I2C to control an I2C-based LED driver.

Project Description: Write a C program to set brightness on I2C LED driver (e.g., PCA9685).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C PWM.
- **Steps**:
    1. Configure I2C.
    2. Set LEDn_ON/OFF registers.
    3. Print brightness.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void i2c_set_led(unsigned char addr, unsigned char channel, unsigned short duty) {
    printf("LED %d brightness %d\n", channel, duty);
}

int main(void) {
    // In real: I2C config
    i2c_set_led(0x40, 0, 2048);  // 50%
    printf("I2C LED driver controlled.\n");
    return 0;
}
```

Expert Tips

- PCA9685: 16 channels.
- Global prescaler.
- I2C broadcast.
- Sync pins.

Project 244: Create a project to use SPI to interface with a display module.

Project Description: Write a C program to send graphics data to SPI display (e.g., ST7735).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI command/data.
- **Steps**:
    1. Configure SPI.
    2. Init sequence.
    3. Send pixel data.
    4. Print.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void spi_send_command(unsigned char cmd) {
    printf("Display cmd 0x%02X\n", cmd);
}

void spi_send_data(unsigned char data) {
    printf("Pixel data 0x%02X\n", data);
}

int main(void) {
    // In real: SPI config; DC pin
    spi_send_command(0x11);  // Sleep out
    delay_ms(120);
    spi_send_command(0x29);  // Display on
    spi_send_data(0xFF);  // White pixel
    printf("SPI display interfaced.\n");
    return 0;
}
```

Expert Tips

- ST7735 init sequence from datasheet.
- Window set for area.
- RGB565 format.
- Rotation control.

Project 245: Implement a project to use UART to send a simple JSON string.

Project Description: Write a C program to send sensor data in JSON over UART.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: JSON formatting.
- **Steps**:
    1. Read data.
    2. sprintf JSON.
    3. Send UART.
    4. Delay.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```
void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: UART config
    float temp = 25.5;
    char buf[100];
    sprintf(buf, "{\"temp\":%.1f,\"status\":\"OK\"}\n", temp);
    uart_send_string(buf);
    printf("JSON sent.\n");
    return 0;
}
```

Expert Tips

- Escape quotes.
- Validate JSON.
- Compact for bandwidth.
- Parser on receiver.

Project 246: Configure I2C to read a proximity sensor.

Project Description: Write a C program to read distance from I2C proximity sensor (e.g., VL53L0X).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C read.
- **Steps**:
    1. Configure I2C and sensor.
    2. Trigger measurement.
    3. Read distance.
    4. Print mm.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned short i2c_read_word(unsigned char addr, unsigned char reg) {
    return (rand() % 8192);  // Simulate 0-8m
}

int main(void) {
    // In real: I2C config; init VL53L0X
    i2c_write_byte(0x29, 0x80);  // Start
    delay_ms(50);
    unsigned short distance = i2c_read_word(0x29, 0x09);  // Range
    printf("Distance: %u mm\n", distance);
    return 0;
}
```

Expert Tips

- VL53L0X address 0x29.
- Ranging mode.
- Error codes.
- Continuous mode.

Project 247: Create a project to use SPI to control a digital potentiometer.

Project Description: Write a C program to set resistance on SPI digital pot (e.g., MCP41xxx).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI write.
- **Steps**:
    1. Configure SPI.
    2. Send command and value.
    3. Print resistance.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void spi_send_word(unsigned short data) {
    printf("Pot set to 0x%04X\n", data);
}

int main(void) {
    // In real: SPI config
    unsigned short value = 128;  // 50%
    spi_send_word(0x1100 | value);  // Write wiper 0
    float resistance = (value / 256.0) * 100000.0;  // 100k pot
    printf("Resistance: %.0f ohms\n", resistance);
    return 0;
}
```

Expert Tips

- MCP41010: 8-bit, command 0x11 for wiper 0.
- Shutdown mode.
- Tolerance.
- Readback.

Project 248: Implement a project to use UART to receive and parse a command string.

Project Description: Write a C program to receive UART command string (e.g., "SET_PWM 128") and parse/set PWM.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: String parsing with sscanf.
- **Steps**:
    1. Receive full line.
    2. sscanf for command and param.
    3. Execute.
    4. Respond.

C Code Implementation

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

char uart_receive_string(void) {
    return "SET_PWM 128\n";  // Simulate
}

void uart_send_string(const char* str) {
    printf("%s", str);
}

int main(void) {
    // In real: UART config
    char cmd[30];
    strcpy(cmd, uart_receive_string());
    char action[10];
    int value;
    if (sscanf(cmd, "%s %d", action, &value) == 2) {
        if (strcmp(action, "SET_PWM") == 0) {
            printf("PWM set to %d\n", value);  // In real: OCR0A = value;
            uart_send_string("OK\n");
        }
    }
    return 0;
}
```

Expert Tips

- Validate param range.
- Multiple commands.
- Error messages.
- Buffer security.

Project 249: Configure I2C to communicate with an I2C RTC and display time.

Project Description: Write a C program to read and display time from I2C RTC (e.g., DS3231).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C BCD read.

- **Steps**:
    1. Configure I2C.
    2. Read time registers.
    3. Convert BCD.
    4. Print HH:MM:SS.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned char i2c_read_byte(unsigned char addr, unsigned char reg) {
    return (rand() % 256);
}

unsigned char bcd_to_dec(unsigned char bcd) {
    return ((bcd >> 4) * 10) + (bcd & 0x0F);
}

int main(void) {
    // In real: I2C config
    unsigned char sec = i2c_read_byte(0x68, 0x00);
    unsigned char min = i2c_read_byte(0x68, 0x01);
    unsigned char hour = i2c_read_byte(0x68, 0x02);
    int s = bcd_to_dec(sec & 0x7F);
    int m = bcd_to_dec(min);
    int h = bcd_to_dec(hour & 0x3F);
    printf("RTC Time: %02d:%02d:%02d\n", h, m, s);
    return 0;
}
```

Expert Tips

- DS3231 address 0x68.
- 24h format.
- Set if not running.
- Alarm features.

Project 250: Create a project to use SPI to read a temperature sensor and log data.

Project Description: Write a C program to read SPI temp sensor and log values over time.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Periodic read and log.
- **Steps**:
    1. Configure SPI.
    2. Read temp.
    3. Log to buffer or UART.
    4. Delay.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

float spi_read_temp(void) {
    unsigned char high = (rand() % 256);
    unsigned char low = (rand() % 256);
    int raw = (high << 8) | low;
    return raw * 0.0625;  // Simulate
}

int main(void) {
    // In real: SPI config
    float log[10];
    int index = 0;
    printf("Logging SPI temperature.\n");

    while (1) {
        float temp = spi_read_temp();
        log[index % 10] = temp;
        index++;
        printf("Log[%d]: %.2f°C\n", index-1, temp);
        if (index % 10 == 0) {
            float avg = 0;
            for (int i = 0; i < 10; i++) avg += log[i];
            printf("Average: %.2f°C\n", avg / 10);
        }
        delay_ms(5000);
    }
    return 0;
}
```

Expert Tips

- Persistent log.
- Timestamp.
- Min/max.
- Overflow.

Debugging and Basic Testing

Project 251: Write a program to print debug messages to a serial console.

Project Description: Develop a C program that prints debug messages (e.g., initialization status, variable values) to a serial console using UART for monitoring program execution.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: UART-based logging with printf-like output.
- **Steps**:
    1. Configure UART for serial output (e.g., 9600 baud).
    2. Define a debug function to print formatted messages with timestamps or labels.
    3. In main, print initialization messages and loop with periodic debug output.
    4. Use printf for simulation; in real, implement UART transmit.

C Code Implementation

```
#include <stdio.h>  // For simulation; use <avr/io.h> in real
// In real: #include <avr/io.h>, #include <util/setbaud.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void debug_print(const char* msg) {
    printf("[DEBUG] %s\n", msg);  // In real: UART transmit with timestamp
}

int main(void) {
    // In real: UART config (UBRR = 103 for 9600 baud at 16MHz)
    debug_print("System initialized");
    debug_print("Entering main loop");

    int counter = 0;
    while (1) {
        char buf[20];
        sprintf(buf, "Counter: %d", counter++);
        debug_print(buf);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use macros like #define DEBUG_PRINT(...) for conditional compilation to disable debug in production.
- Implement a ring buffer for debug messages to avoid blocking.
- Add severity levels (INFO, WARN, ERROR) for filtering.
- In real systems, use a USB-serial adapter and tools like PuTTY for viewing.

Project 252: Create a project to toggle an LED if a GPIO pin is stuck high.

Project Description: Write a C program to monitor a GPIO input pin and toggle an LED if the input remains stuck high for more than 5 seconds, indicating a fault.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timeout-based fault detection with timer.
- **Steps**:
    1. Configure input pin and LED output.
    2. Use a timer or counter to track high state duration.
    3. If high > 5s, toggle LED and print fault message.
    4. Reset counter on low.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_gpio(void) {
    return 1;  // Simulate stuck high; in real: (PINB & (1 << 1)) ? 1 : 0
}

int main(void) {
    // In real: DDRB |= (1 << 0); // LED; DDRB &= ~(1 << 1); // Input
    int high_count = 0;
    int led_state = 0;
    printf("Monitoring GPIO for stuck high.\n");

    while (1) {
        if (read_gpio()) {
            high_count++;
            if (high_count > 5000) {  // 5s at 1ms intervals
                led_state = !led_state;
                printf("FAULT: GPIO stuck high! LED toggled to %d\n", led_state);
                // In real: PORTB ^= (1 << 0);
                high_count = 0;  // Reset after fault
            }
        } else {
            high_count = 0;
        }
        delay_ms(1);
    }
    return 0;
}
```

Expert Tips

- Use hardware timer for accurate timing instead of delay loops.
- Add pull-down resistor to prevent floating inputs.
- Log fault events to non-volatile memory.
- Implement recovery mechanism after fault detection.

Project 253: Implement a project to log button press events to a serial console.

Project Description: Write a C program to detect button presses with debouncing and log each event (timestamp, state) to the serial console.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Debounced edge detection with logging.
- **Steps**:
    1. Configure button input with pull-up.
    2. Implement 50ms debounce timer.
    3. On confirmed press/release, log event with counter as timestamp.
    4. Print to console.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_button(void) {
    return (rand() % 2);  // Simulate; in real: !(PINB & (1 << 1))
}

int main(void) {
    // In real: DDRB &= ~(1 << 1); PORTB |= (1 << 1);
    int debounce_count = 0;
    int last_state = 1;
    int event_count = 0;
    printf("Logging button events.\n");

    while (1) {
        int current = read_button();
        if (current != last_state) {
            debounce_count++;
            if (debounce_count > 50) {  // 50ms debounce
                if (current != last_state) {
                    event_count++;
                    char buf[30];
                    sprintf(buf, "Event %d: Button %s (time: %d)\n", event_count, current ? "RELEASED"
: "PRESSED", event_count * 100);
                    printf("%s", buf);
                    last_state = current;
                    debounce_count = 0;
                }
            }
        } else {
            debounce_count = 0;
        }
        delay_ms(1);
    }
    return 0;
}
```

Expert Tips

- Use timer interrupt for precise debouncing.
- Store logs in array for later dump.
- Add edge type (rising/falling).
- Filter noise with capacitor.

Project 254: Configure a program to print ADC values for debugging.

Project Description: Write a C program to continuously read ADC from a channel and print raw values to the console for debugging sensor issues.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Continuous ADC polling and output.
- **Steps**:
    1. Configure ADC (channel, reference).
    2. In loop, read ADC and print raw value.
    3. Add delay to control output rate.
    4. Print voltage for context.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(int channel) {
    return (rand() % 1024);  // Simulate; in real: ADMUX = channel | (1 << REFS0); ADCSRA |= (1 << ADSC); while (ADCSRA & (1 << ADSC)); return ADC;
}

int main(void) {
    // In real: ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);
    printf("Continuous ADC debugging (Channel 0):\n");

    while (1) {
        unsigned int val = read_adc(0);
        float voltage = (val * 5.0) / 1023.0;
        printf("ADC: %u (%.2fV)\n", val, voltage);
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Enable free-running mode for continuous reads.
- Average 16 samples to reduce noise.
- Log to file for analysis.
- Check reference voltage stability.

Project 255: Create a project to use assertions to check GPIO states.

Project Description: Write a C program that uses assert to verify GPIO pin states during initialization and runtime, halting on failure with debug message.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Assert-based validation.
- **Steps**:
    1. Include <assert.h>.
    2. After GPIO config, assert pin state (e.g., input high with pull-up).
    3. In loop, periodically assert conditions.
    4. On failure, print error.

C Code Implementation

```
#include <stdio.h>
#include <assert.h>  // For simulation; define assert macro in real
// In real: #define assert(expr) if (!(expr)) { printf("ASSERT FAILED: %s\n", #expr); while(1); }

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_gpio(void) {
    return 1;  // Simulate
}

int main(void) {
    // In real: DDRB &= ~(1 << 1); PORTB |= (1 << 1);
    assert(read_gpio() == 1);  // Check pull-up
    printf("GPIO assertion passed.\n");

    while (1) {
        assert(read_gpio() == 1);  // Runtime check
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Disable asserts in release with NDEBUG.
- Use static asserts for compile-time.
- Log assert failures to UART.
- Recover instead of halt in production.

Project 256: Implement a project to log timer interrupt events to a serial console.

Project Description: Write a C program to log timer interrupt occurrences (count, timestamp) to the serial console for timing analysis.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: ISR logging with UART.
- **Steps**:
    1. Configure timer interrupt.
    2. In ISR, increment counter and flag log.
    3. In main, check flag and print to UART.
    4. Use delay to simulate.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>

volatile int interrupt_count = 0;
volatile int log_flag = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void isr_timer(void) {
    interrupt_count++;
    log_flag = 1;
    // In real: ISR body
}

int main(void) {
    // In real: Timer config, enable interrupt
    printf("Logging timer interrupts.\n");

    while (1) {
        if (log_flag) {
            printf("Timer interrupt #%d\n", interrupt_count);
            log_flag = 0;
        }
        delay_ms(100);  // Simulate interrupt every 100ms
        isr_timer();
    }
    return 0;
}
```

Expert Tips

- Use a queue for log entries to avoid ISR blocking.
- Timestamp with high-res timer.
- Rate-limit logging.
- Use JTAG for real-time debug.

Project 257: Write a program to print memory usage for debugging.

Project Description: Write a C program to monitor and print heap/stack usage during runtime for memory leak detection.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Manual memory tracking.
- **Steps**:
    1. Define stack/heap markers.
    2. Allocate/deallocate and track bytes used.
    3. Periodically print usage.
    4. Alert on high usage.

C Code Implementation

```c
#include <stdio.h>
#include <stdlib.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    size_t heap_used = 0;
    printf("Memory usage debugging.\n");

    while (1) {
        // Simulate allocation
        void* ptr = malloc(100);
        if (ptr) heap_used += 100;
        printf("Heap used: %zu bytes\n", heap_used);
        // In real: free(ptr); heap_used -= 100;

        if (heap_used > 1000) {
            printf("WARNING: High memory usage!\n");
        }
        delay_ms(2000);
    }
    return 0;
}
```

Expert Tips

- Use __malloc_size for heap in some MCUs.
- Stack canary for overflow detection.
- Static analysis tools like PC-Lint.
- Free memory in production.

Project 258: Create a project to log sensor data with timestamps.

Project Description: Write a C program to read a sensor (ADC) and log data with simulated timestamps to the console.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timestamped logging.
- **Steps**:
    1. Simulate timestamp with counter.
    2. Read sensor.

3. Format "timestamp,sensor_value".
4. Print/log.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_sensor(void) {
    return (rand() % 1024);
}

int main(void) {
    unsigned long timestamp = 0;
    printf("Sensor logging with timestamps.\n");

    while (1) {
        unsigned int value = read_sensor();
        char buf[30];
        sprintf(buf, "%lu,%u\n", timestamp++, value);
        printf("%s", buf);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use RTC for real timestamps.
- CSV for easy import to Excel.
- Log to EEPROM or SD.
- Compression for storage.

Project 259: Implement a project to use a breakpoint to debug a loop.

Project Description: Write a C program with a loop that has a potential infinite loop issue; include comments for breakpoint placement to debug.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Breakpoint simulation with conditional print.
- **Steps**:
    1. Implement loop with condition.
    2. Add debug prints or halts for breakpoints.
    3. Simulate infinite loop detection.
    4. Print loop count.

C Code Implementation

```c
#include <stdio.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    int loop_count = 0;
    int condition = 1;  // Simulate variable
    printf("Debug loop with breakpoints.\n");

    while (condition && loop_count < 100) {  // Breakpoint here: check condition
        loop_count++;  // Breakpoint here: monitor count
        printf("Loop iteration: %d\n", loop_count);
        if (loop_count % 10 == 0) condition = 0;  // Exit condition
        delay_ms(100);
    }
    if (loop_count >= 100) printf("Infinite loop detected!\n");
    return 0;
}
```

Expert Tips

- Use GDB or AVR Studio for real breakpoints.
- Watch variables in debugger.
- Step into/out/over.
- Conditional breakpoints.

Project 260: Configure a program to print error codes for invalid inputs.

Project Description: Write a C program that validates inputs (e.g., ADC channel) and prints specific error codes on invalid cases.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Input validation with enums.
- **Steps**:
    1. Define error enum.
    2. Check input range.
    3. Print error code and message.
    4. Handle valid case.

C Code Implementation

```c
#include <stdio.h>

enum ErrorCode { ERR_NONE = 0, ERR_INVALID_CHANNEL = 1, ERR_OUT_OF_RANGE = 2 };

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```c
int main(void) {
    int channel = 5;  // Simulate input; valid 0-7
    enum ErrorCode err = ERR_NONE;

    if (channel < 0 || channel > 7) {
        err = ERR_INVALID_CHANNEL;
    }

    if (err != ERR_NONE) {
        printf("Error %d: Invalid input\n", err);
    } else {
        printf("Valid channel %d\n", channel);
    }
    return 0;
}
```

Expert Tips

- Use switch for error handling.
- Return codes from functions.
- Log errors with UART.
- Graceful degradation.

Project 261: Create a project to log UART receive errors to a serial console.

Project Description: Write a C program to detect UART errors (framing, parity, overrun) and log them to another console or buffer.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: UART status check.
- **Steps**:
    1. Configure UART.
    2. On RX, check error flags.
    3. Log error type.
    4. Clear flags.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

char uart_receive(void) {
    char c = getchar();  // Simulate
    if (rand() % 10 == 0) printf("UART Error: Framing\n");  // Simulate error
    return c;
}

int main(void) {
    // In real: UART config
    printf("Logging UART errors.\n");
```

```
    while (1) {
        char c = uart_receive();
        printf("Received: %c\n", c);
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Check UCSRA flags (FE, DOR, UPE).
- Clear errors after logging.
- Retry on error.
- Buffer to avoid loss.

Project 262: Implement a project to use a debugger to step through a state machine.

Project Description: Write a C program implementing a simple state machine with debug points for stepping in a debugger.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: State machine with debug prints.
- **Steps**:
    1. Define states enum.
    2. Switch on state with debug prints.
    3. Transition based on input.
    4. Comment breakpoint locations.

C Code Implementation

```
#include <stdio.h>

enum State { STATE1, STATE2, STATE3 };

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    enum State current = STATE1;
    int input = 0;
    printf("State machine for debugging.\n");

    while (1) {
        // Breakpoint: Check current state
        switch (current) {
            case STATE1:
                printf("In STATE1\n");  // Breakpoint: Step here
                if (input == 1) current = STATE2;
                break;
            case STATE2:
                printf("In STATE2\n");  // Breakpoint: Monitor transition
                current = STATE3;
                break;
```

```
            case STATE3:
                printf("In STATE3\n");
                current = STATE1;
                break;
        }
        input = (input + 1) % 2;  // Simulate input
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use GDB: break main, step, watch variables.
- Print state entry/exit.
- Conditional transitions.
- Visualize with UML.

Project 263: Write a program to print stack usage for debugging.

Project Description: Write a C program to track and print maximum stack usage during runtime using a canary value.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Stack watermarking.
- **Steps**:
    1. Place canary at stack end.
    2. In functions, check canary for overflow.
    3. Track max depth.
    4. Print usage.

C Code Implementation

```
#include <stdio.h>

volatile char stack_canary = 0xAA;

void deep_function(int depth) {
    if (depth > 0) deep_function(depth - 1);
    if (stack_canary != 0xAA) printf("Stack overflow!\n");
}

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    int max_depth = 0;
    printf("Stack usage debugging.\n");

    for (int d = 1; d < 50; d++) {
        deep_function(d);
        max_depth = d;
        printf("Max stack depth: %d\n", max_depth);
        delay_ms(100);
    }
```

```
    return 0;
}
```

Expert Tips

- Use linker script for stack size.
- __stack for canary in GCC.
- Periodic check in idle task.
- Increase stack if overflow.

Project 264: Create a project to log I2C communication errors.

Project Description: Write a C program to detect I2C errors (NACK, timeout) and log them with details to console.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: I2C status checking.
- **Steps**:
    1. During I2C transaction, check TWSR.
    2. On error, log code and address.
    3. Retry or halt.
    4. Print log.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void i2c_transaction(unsigned char addr) {
    if (rand() % 5 == 0) {  // Simulate 20% error
        printf("I2C Error: NACK from 0x%02X\n", addr);
    } else {
        printf("I2C OK to 0x%02X\n", addr);
    }
}

int main(void) {
    // In real: I2C config
    printf("Logging I2C errors.\n");

    while (1) {
        i2c_transaction(0x68);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Check TWINT, TWA, TWSTA flags.
- Timeout with timer.

- Bus recovery (clock stretch).
- SCL/SDA pull-ups.

Project 265: Implement a project to use a debugger to monitor a variable.

Project Description: Write a C program with a variable that changes in a loop; include comments for debugger watchpoints.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Variable monitoring simulation.
- **Steps**:
    1. Declare global variable.
    2. Update in loop.
    3. Print value (simulate watch).
    4. Comment watchpoint.

C Code Implementation

```c
#include <stdio.h>

volatile int monitored_var = 0;  // Watch this in debugger

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    printf("Monitor 'monitored_var' in debugger.\n");

    while (1) {
        // Watchpoint: Break when monitored_var == 50
        monitored_var = (monitored_var + 1) % 100;
        printf("Variable: %d\n", monitored_var);
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- GDB: watch monitored_var.
- Conditional watch: watch if expr.
- Hardware watchpoints limited.
- Printf for simulation.

Project 266: Configure a program to print timer overflow events.

Project Description: Write a C program to detect and print timer overflow events with count and timestamp.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Overflow interrupt logging.

- **Steps**:
    1. Configure timer overflow interrupt.
    2. In ISR, increment counter and log.
    3. Print in main.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>

volatile int overflow_count = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void isr_overflow(void) {
    overflow_count++;
    printf("Timer overflow #%d\n", overflow_count);
}

int main(void) {
    // In real: TCCR0B = (1 << CS02); TIMSK0 = (1 << TOIE0); sei();
    printf("Logging timer overflows.\n");

    while (1) {
        delay_ms(16);  // Simulate overflow every 16ms
        isr_overflow();
    }
    return 0;
}
```

Expert Tips

- Use 16-bit timer for less frequent overflows.
- Pre-scale to adjust rate.
- Chain timers for longer periods.
- ISR minimal code.

Project 267: Create a project to log SPI communication failures.

Project Description: Write a C program to detect SPI errors (e.g., mode mismatch) and log failures to console.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: SPI status check.
- **Steps**:
    1. During SPI transfer, check flags.
    2. On failure, log error type.
    3. Retry.
    4. Print.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int spi_transfer(unsigned char data) {
    if (rand() % 10 == 0) {  // 10% failure
        printf("SPI Failure: Transfer error\n");
        return -1;
    }
    return data;  // Success
}

int main(void) {
    // In real: SPI config
    printf("Logging SPI failures.\n");

    while (1) {
        int result = spi_transfer(0x55);
        if (result < 0) {
            printf("Retry transfer...\n");
        } else {
            printf("SPI OK: 0x%02X\n", result);
        }
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Check SPSR SPIF and WCOL.
- Slave timeout.
- Bus capacitance.
- Oscilloscope for signals.

Project 268: Implement a project to use a debugger to check register values.

Project Description: Write a C program that manipulates registers; include comments for debugger register inspection.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Register access with debug points.
- **Steps**:
    1. Access MCU registers (simulated).
    2. Modify and print.
    3. Comment inspection points.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

volatile unsigned char reg_sim = 0x00;  // Simulate PORTB

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    printf("Check registers in debugger.\n");

    // Breakpoint: Inspect reg_sim before write
    reg_sim |= 0x01;  // Set bit 0
    printf("Register after set: 0x%02X\n", reg_sim);

    // Breakpoint: Inspect after toggle
    reg_sim ^= 0x01;
    printf("Register after toggle: 0x%02X\n", reg_sim);

    while (1) delay_ms(1000);
    return 0;
}
```

Expert Tips

- GDB: info registers.
- Memory window for SFRs.
- Watch specific bits.
- Simulator for offline debug.

Project 269: Write a program to print ADC conversion errors.

Project Description: Write a C program to detect ADC errors (e.g., timeout, reference issue) and print error details.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: ADC status validation.
- **Steps**:
    1. Start conversion.
    2. Wait with timeout.
    3. Check ADIF, print error if not set.
    4. Log.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```c
unsigned int adc_convert(void) {
    if (rand() % 20 == 0) {  // 5% error
        printf("ADC Error: Conversion timeout\n");
        return 0;
    }
    return (rand() % 1024);  // Success
}

int main(void) {
    // In real: ADCSRA |= (1 << ADSC);
    printf("ADC conversion with error logging.\n");

    while (1) {
        unsigned int val = adc_convert();
        if (val > 0) printf("ADC: %u\n", val);
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Check ADSC flag for busy.
- Reference stability.
- Interrupt for completion.
- Noise filtering.

Project 270: Create a project to log button debounce issues.

Project Description: Write a C program to detect and log button bounce duration exceeding expected (e.g., >20ms).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bounce timing log.
- **Steps**:
    1. Detect edge.
    2. Time unstable period.
    3. If > threshold, log issue.
    4. Debounce properly.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_button(void) {
    return (rand() % 2);  // Simulate bounce
}

int main(void) {
    // In real: Input config
    int stable_count = 0;
```

```
    int last_state = 1;
    int bounce_start = 0;
    printf("Logging button debounce issues.\n");

    while (1) {
        int state = read_button();
        if (state != last_state) {
            if (stable_count > 20) {  // Debounced
                printf("Valid press\n");
            } else if (stable_count > 0) {
                printf("Bounce issue: %d ms\n", stable_count);
            }
            stable_count = 0;
        } else {
            stable_count++;
        }
        last_state = state;
        delay_ms(1);
    }
    return 0;
}
```

Expert Tips

- Hardware debounce with capacitor.
- Log bounce duration histogram.
- Adjust software threshold.
- Test with scope.

Project 271: Implement a project to use a debugger to trace a function call.

Project Description: Write a C program with recursive function; comment for call tracing in debugger.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Recursive trace with prints.
- **Steps**:
    1. Implement factorial recursive.
    2. Print entry/exit.
    3. Comment trace points.

C Code Implementation

```
#include <stdio.h>

int factorial(int n) {
    printf("Entering factorial(%d)\n", n);  // Breakpoint: Trace entry
    if (n <= 1) {
        printf("Returning 1 from factorial(%d)\n", n);  // Breakpoint: Base case
        return 1;
    }
    int result = n * factorial(n - 1);  // Breakpoint: Recursive call
    printf("Returning %d from factorial(%d)\n", result, n);  // Breakpoint: Exit
    return result;
}

int main(void) {
    printf("Trace factorial calls.\n");
    int res = factorial(5);
```

```
    printf("Result: %d\n", res);
    return 0;
}
```

Expert Tips

- GDB: backtrace on break.
- Step into recursion.
- Watch stack depth.
- Avoid deep recursion in embedded.

Project 272: Configure a program to print memory allocation errors.

Project Description: Write a C program using malloc; print errors on allocation failure.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Malloc check.
- **Steps**:
    1. Attempt malloc.
    2. If NULL, print error.
    3. Track total allocated.
    4. Free periodically.

C Code Implementation

```c
#include <stdio.h>
#include <stdlib.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    size_t total = 0;
    printf("Memory allocation debugging.\n");

    while (1) {
        void* ptr = malloc(100);
        if (ptr == NULL) {
            printf("Allocation error: Out of memory (total: %zu)\n", total);
        } else {
            total += 100;
            printf("Allocated 100 bytes, total: %zu\n", total);
        }
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Use static allocation in embedded.
- Heap fragmentation check.
- Custom allocator.

- Monitor free heap.

Project 273: Create a project to log interrupt conflicts for debugging.

Project Description: Write a C program to detect nested interrupts or priority issues and log conflicts.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Nesting counter.
- **Steps**:
    1. Global nest level.
    2. Increment in ISR entry.
    3. If >1, log conflict.
    4. Decrement exit.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>

volatile int nest_level = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void isr_sim(void) {
    nest_level++;
    if (nest_level > 1) printf("Interrupt nesting conflict: level %d\n", nest_level);
    // ISR body
    nest_level--;
}

int main(void) {
    printf("Logging interrupt conflicts.\n");

    while (1) {
        delay_ms(100);
        isr_sim();  // Simulate interrupt
        if (rand() % 10 == 0) isr_sim();  // Simulate nesting
    }
    return 0;
}
```

Expert Tips

- Global interrupt enable/disable.
- Priority levels.
- RTOS for handling.
- Scope for signal integrity.

Project 274: Implement a project to use a debugger to monitor GPIO changes.

Project Description: Write a C program that changes GPIO in loop; comment for data breakpoint on change.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: GPIO toggle with watch.
- **Steps**:
    1. Toggle GPIO variable.
    2. Print change.
    3. Comment watchpoint.

C Code Implementation

```c
#include <stdio.h>

volatile int gpio_state = 0;  // Watch this for changes

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    printf("Monitor GPIO changes in debugger.\n");

    while (1) {
        // Data breakpoint: Break on gpio_state write
        gpio_state = !gpio_state;
        printf("GPIO state changed to %d\n", gpio_state);
        // In real: PORTB = gpio_state;
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- GDB: watch gpio_state.
- Hardware breakpoint on address.
- Conditional on value.
- Logic analyzer alternative.

Project 275: Write a program to print sensor calibration errors.

Project Description: Write a C program to calibrate sensor (ADC) and print deviation errors.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Calibration validation.
- **Steps**:
    1. Read known values.
    2. Calculate expected vs actual.
    3. If error > tolerance, print.
    4. Log max error.

C Code Implementation

```c
#include <stdio.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_sensor(float expected) {
    return (unsigned int)(expected * 205 + (rand() % 20 - 10));  // Simulate with noise
}

int main(void) {
    float tolerance = 5.0;
    printf("Sensor calibration error logging.\n");

    for (int i = 0; i < 5; i++) {
        float expected = i * 1.0;
        unsigned int actual = read_sensor(expected);
        float error = fabs(actual / 205.0 - expected);
        if (error > tolerance) {
            printf("Calibration error at %fV: %.2f (tolerance %.1f)\n", expected, error, tolerance);
        }
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Multi-point calibration.
- Store offsets in EEPROM.
- Auto-recalibrate.
- Temperature compensation.

Project 276: Create a project to log UART buffer overflow events.

Project Description: Write a C program with UART RX buffer; log overflows when full.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Buffer management.
- **Steps**:
    1. Define circular buffer.
    2. On RX, add if space; else log overflow.
    3. Print buffer on command.

C Code Implementation

```c
#include <stdio.h>
#define BUFFER_SIZE 10

char rx_buffer[BUFFER_SIZE];
int head = 0, tail = 0;
int overflow_count = 0;
```

```
void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void rx_char(char c) {
    if ((head + 1) % BUFFER_SIZE == tail) {
        overflow_count++;
        printf("UART Buffer overflow #%d\n", overflow_count);
    } else {
        rx_buffer[head] = c;
        head = (head + 1) % BUFFER_SIZE;
    }
}

int main(void) {
    printf("UART buffer overflow logging.\n");

    while (1) {
        char sim_c = 'A' + (rand() % 26);
        rx_char(sim_c);
        if (overflow_count > 0) printf("Current overflows: %d\n", overflow_count);
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Interrupt-driven RX.
- Increase buffer size.
- Flow control (RTS/CTS).
- Discard or oldest on overflow.

Project 277: Implement a project to use a debugger to step through an ISR.

Project Description: Write a C program with ISR; comment for stepping in debugger.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: ISR with debug prints.
- **Steps**:
    1. Configure interrupt.
    2. ISR with steps and prints.
    3. Simulate trigger.

C Code Implementation

```
#include <stdio.h>
// In real: #include <avr/io.h>, #include <avr/interrupt.h>

volatile int isr_trigger = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```c
void isr_example(void) {
    printf("ISR entry\n");  // Breakpoint 1
    int local = 0;  // Breakpoint 2: Inspect local
    local++;  // Step here
    printf("ISR local: %d\n", local);  // Breakpoint 3
    // In real: ISR body
}

int main(void) {
    printf("Step through ISR in debugger.\n");

    while (1) {
        if (rand() % 10 == 0) isr_example();  // Trigger
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- GDB: handle sigint for interrupts.
- Disable global interrupts in ISR.
- Minimal ISR code.
- Simulator breakpoints.

Project 278: Configure a program to print timer configuration errors.

Project Description: Write a C program to validate timer setup (prescaler, mode) and print errors if invalid.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Configuration validation.
- **Steps**:
    1. Set timer params.
    2. Check valid combinations.
    3. Print error if invalid.
    4. Proceed if OK.

C Code Implementation

```c
#include <stdio.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    int prescaler = 1024;  // Simulate
    int mode = 1;  // CT    printf("Timer config validation.\n");

    if (prescaler > 1024 || prescaler < 1) {
        printf("Error: Invalid prescaler %d\n", prescaler);
    } else if (mode < 0 || mode > 15) {
        printf("Error: Invalid mode %d\n", mode);
    } else {
        printf("Timer config OK: Prescaler %d, Mode %d\n", prescaler, mode);
    }
    return 0;               }
```

Expert Tips

- Datasheet limits.
- Calculate overflow time.
- Auto-correct if possible.
- Log at startup.

Project 279: Create a project to log I2C timeout errors.

Project Description: Write a C program to implement I2C with timeout and log if no response.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timeout counter.
- **Steps**:
    1. Start I2C transaction.
    2. Count clocks if no ACK.
    3. If timeout, log.
    4. Abort.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int i2c_start(unsigned char addr) {
    if (rand() % 5 == 0) {  // 20% timeout
        printf("I2C Timeout error for addr 0x%02X\n", addr);
        return -1;
    }
    return 0;  // OK
}

int main(void) {
    printf("Logging I2C timeouts.\n");

    while (1) {
        i2c_start(0x68);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Timer for timeout.
- Bus recovery (9 clocks).
- SCL stuck check.
- Retry logic.

Project 280: Implement a project to use a debugger to check loop execution time.

Project Description: Write a C program with a loop; use debugger to measure execution time with breakpoints.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timed loop with prints.
- **Steps**:
    1. Loop with counter.
    2. Print start/end timestamps (simulate).
    3. Comment timing breakpoints.

C Code Implementation

```c
#include <stdio.h>

volatile unsigned long start_time, end_time;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    printf("Measure loop time in debugger.\n");

    for (int i = 0; i < 1000; i++) {
        start_time = 0;  // Breakpoint: Start timer
        // Loop body
        volatile int dummy = i * 2;  // Simulate work
        end_time = 1;  // Breakpoint: End timer, calculate delta
        // In debugger: end - start
    }
    printf("Loop completed.\n");
    return 0;
}
```

Expert Tips

- GDB: info clock or perf.
- High-res timer.
- Optimize flags off for debug.
- Profile with tools.

Project 281: Write a program to print stack overflow warnings.

Project Description: Write a C program to periodically check stack pointer and warn if near end.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Stack pointer monitoring.
- **Steps**:
    1. Get stack start/end from linker.
    2. Check SP distance to end.
    3. Warn if < threshold.

4. Print.

C Code Implementation

```c
#include <stdio.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    unsigned long stack_end = 0x08FF;  // Simulate stack end
    unsigned long threshold = 100;
    printf("Stack overflow monitoring.\n");

    while (1) {
        unsigned long sp_sim = 0x0800 + (rand() % 200);  // Simulate SP
        unsigned long usage = stack_end - sp_sim;
        if (usage < threshold) {
            printf("Stack overflow warning: %lu bytes left\n", usage);
        } else {
            printf("Stack usage: %lu bytes\n", usage);
        }
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Linker script for stack size.
- Interrupt vector for canary.
- RTOS stack check.
- Increase if needed.

Project 282: Create a project to log SPI CRC errors for debugging.

Project Description: Write a C program to implement SPI with CRC and log calculation mismatches.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: CRC validation.
- **Steps**:
    1. Send data + CRC.
    2. Receive and recalculate CRC.
    3. If mismatch, log error.
    4. Print.

C Code Implementation

```c
#include <stdio.h>

unsigned char calc_crc(unsigned char data) {
    return data ^ 0xFF;  // Simple CR}
```

```c
void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    unsigned char tx_data = 0x55;
    unsigned char rx_data = 0x55 ^ 0x01;  // Simulate error
    unsigned char tx_crc = calc_crc(tx_data);
    unsigned char rx_crc = rx_data ^ 0xFF;  // Recal    if (tx_crc != rx_crc) {
        printf("SPI CRC error: Expected 0x%02X, got 0x%02X\n", tx_crc, rx_crc);
    }
    return 0;
}
```

Expert Tips

- CRC-8 or CCITT.
- Hardware CRC peripheral.
- Log packet.
- Retry on error.

Project 283: Implement a project to use a debugger to monitor ADC values.

Project Description: Write a C program reading ADC; comment for watch on ADC result.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: ADC read with watch.
- **Steps**:
    1. Read ADC.
    2. Store in variable.
    3. Print (simulate watch).

C Code Implementation

```c
#include <stdio.h>

volatile unsigned int adc_value = 0;  // Watch this

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int read_adc(void) {
    adc_value = (rand() % 1024);  // Simulate
    return adc_value;  // Breakpoint: Watch adc_value change
}

int main(void) {
    printf("Monitor ADC value in debugger.\n");

    while (1) {
        unsigned int val = read_adc();
        printf("ADC: %u\n", val);
        delay_ms(100);
    }
    return 0;               }
```

Expert Tips

- GDB: watch adc_value.
- Conditional: watch if >500.
- Plot in debugger.
- DMA for continuous.

Project 284: Configure a program to print button press timing errors.

Project Description: Write a C program to measure button press duration and log if outside expected range (50ms-2s).

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timing measurement.
- **Steps**:
    1. Detect press start.
    2. Measure duration.
    3. If out of range, log error.
    4. Print.

C Code Implementation

```c
#include <stdio.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int read_button(void) {
    static int pressed = 0;
    pressed = (rand() % 2000 < 100) ? 1 : 0;  // Simulate press
    return pressed;
}

int main(void) {
    int start_time = 0;
    int pressed = 0;
    printf("Button timing error logging.\n");

    while (1) {
        int btn = read_button();
        if (btn && !pressed) {
            start_time = 0;  // Start timer
            pressed = 1;
        } else if (!btn && pressed) {
            int duration = 100;  // Simulate
            if (duration < 50 || duration > 2000) {
                printf("Timing error: Press %d ms (expected 50-2000)\n", duration);
            }
            pressed = 0;
        }
        delay_ms(1);
    }
    return 0;
}
```

Expert Tips

- Use timer for accurate measure.
- Log histogram.
- Debounce separately.
- User feedback.

Project 285: Create a project to log UART framing errors.

Project Description: Write a C program to detect UART framing errors and log count/type.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Error flag check.
- **Steps**:
    1. On RX, check FE flag.
    2. If set, log and clear.
    3. Count errors.
    4. Print.

C Code Implementation

```c
#include <stdio.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

char uart_rx(void) {
    char c = 'A';
    if (rand() % 20 == 0) {
        printf("Framing error on RX\n");
    }
    return c;
}

int main(void) {
    int error_count = 0;
    printf("Logging UART framing errors.\n");

    while (1) {
        uart_rx();
        error_count++;  // Simulate
        if (error_count % 10 == 0) printf("Total framing errors: %d\n", error_count);
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- UCSRA & (1 << FE).
- Baud rate mismatch cause.
- Clear error.
- Auto-baud detect.

Project 286: Implement a project to use a debugger to trace a state transition.

Project Description: Write a C program with state machine; comment transitions for tracing.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: State trace prints.
- **Steps**:
    1. Enum states.
    2. Switch with transition logs.
    3. Comment trace.

C Code Implementation

```c
#include <stdio.h>

enum State { IDLE, RUNNING, ERROR };

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    enum State state = IDLE;
    printf("Trace state transitions.\n");

    while (1) {
        // Breakpoint: Before switch
        switch (state) {
            case IDLE:
                printf("IDLE -> RUNNING\n");  // Trace transition
                state = RUNNING;  // Breakpoint here
                break;
            case RUNNING:
                if (rand() % 10 == 0) {
                    printf("RUNNING -> ERROR\n");
                    state = ERROR;
                }
                break;
            case ERROR:
                printf("ERROR -> IDLE\n");
                state = IDLE;
                break;
        }
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Log old/new state.
- GDB: watch state.
- Diagram for complex.
- Enter/exit actions.

Project 287: Write a program to print memory alignment errors.

Project Description: Write a C program to check pointer alignment (e.g., 4-byte) and print errors.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Alignment validation.
- **Steps**:
    1. Malloc and check %4 ==0.
    2. If not, print error.
    3. Use aligned_alloc if available.

C Code Implementation

```c
#include <stdio.h>
#include <stdlib.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    void* ptr = malloc(100);
    if (((unsigned long)ptr % 4) != 0) {
        printf("Memory alignment error: ptr %p not 4-byte aligned\n", ptr);
    } else {
        printf("Aligned OK: %p\n", ptr);
    }
    free(ptr);
    return 0;
}
```

Expert Tips

- **attribute**((aligned(4))).
- DMA requires alignment.
- Compiler flags.
- Check all pointers.

Project 288: Create a project to log timer interrupt latency.

Project Description: Write a C program to measure and log time from interrupt trigger to ISR entry.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: High-res timing.
- **Steps**:
    1. Timestamp on trigger.
    2. Timestamp in ISR.
    3. Calculate latency.
    4. Log if high.

C Code Implementation

```c
#include <stdio.h>

volatile unsigned long trigger_time = 0;
volatile unsigned long isr_time = 0;
volatile int latency_log = 0;

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

void simulate_trigger(void) {
    trigger_time = 0;  // High-res time
}

void isr_entry(void) {
    isr_time = 1;  // High-res time
    int latency = isr_time - trigger_time;
    if (latency > 10) {  // Threshold
        printf("Interrupt latency: %d us\n", latency);
    }
}

int main(void) {
    printf("Logging timer latency.\n");

    while (1) {
        simulate_trigger();
        delay_ms(1);  // Simulate delay
        isr_entry();
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Use cycle counter.
- RTOS latency.
- Priority adjustment.
- Oscilloscope.

Project 289: Implement a project to use a debugger to monitor sensor data.

Project Description: Write a C program reading sensor; comment for watch on data variable.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Sensor read with watch.
- **Steps**:
    1. Read sensor.
    2. Store in variable.
    3. Print.

C Code Implementation

```c
#include <stdio.h>

volatile float sensor_data = 0.0;  // Watch this

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

float read_sensor(void) {
    sensor_data = 25.0 + (rand() % 10 - 5);  // Simulate
    return sensor_data;  // Breakpoint: Monitor change
}

int main(void) {
    printf("Monitor sensor_data in debugger.\n");

    while (1) {
        float val = read_sensor();
        printf("Sensor: %.2f\n", val);
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- Watch expression: sensor_data > 30.
- Plot over time.
- Conditional break.
- Log for analysis.

Project 290: Configure a program to print I2C bus errors.

Project Description: Write a C program to detect I2C bus errors (arbitration lost, bus error) and print details.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: TWCR status check.
- **Steps**:
    1. After transaction, check TWSR.
    2. If error code, print.
    3. Clear.

C Code Implementation

```c
#include <stdio.h>
// In real: #include <avr/io.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```c
int i2c_bus_check(void) {
    if (rand() % 10 == 0) {
        printf("I2C Bus error: Arbitration lost\n");
        return -1;
    }
    return 0;
}

int main(void) {
    printf("I2C bus error logging.\n");

    while (1) {
        i2c_bus_check();
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- TWSR error codes (0x38 arb lost).
- SCL/SDA monitoring.
- Pull-up strength.
- Recovery sequence.

Project 291: Create a project to log SPI slave select issues.

Project Description: Write a C program to detect SS timing errors (too short/long) and log.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Timing validation.
- **Steps**:
    1. Assert/deassert SS.
    2. Measure duration.
    3. If out of range, log.
    4. Print.

C Code Implementation

```c
#include <stdio.h>

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

int main(void) {
    int min_time = 10;  // us
    int max_time = 1000;
    printf("SPI SS timing logging.\n");

    while (1) {
        unsigned long start = 0;
        // SS low
        delay_us(50 + (rand() % 900));  // Variable time
        unsigned long duration = 50 + (rand() % 900);  // Simulate
```

```
        if (duration < min_time || duration > max_time) {
            printf("SS timing error: %lu us (range %d-%d)\n", duration, min_time, max_time);
        }
        // SS high
        delay_us(1000);
    }
    return 0;
}
```

Expert Tips

- Scope SS pulse.
- Minimum setup time.
- Glitch filter.
- Auto SS if multi-slave.

Project 292: Implement a project to use a debugger to check PWM settings.

Project Description: Write a C program setting PWM; comment for register inspection.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: PWM config with prints.
- **Steps**:
    1. Set timer registers.
    2. Print values.
    3. Comment inspect.

C Code Implementation

```
#include <stdio.h>

volatile unsigned char ocr_sim = 0;  // Simulate OCR0A

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    printf("Check PWM settings in debugger.\n");

    ocr_sim = 128;  // 50% duty  // Breakpoint: Inspect ocr_sim
    printf("PWM OCR: %d (50%% duty)\n", ocr_sim);
    // In real: OCR0A = 128; TCCR0A = (1 << COM0A1) | (1 << WGM01);
    delay_ms(1000);
    return 0;
}
```

Expert Tips

- GDB: p/x OCR0A.
- Verify frequency.
- Duty calculation.
- Scope output.

Project 293: Write a program to print ADC sampling errors.

Project Description: Write a C program to detect ADC sampling jitter or inconsistency and print errors.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Variance calculation.
- **Steps**:
    1. Sample multiple times.
    2. Calculate std dev.
    3. If high, error.
    4. Print.

C Code Implementation

```c
#include <stdio.h>
#include <math.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

unsigned int sample_adc(void) {
    return (rand() % 1024);
}

int main(void) {
    float threshold = 5.0;
    printf("ADC sampling error logging.\n");

    while (1) {
        float mean = 0, variance = 0;
        for (int i = 0; i < 10; i++) {
            unsigned int val = sample_adc();
            mean += val;
            delay_ms(1);
        }
        mean /= 10;
        for (int i = 0; i < 10; i++) {
            unsigned int val = sample_adc();
            variance += (val - mean) * (val - mean);
        }
        variance = sqrt(variance / 10);
        if (variance > threshold) printf("Sampling error: variance %.2f\n", variance);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- Clock stability.
- Averaging filter.
- External reference.
- Ground noise.

Project 294: Create a project to log button bounce issues for debugging.

Project Description: Write a C program to count and log button bounce events exceeding normal.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Bounce counter.
- **Steps**:
    1. Poll button fast.
    2. Count state changes in debounce window.
    3. If > expected, log.
    4. Print.

C Code Implementation

```c
#include <stdio.h>

void delay_us(int us) {
    volatile unsigned long i;
    for (i = 0; i < us * 10UL; i++);
}

int read_button(void) {
    return (rand() % 2);
}

int main(void) {
    int bounce_count = 0;
    int expected = 1;
    printf("Button bounce logging.\n");

    while (1) {
        int changes = 0;
        int last = read_button();
        for (int i = 0; i < 50; i++) {  // 50us window
            int curr = read_button();
            if (curr != last) changes++;
            last = curr;
            delay_us(1);
        }
        if (changes > expected) {
            bounce_count++;
            printf("Bounce issue: %d changes (count %d)\n", changes, bounce_count);
        }
        delay_us(10000);  // 10ms poll
    }
    return 0;
}
```

Expert Tips

- Hardware capacitor.
- Software median filter.
- Log duration.
- Test switch quality.

Project 295: Implement a project to use a debugger to monitor interrupt flags.

Project Description: Write a C program setting/clearing interrupt flags; comment for flag monitoring.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Flag manipulation.
- **Steps**:
    1. Set flag.
    2. Clear in ISR sim.
    3. Print flags.

C Code Implementation

```c
#include <stdio.h>

volatile unsigned char int_flag = 0;  // Watch flags

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    printf("Monitor interrupt flags.\n");

    while (1) {
        int_flag |= 0x01;  // Set flag  // Breakpoint: Check int_flag
        printf("Flag set: 0x%02X\n", int_flag);
        delay_ms(500);
        int_flag &= ~0x01;  // Clear  // Breakpoint: Monitor clear
        printf("Flag cleared: 0x%02X\n", int_flag);
        delay_ms(500);
    }
    return 0;
}
```

Expert Tips

- GDB: watch int_flag & 0x01.
- Clear in ISR.
- Pending flags.
- Priority.

Project 296: Configure a program to print UART parity errors.

Project Description: Write a C program to detect UART parity errors and print count.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Parity flag check.
- **Steps**:
    1. On RX, check UPE.
    2. Log if set.
    3. Clear.

C Code Implementation

```c
#include <stdio.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

char uart_rx(void) {
    char c = 'A';
    if (rand() % 20 == 0) {
        printf("Parity error on RX\n");
    }
    return c;
}

int main(void) {
    int parity_errors = 0;
    printf("Logging UART parity errors.\n");

    while (1) {
        uart_rx();
        parity_errors++;  // Simulate
        if (parity_errors % 5 == 0) printf("Parity errors: %d\n", parity_errors);
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- UCSRA & (1 << UPE).
- Even/odd parity config.
- Baud mismatch cause.
- Ignore or retry.

Project 297: Create a project to log timer overflow errors.

Project Description: Write a C program to detect unexpected timer overflows and log with context.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Expected vs actual count.
- **Steps**:
    1. Set expected overflows.
    2. Count actual.
    3. If mismatch, log.
    4. Print.

C Code Implementation

```c
#include <stdio.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}
```

```c
int main(void) {
    int expected_overflows = 10;
    int actual = 0;
    printf("Timer overflow error logging.\n");

    while (1) {
        actual++;  // Simulate overflow
        if (actual > expected_overflows) {
            printf("Overflow error: Expected %d, actual %d\n", expected_overflows, actual);
            actual = 0;
        }
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- Prescaler misconfig.
- Clock source.
- Interrupt missed.
- Reset counter.

Project 298: Implement a project to use a debugger to trace memory writes.

Project Description: Write a C program writing to array; comment for memory write breakpoint.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Array write with watch.
- **Steps**:
    1. Declare array.
    2. Write in loop.
    3. Comment breakpoint.

C Code Implementation

```c
#include <stdio.h>

char memory_array[10] = {0};  // Watch writes to this

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    printf("Trace memory writes in debugger.\n");

    for (int i = 0; i < 10; i++) {
        memory_array[i] = i;  // Breakpoint: Memory write to memory_array[i]
        printf("Wrote %d to index %d\n", i, i);
        delay_ms(100);
    }
    return 0;
}
```

Expert Tips

- GDB: watch memory_array[5].
- Hardware breakpoint on address range.
- Conditional write.
- Valgrind for leaks.

Project 299: Write a program to print debugging messages for a state machine.

Project Description: Write a C program with state machine logging entry, exit, transitions.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Verbose state logging.
- **Steps**:
    1. Enum states.
    2. Log entry/exit/transitions.
    3. Print to console.

C Code Implementation

```c
#include <stdio.h>

enum State { S_IDLE, S_ACTIVE, S_ERROR };

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int main(void) {
    enum State current = S_IDLE;
    enum State previous = S_IDLE;
    printf("State machine debugging.\n");

    while (1) {
        previous = current;
        // Simulate transition
        if (current == S_IDLE) current = S_ACTIVE;
        else if (current == S_ACTIVE) current = S_ERROR;
        else current = S_IDLE;

        if (current != previous) {
            printf("Transition: %d -> %d\n", previous, current);
        }
        printf("Entry %d\n", current);  // Entry log
        delay_ms(1000);
        printf("Exit %d\n", current);  // Exit log
    }
    return 0;
}
```

Expert Tips

- Log with timestamp.
- State duration.
- Invalid transitions.

- Finite state diagram.

Project 300: Create a project to log communication errors for a UART-based protocol.

Project Description: Write a C program implementing simple UART protocol; log checksum, timeout errors.

Way to Solve (Algorithm Used, Steps)

- **Algorithm Used**: Protocol validation.
- **Steps**:
    1. Receive packet (header, data, checksum).
    2. Check checksum.
    3. If fail, log error.
    4. Print.

C Code Implementation

```
#include <stdio.h>
#include <string.h>

void delay_ms(int ms) {
    volatile unsigned long i;
    for (i = 0; i < ms * 1000UL; i++);
}

int receive_packet(char* packet, int len) {
    strcpy(packet, "\xAA\x01\x02\x03");  // Simulate, checksum wrong
    unsigned char calc_checksum = 0;
    for (int i = 1; i < 3; i++) calc_checksum += (unsigned char)packet[i];
    unsigned char rx_checksum = (unsigned char)packet[3];
    if (calc_checksum != rx_checksum) {
        printf("Protocol error: Checksum mismatch (calc 0x%02X, rx 0x%02X)\n", calc_checksum,
rx_checksum);
        return -1;
    }
    return 0;
}

int main(void) {
    char packet[4];
    printf("UART protocol error logging.\n");

    while (1) {
        receive_packet(packet, 4);
        delay_ms(1000);
    }
    return 0;
}
```

Expert Tips

- CRC instead of sum.
- Sequence numbers.
- ACK/NAK.
- Timeout retry.